Laporan Tugas Besar 1

IF3070 Dasar Inteligensi Artifisial



Oleh

Kelompok 03 - rata kiri

M Rafi' Abdurrahman	18224004
Muhammad Arkan Dhaifullah	18224116
Rachel Elnora Mannuelle Sidanta	18224125

Tautan *repository* GitHub:

https://github.com/abdurrahman-rafi/IF3070_G03_ratakiri

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO & INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

Daftar Isi **2**
1. Deskripsi Persoalan **11**
1.1. Ringkasan Masalah 11
1.2. Variabel dan Representasinya 11
1.3. Objective Function 12
1.4. Constraint 13
1.5. State dan Move 13
2. Pembahasan **14**
2.1. Implementasi Algoritma Local Search 14
2.1.1. Komponen Umum 14
2.1.2. Hill-Climbing 16
2.1.2.1. Steepest Ascent Hill-Climbing 20
2.1.2.2. Stochastic Hill-Climbing 20
2.1.2.3. Hill-Climbing with Sideways Move 21
2.1.2.4. Random Restart Hill-Climbing 21
2.1.3. Simulated Annealing 21
2.1.4. Genetic Algorithm 25
2.2. Hasil Eksperimen 29
2.2.1. Steepest Ascent Hill-Climbing 31
2.2.1.1. instance_example 31
2.2.1.2. instance_multiship_hard 32
2.2.1.3. instance_multiship 36
2.2.1.4. instance_small 39
2.2.1.5. instance_tight_space 40
2.2.1.6. instance_tight_weight 42
2.2.2. Stochastic Hill-Climbing 44
2.2.2.1. instance_example 44
2.2.2.2. instance_multiship_hard 46
2.2.2.3. instance_multiship 49
2.2.2.4. instance_small 52
2.2.2.5. instance_tight_space 53
2.2.2.6. instance_tight_weight 55
2.2.3. Hill-Climbing with Sideways Move 57

2.2.3.1. instance_example..... 57
2.2.3.2. instance_multiship_hard.....59
2.2.3.3. instance_multiship.....62
2.2.3.4. instance_small.....65
2.2.3.5. instance_tight_space..... 67
2.2.3.6. instance_tight_weight.....68
2.2.4. Random Restart Hill-Climbing.....70
2.2.4.1. instance_example..... 70
2.2.4.2. instance_multiship_hard.....72
2.2.4.3. instance_multiship.....75
2.2.4.4. instance_small.....78
2.2.4.5. instance_tight_space..... 80
2.2.4.6. instance_tight_weight.....81
2.2.5. Simulated Annealing.....83
2.2.5.1. instance_example..... 83
2.2.5.2. instance_multiship_hard.....85
2.2.5.3. instance_multiship.....88
2.2.5.4. instance_small.....91
2.2.5.5. instance_tight_space..... 93
2.2.5.6. instance_tight_weight.....94
2.2.6. Genetic Algorithm..... 96
2.2.6.1. instance_example..... 96
2.2.6.2. instance_multiship_hard.....98
2.2.6.3. instance_multiship.....101
2.2.6.4. instance_small..... 103
2.2.6.5. instance_tight_space.....105
2.2.6.6. instance_tight_weight.....108
2.3. Analisis.....110
3. Kesimpulan dan Saran.....114
3.1. Kesimpulan.....114
3.2. Saran.....114
4. Pembagian Tugas..... 116
5. Referensi..... 116
6. Lampiran..... 117

6.1. Form Penggunaan Generative AI..... 117

Daftar Gambar

Gambar 2.2.1.1.1. Steepest ascent, seed 945: objective 260 → 480; 5 iterasi.....32
Gambar 2.2.1.1.2. Steepest ascent, seed 946: objective 370 → 485; 3 iterasi.....33
Gambar 2.2.1.1.3. Steepest ascent, seed 947: objective 195 → 495; 5 iterasi.....33
Gambar 2.2.1.2.1. Steepest ascent, seed 945: objective 670 → 945; 8 iterasi.....35
Gambar 2.2.1.2.2. Steepest ascent, seed 946: objective 750 → 965; 10 iterasi..... 36
Gambar 2.2.1.2.3. Steepest ascent, seed 947: objective 745 → 955; 8 iterasi.....37
Gambar 2.2.1.3.1. Steepest ascent, seed 945: objective 400 → 590; 3 iterasi.....38
Gambar 2.2.1.3.2. Steepest ascent, seed 946: objective 250 → 590; 5 iterasi.....39
Gambar 2.2.1.3.3. Steepest ascent, seed 947: objective 355 → 590; 4 iterasi.....40
Gambar 2.2.1.4.1. Steepest ascent, seed 945: objective 145 → 185; 1 iterasi.....41
Gambar 2.2.1.4.2. Steepest ascent, seed 946: objective 170 → 195; 2 iterasi..... 41
Gambar 2.2.1.4.3. Steepest ascent, seed 947: objective 115 → 215; 2 iterasi.....41
Gambar 2.2.1.5.1. Steepest ascent, seed 945: objective 275 → 375; 4 iterasi.....42
Gambar 2.2.1.5.2. Steepest ascent, seed 946: objective 270 → 380; 4 iterasi.....43
Gambar 2.2.1.5.3. Steepest ascent, seed 947: objective 175 → 335; 3 iterasi.....43
Gambar 2.2.1.6.1. Steepest ascent, seed 945: objective 245 → 300; 2 iterasi.....44
Gambar 2.2.1.6.2. Steepest ascent, seed 946: objective 240 → 310; 2 iterasi.....44
Gambar 2.2.1.6.3. Steepest ascent, seed 947: objective 185 → 290; 3 iterasi.....45
Gambar 2.2.2.1.1. Stochastic, seed 945: objective 260 → 480; 5 iterasi.....46
Gambar 2.2.2.1.2. Stochastic, seed 946: objective 370 → 480; 3 iterasi.....46
Gambar 2.2.2.1.3. Stochastic, seed 947: objective 195 → 480; 7 iterasi.....47
Gambar 2.2.2.2.1. Stochastic, seed 945: objective 670 → 950; 18 iterasi.....48
Gambar 2.2.2.2.2. Stochastic, seed 946: objective 750 → 935; 11 iterasi.....49
Gambar 2.2.2.2.3. Stochastic, seed 947: objective 745 → 955; 9 iterasi.....50
Gambar 2.2.2.3.1. Stochastic, seed 945: objective 400 → 480; 3 iterasi.....51
Gambar 2.2.2.3.2. Stochastic, seed 946: objective 250 → 590; 5 iterasi.....52
Gambar 2.2.2.3.3. Stochastic, seed 947: objective 355 → 590; 5 iterasi.....53
Gambar 2.2.2.4.1. Stochastic, seed 945: objective 145 → 185; 1 iterasi.....54
Gambar 2.2.2.4.2. Stochastic, seed 946: objective 170 → 195; 2 iterasi.....54
Gambar 2.2.2.4.3. Stochastic, seed 947: objective 115 → 185; 3 iterasi.....54
Gambar 2.2.2.5.1. Stochastic, seed 945: objective 275 → 375; 4 iterasi.....55
Gambar 2.2.2.5.2. Stochastic, seed 946: objective 270 → 335; 2 iterasi.....56
Gambar 2.2.2.5.3. Stochastic, seed 947: objective 175 → 305; 3 iterasi.....56

Gambar 2.2.2.6.1. Stochastic, seed 945: objective 245 → 300; 4 iterasi..... 57
Gambar 2.2.2.6.2. Stochastic, seed 946: objective 240 → 315; 5 iterasi..... 57
Gambar 2.2.2.6.3. Stochastic, seed 947: objective 185 → 315; 6 iterasi..... 58
Gambar 2.2.3.1.1. Sideways move, seed 945: objective 260 → 480; 105 iterasi; 100 sideways..... 59
Gambar 2.2.3.1.2. Sideways move, seed 946: objective 370 → 485; 103 iterasi; 100 sideways..... 59
Gambar 2.2.3.1.3. Sideways move, seed 947: objective 195 → 495; 105 iterasi; 100 sideways..... 60
Gambar 2.2.3.2.1. Sideways move, seed 945: objective 670 → 1015; 163 iterasi; 147 sideways..... 61
Gambar 2.2.3.2.2. Sideways move, seed 946: objective 750 → 1000; 192 iterasi; 177 sideways..... 62
Gambar 2.2.3.2.3. Sideways move, seed 947: objective 745 → 1045; 258 iterasi; 243 sideways..... 63
Gambar 2.2.3.3.1. Sideways move, seed 945: objective 400 → 590; 103 iterasi; 100 sideways..... 64
Gambar 2.2.3.3.2. Sideways move, seed 946: objective 250 → 590; 105 iterasi; 100 sideways..... 65
Gambar 2.2.3.3.3. Sideways move, seed 947: objective 355 → 590; 104 iterasi; 100 sideways..... 66
Gambar 2.2.3.4.1. Sideways move, seed 945: objective 145 → 200; 131 iterasi; 129 sideways..... 67
Gambar 2.2.3.4.2. Sideways move, seed 946: objective 170 → 200; 104 iterasi; 101 sideways..... 67
Gambar 2.2.3.4.3. Sideways move, seed 947: objective 115 → 215; 102 iterasi; 100 sideways..... 67
Gambar 2.2.3.5.1. Sideways move, seed 945: objective 275 → 440; 126 iterasi; 118 sideways..... 68
Gambar 2.2.3.5.2. Sideways move, seed 946: objective 270 → 420; 112 iterasi; 105 sideways..... 69
Gambar 2.2.3.5.3. Sideways move, seed 947: objective 175 → 445; 154 iterasi; 145 sideways..... 69
Gambar 2.2.3.6.1. Sideways move, seed 945: objective 245 → 315; 105 iterasi; 102 sideways..... 70
Gambar 2.2.3.6.2. Sideways move, seed 946: objective 240 → 310; 102 iterasi; 100 sideways..... 70

Gambar 2.2.3.6.3. Sideways move, seed 947: objective 185 → 305; 105 iterasi; 101 sideways..... 71
Gambar 2.2.4.1.1. Random restart, seed 945: objective 260 → 495; 42 iterasi; 10 restart..... 72
Gambar 2.2.4.1.2. Random restart, seed 946: objective 370 → 495; 40 iterasi; 10 restart..... 72
Gambar 2.2.4.1.3. Random restart, seed 947: objective 195 → 495; 41 iterasi; 10 restart..... 73
Gambar 2.2.4.2.1. Random restart, seed 945: objective 670 → 990; 90 iterasi; 10 restart..... 74
Gambar 2.2.4.2.2. Random restart, seed 946: objective 750 → 975; 80 iterasi; 10 restart..... 75
Gambar 2.2.4.2.3. Random restart, seed 947: objective 745 → 1000; 98 iterasi; 10 restart..... 76
Gambar 2.2.4.3.1. Random restart, seed 945: objective 400 → 590; 44 iterasi; 10 restart..... 77
Gambar 2.2.4.3.2. Random restart, seed 946: objective 250 → 590; 46 iterasi; 10 restart..... 78
Gambar 2.2.4.3.3. Random restart, seed 947: objective 355 → 590; 48 iterasi; 10 restart..... 79
Gambar 2.2.4.4.1. Random restart, seed 945: objective 145 → 200; 27 iterasi; 10 restart..... 80
Gambar 2.2.4.4.2. Random restart, seed 946: objective 170 → 200; 29 iterasi; 10 restart..... 80
Gambar 2.2.4.4.3. Random restart, seed 947: objective 115 → 215; 25 iterasi; 10 restart..... 80
Gambar 2.2.4.5.1. Random restart, seed 945: objective 275 → 395; 33 iterasi; 10 restart..... 81
Gambar 2.2.4.5.2. Random restart, seed 946: objective 270 → 385; 40 iterasi; 10 restart..... 82
Gambar 2.2.4.5.3. Random restart, seed 947: objective 175 → 440; 39 iterasi; 10 restart..... 82
Gambar 2.2.4.6.1. Random restart, seed 945: objective 245 → 315; 23 iterasi; 10 restart..... 83
Gambar 2.2.4.6.2. Random restart, seed 946: objective 240 → 315; 19 iterasi; 10 restart..... 83
Gambar 2.2.4.6.3. Random restart, seed 947: objective 185 → 315; 26 iterasi; 10

restart..... 84
Gambar 2.2.5.1.1. State awal, solusi valid terbaik, dan plot SA pada instance_example, seed 945: objective 260 menjadi 260; stuck 1 episode..... 85
Gambar 2.2.5.1.2. State awal, solusi valid terbaik, dan plot SA pada instance_example, seed 946: objective 370 menjadi 480; stuck 2 episode..... 85
Gambar 2.2.5.1.3. State awal, solusi valid terbaik, dan plot SA pada instance_example, seed 947: objective 195 menjadi 300; stuck 1 episode..... 86
Gambar 2.2.5.2.1. State awal, solusi valid terbaik, dan plot SA pada instance_multiship_hard, seed 945: objective 670 menjadi 890; stuck 9 episode.....87
Gambar 2.2.5.2.2. State awal, solusi valid terbaik, dan plot SA pada instance_multiship_hard, seed 946: objective 750 menjadi 885; stuck 5 episode.....88
Gambar 2.2.5.2.3. State awal, solusi valid terbaik, dan plot SA pada instance_multiship_hard, seed 947: objective 745 menjadi 870; stuck 4 episode.....89
Gambar 2.2.5.3.1. State awal, solusi valid terbaik, dan plot SA pada instance_multiship, seed 945: objective 400 menjadi 400; stuck 1 episode..... 90
Gambar 2.2.5.3.2. State awal, solusi valid terbaik, dan plot SA pada instance_multiship, seed 946: objective 250 menjadi 525; stuck 3 episode..... 91
Gambar 2.2.5.3.3. State awal, solusi valid terbaik, dan plot SA pada instance_multiship, seed 947: objective 355 menjadi 590; stuck 3 episode..... 92
Gambar 2.2.5.4.1. State awal, solusi valid terbaik, dan plot SA pada instance_small, seed 945: objective 145 menjadi 145; stuck 1 episode..... 93
Gambar 2.2.5.4.2. State awal, solusi valid terbaik, dan plot SA pada instance_small, seed 946: objective 170 menjadi 185; stuck 1 episode..... 93
Gambar 2.2.5.4.3. State awal, solusi valid terbaik, dan plot SA pada instance_small, seed 947: objective 115 menjadi 115; stuck 1 episode..... 93
Gambar 2.2.5.5.1. State awal, solusi valid terbaik, dan plot SA pada instance_tight_space, seed 945: objective 275 menjadi 275; stuck 1 episode..... 94
Gambar 2.2.5.5.2. State awal, solusi valid terbaik, dan plot SA pada instance_tight_space, seed 946: objective 270 menjadi 270; stuck 1 episode..... 95
Gambar 2.2.5.5.3. State awal, solusi valid terbaik, dan plot SA pada instance_tight_space, seed 947: objective 175 menjadi 175; stuck 1 episode..... 95
Gambar 2.2.5.6.1. State awal, solusi valid terbaik, dan plot SA pada instance_tight_weight, seed 945: objective 245 menjadi 250; stuck 1 episode..... 96
Gambar 2.2.5.6.2. State awal, solusi valid terbaik, dan plot SA pada instance_tight_weight, seed 946: objective 240 menjadi 280; stuck 1 episode..... 96
Gambar 2.2.5.6.3. State awal, solusi valid terbaik, dan plot SA pada instance_tight_weight, seed 947: objective 185 menjadi 185; stuck 1 episode..... 97

Daftar Tabel

Tabel 2.2.1.1. instance_example Steepest Ascent Hill-Climbing..... 32
Tabel 2.2.1.2. instance_multiship_hard Steepest Ascent Hill-Climbing..... 34
Tabel 2.2.1.3. instance_multiship Steepest Ascent Hill-Climbing.....38
Tabel 2.2.1.4. instance_small Steepest Ascent Hill-Climbing..... 40
Tabel 2.2.1.5. instance_tight_space Steepest Ascent Hill-Climbing.....42
Tabel 2.2.1.6. instance_tight_weight Steepest Ascent Hill-Climbing.....43
Tabel 2.2.2.1. instance_example Stochastic Hill-Climbing.....45
Tabel 2.2.2.2. instance_multiship_hard Stochastic Hill-Climbing.....47
Tabel 2.2.2.3. instance_multiship Stochastic Hill-Climbing..... 51
Tabel 2.2.2.4. instance_small Stochastic Hill-Climbing.....53
Tabel 2.2.2.5. instance_tight_space Stochastic Hill-Climbing.....55
Tabel 2.2.2.6. instance_tight_weight Stochastic Hill-Climbing..... 56
Tabel 2.2.3.1. instance_example Hill-Climbing with Sideways Move.....58
Tabel 2.2.3.2. instance_multiship_hard Hill-Climbing with Sideways Move.....60
Tabel 2.2.3.3. instance_multiship Hill-Climbing with Sideways Move..... 64
Tabel 2.2.3.4. instance_small Hill-Climbing with Sideways Move.....66
Tabel 2.2.3.5. instance_tight_space Hill-Climbing with Sideways Move.....68
Tabel 2.2.3.6. instance_tight_weight Hill-Climbing with Sideways Move..... 69
Tabel 2.2.4.1. instance_example Random Restart Hill-Climbing..... 71
Tabel 2.2.4.2. instance_multiship_hard Random Restart Hill-Climbing..... 73
Tabel 2.2.4.3. instance_multiship Random Restart Hill-Climbing.....77
Tabel 2.2.4.4. instance_small Random Restart Hill-Climbing..... 79
Tabel 2.2.4.5. instance_tight_space Random Restart Hill-Climbing..... 81
Tabel 2.2.4.6. instance_tight_weight Random Restart Hill-Climbing.....82
Tabel 2.2.5.1. instance_example Simulated Annealing..... 84
Tabel 2.2.5.2. instance_multiship_hard Simulated Annealing..... 86
Tabel 2.2.5.3. instance_multiship Simulated Annealing.....90
Tabel 2.2.5.4. instance_small Simulated Annealing..... 92
Tabel 2.2.5.5. instance_tight_space Simulated Annealing..... 94
Tabel 2.2.5.6. instance_tight_weight Simulated Annealing.....96
Tabel 2.2.6.1.1. instance_example Genetic Algorithm, Percobaan 1.....97
Tabel 2.2.6.1.1b. Ringkasan Percobaan 1 instance_example.....98
Tabel 2.2.6.1.2. instance_example Genetic Algorithm, Percobaan 2.....98

Tabel 2.2.6.1.2b. Ringkasan Percobaan 2 instance_example.....99
Tabel 2.2.6.2.1. instance_multiship_hard Genetic Algorithm, Percobaan 1.....100
Tabel 2.2.6.2.1b. Ringkasan Percobaan 1 instance_multiship_hard..... 100
Tabel 2.2.6.2.2. instance_multiship_hard Genetic Algorithm, Percobaan 2.....101
Tabel 2.2.6.2.2b. Ringkasan Percobaan 2 instance_multiship_hard..... 101
Tabel 2.2.6.3.1. instance_multiship Genetic Algorithm, Percobaan 1..... 102
Tabel 2.2.6.3.1b. Ringkasan Percobaan 1 instance_multiship.....102
Tabel 2.2.6.3.2. instance_multiship Genetic Algorithm, Percobaan 2..... 103
Tabel 2.2.6.3.2b. Ringkasan Percobaan 2 instance_multiship.....104
Tabel 2.2.6.4.1. instance_small Genetic Algorithm, Percobaan 1.....104
Tabel 2.2.6.4.1b. Ringkasan Percobaan 1 instance_small..... 105
Tabel 2.2.6.4.2. instance_small Genetic Algorithm, Percobaan 2.....105
Tabel 2.2.6.4.2b. Ringkasan Percobaan 2 instance_small..... 106
Tabel 2.2.6.5.1. instance_tight_space Genetic Algorithm, Percobaan 1..... 106
Tabel 2.2.6.5.1b. Ringkasan Percobaan 1 instance_tight_space..... 107
Tabel 2.2.6.5.2. instance_tight_space Genetic Algorithm, Percobaan 2..... 108
Tabel 2.2.6.5.2b. Ringkasan Percobaan 2 instance_tight_space..... 108
Tabel 2.2.6.6.1. instance_tight_weight Genetic Algorithm, Percobaan 1..... 109
Tabel 2.2.6.6.1b. Ringkasan Percobaan 1 instance_tight_weight.....109
Tabel 2.2.6.6.2. instance_tight_weight Genetic Algorithm, Percobaan 2..... 110
Tabel 2.2.6.6.2b. Ringkasan Percobaan 2 instance_tight_weight.....111

1. Deskripsi Persoalan

1.1. Ringkasan Masalah

PT Intim Line mengangkut kendaraan dari beberapa *dealer* dalam satu keberangkatan. Tiap kendaraan punya ukuran, berat, tarif pengiriman (*shipping fee*, juta rupiah), dan ETA. Sistem yang dibangun memutuskan tiga hal: kendaraan mana yang diangkut, di koordinat mana kendaraan itu diletakkan pada geladak, dan dengan orientasi apa (*Horizontal* atau *Vertical*). Tujuannya memaksimalkan total *shipping fee* dari kendaraan yang terangkut, tanpa ada kendaraan yang saling bertumpuk, keluar batas geladak, atau melebihi kapasitas berat kapal.

Eksperimen dijalankan pada enam *instance* di folder data (lihat tabel instance pada bagian Hasil Eksperimen). Instance utama adalah data/instance_example.json (Instance Example) dengan 10 kendaraan dan satu kapal berukuran 10x20 dengan kapasitas 60. Total berat seluruh kendaraan 73 unit yang melebihi kapasitas kapal, sedangkan total luasnya 151 dari 200 sel geladak, jadi pembatas utamanya adalah kapasitas berat. Lima instance lain menguji kondisi yang berbeda, yaitu ruang geladak yang terbatas, kapasitas berat yang ketat, serta dua dan tiga kapal.

1.2. Variabel dan Representasinya

Entitas	Atribut	Representasi di Program
Vehicle	id	Kunci *dict* `vehicles`, *string* (misal “V01”), juga kunci `placement`. Id tidak boleh duplikat.
	w	`vehicles[id]["w"]`, *int* (satuan *grid*)
	l	`vehicles[id]["l"]`, *int*
	fee	`vehicles[id]["fee"]`, *int* (juta rupiah)
	weight	`vehicles[id]["weight"]`, *int* (*unit*)
	eta	`vehicles[id]["eta"]`, *int* (hari). Dimuat, tetapi belum dipakai oleh *objective function*.

	orientasi dan posisi	`placement[id] = {"ship", "x", "y", "orient"}`. `orient` bernilai "H" (ukuran w x l) atau "V" (ukuran ditukar jadi l x w). x, y adalah sel kiri bawah, bilangan bulat mulai dari 1.
	di luar kapal	`placement[id]["ship"] = OUTSIDE` (konstanta -1) dan `x = y = None`. Orientasi tetap disimpan.
Ships	id	Kunci *dict* `ships`, *int*. Tidak boleh sama dengan `OUTSIDE` atau duplikat.
	w, l	`ships[id]["w"]`, `ships[id]["l"]`
	capacity	`ships[id]["capacity"]`. Beban kapal tidak disimpan, dihitung dari `placement` oleh `ship_load(ship)`.

State direpresentasikan oleh kelas `DockState` (`src/model/state.py`). Untuk visualisasi, `build_grids()` mengubah state menjadi *grid* numpy per kapal (sel kosong bernilai -1, sel terisi berisi indeks kendaraan).

1.3. *Objective Function*

Objective yang dimaksimalkan semua algoritma adalah total *shipping fee* kendaraan yang berada di kapal, dan bernilai 0 bila ada constraint yang dilanggar:

$$f(s) = \begin{cases} \sum_{v \in P(s)} \text{fee}_v & \text{jika } s \text{ feasible} \\ 0 & \text{jika } s \text{ melanggar constraint} \end{cases}$$

Dengan P(s) adalah himpunan kendaraan yang ditempatkan di salah satu kapal pada *state* s (`ship != OUTSIDE`). Fungsinya adalah `evaluate` di `src/model/objective.py`, dengan nilai 0 untuk *state* tidak *feasible*. Akibatnya semua *state* yang melanggar sama nilainya, sehingga algoritma tidak punya gradien untuk keluar dari wilayah tidak *feasible*. Itu sebabnya inisialisasi dan GA dibuat selalu menghasilkan state *feasible* (lihat 2.1.4).

1.4. *Constraint*

Tiga *constraint* dari spesifikasi dimodelkan sebagai tiga fungsi pemeriksa di `src/model/constraints.py`, lalu digabung oleh `is_feasible`:

- **Batas geladak** (`has_out_of_bounds`): sel kiri bawah minimal (1, 1) dan sel kanan atas `x + w - 1 <= W`, `y + l - 1 <= L`, dengan w dan l sesuai orientasi (fungsi `footprint`).
- **Tidak *overlap*** (`has_overlap`): tiap pasangan kendaraan di kapal yang sama dicek sekali dengan uji irisan dua persegi panjang (`rectangles_overlap`) di sumbu x dan y sekaligus. Dua kendaraan yang hanya bersentuhan sisi tidak dihitung *overlap*.
- **Kapasitas** (`is_overweight`): jumlah berat kendaraan di tiap kapal tidak boleh melebihi `capacity` kapal itu.

Pelanggaran ditangani dengan tidak membuang *state*: *neighbor* yang melanggar tetap dibangkitkan dan bernilai 0 di `evaluate`. Untuk membangun state *feasible* tersedia `can_place`, yang mengecek satu kendaraan di satu posisi terhadap ketiga *constraint* dengan menganggap posisi lama kendaraan itu kosong. `can_place` dipakai oleh inisialisasi *state*, *hill-climbing* (`neighbor_value`), dan fungsi *repair* pada GA.

1.5. *State* dan *Move*

***Initial state*:** `DockState.initial_state` membuat *state* acak yang selalu *feasible*. Tiap kendaraan diberi orientasi acak dan kapal acak (atau OUTSIDE). Bila ukurannya tidak muat, orientasi dibalik. Lalu koordinat acak dicoba sampai 50 kali dengan `can_place`. Kendaraan yang gagal tetap OUTSIDE.

***Final state*:** Pemetaan semua kendaraan (di dalam dan di luar kapal) beserta nilai *objective*-nya. `SearchRecorder.finish` menyimpannya ke JSON (*state* awal, *state* akhir, *state feasible* terbaik, riwayat *objective*, iterasi, durasi), dan `build_grids` menyiapkan *grid* geladak untuk digambar.

***Move*:** Tiga *move* ada di `src/moves/moves.py`:

Move	Fungsi	Cara Kerja
Tukar dua kendaraan	`swap_move`	Menukar (*ship*, x, y) dua kendaraan, orientasi tetap milik masing-masing. Bila salah satu OUTSIDE, yang di luar masuk dan yang di dalam keluar.
Pindah koordinat	`place_move`	Memindahkan satu kendaraan ke (*ship*, x, y) lain: dari luar ke dalam, antar kapal, atau di kapal yang sama. Posisi yang akan keluar geladak tidak pernah dibuat.
Ubah orientasi	`rotate_move`	Membalik H dan V, titik kiri bawah tetap. Kendaraan persegi (w = l) dilewati karena rotasinya tidak mengubah apa pun.

`random_neighbor` memilih jenis *move* dengan bobot `swap 0.4`, `relocate 0.4`, `rotate 0.2`. `all_neighbors` membangkitkan seluruh *neighbor* sebagai generator untuk algoritma yang butuh semua *neighbor*.

2. Pembahasan

2.1. Implementasi Algoritma *Local Search*

2.1.1. Komponen Umum

File	Fungsi/Kelas	Peran
`src/model/instance.py`	`load_instance`, `footprint`, `other_orient`, `fits`	Membaca *instance* JSON dan memvalidasi id. Menghitung ukuran kendaraan menurut orientasi, membalik orientasi, dan mengecek apakah kendaraan muat di geladak.
`src/model/state.`	`DockState`	Menyimpan `placement`. Metode: `initial_state` (*state* awal acak yang *feasible*), `copy` (menyalin

py		hanya placement), placed_vehicles, ship_load, random_neighbor, all_neighbors, to_dict, build_grids, to_chromosome, from_chromosome.
src/model/constraints.py	rectangles_overlap, can_place, has_out_of_bounds, has_overlap, is_overweight, is_feasible	Pengecekan *constraint* (lihat 1.4).
src/model/objective.py	total_fee, evaluate	Menghitung *objective* (lihat 1.3).
src/moves/moves.py	swap_move, place_move, rotate_move, random_neighbor, all_neighbors	Pembangkit *neighbor*
src/algorithms/base.py	SearchRecorder	Pencatat hasil yang dipakai semua algoritma: riwayat *objective* per iterasi, rata-rata populasi (GA), peluang penerimaan $e^{\frac{dE}{T}}$ (untuk SA), *state feasible* terbaik, durasi, dan *finish* yang menghasilkan *dict* siap disimpan ke JSON.
src/experime	run_repeated, save_result	Menjalankan satu algoritma n kali. *Run* ke-k memakai seed base_seed + k, sehingga

nts/runner.py		*state* awal dan hasilnya bisa diulang. Hasil disimpan di output/<instance>/<algoritma>/.
src/experiments/ga_experiments.py	run_all	Menjalankan dua skenario GA sesuai spesifikasi.

Semua *state* yang dibangkitkan oleh *initial state* dan oleh operator GA diperlakukan sebagai objek baru (copy), jadi *state* asal tidak berubah ketika *neighbor* dibuat. Data statis kendaraan dan kapal dibagi antar *state* sehingga penyalinan murah.

2.1.2. *Hill-Climbing*

Implementasi ada di src/algorithms/hill_climbing.py. Keempat varian memakai fungsi pendukung yang sama:

- scored_neighbors membangkitkan semua *neighbor* lewat all_neighbors (*swap*, pindah koordinat, dan ubah orientasi) dan memberi nilai pada tiap *neighbor*.
- neighbor_value tidak memanggil evaluate pada seluruh *state*. Fungsi ini hanya mengecek kendaraan yang berpindah dengan can_place, lalu menghitung total_fee. Hasilnya 0 bila melanggar *constraint*. Karena *state* asal selalu *feasible*, nilainya sama dengan evaluate(neighbor).
- best_neighbors mengembalikan semua *neighbor* dengan nilai tertinggi, sehingga pilihan di antara nilai yang sama bisa diacak.

- `climb` adalah salah satu lintasan *steepest ascent*. Fungsi ini dipakai oleh *Steepest Ascent*, *Sideways Move*, dan *Random Restart*.

Semua varian mencatat nilai tiap iterasi lewat `SearchRecorder`, jadi data untuk plot *objective* terhadap iterasi ada di JSON hasil.

```
from .base import SearchRecorder
from ..model.instance import OUTSIDE
from ..model.constraints import can_place
from ..model.objective import total_fee
from ..model.state import DockState


def neighbor_value(neighbor, moved_vehicles):
    for vid in moved_vehicles:
        p = neighbor.placement[vid]
        if p["ship"] != OUTSIDE and not can_place(neighbor,
vid, p["ship"], p["x"], p["y"], p["orient"]):
            return 0
    return total_fee(neighbor)


def scored_neighbors(state):
    for moved_vehicles, neighbor in state.all_neighbors():
        yield neighbor, neighbor_value(neighbor,
moved_vehicles)


def best_neighbors(state):
    best, best_value = [], None
    for neighbor, value in scored_neighbors(state):
        if best_value is None or value > best_value:
            best, best_value = [neighbor], value
        elif value == best_value:
            best.append(neighbor)
    return best, best_value


def climb(state, rng, max_iterations, max_sideways,
recorder):
```

```
    value = recorder.record(state)
    iterations = sideways = total_sideways = 0
    while iterations < max_iterations:
        candidates, candidate_value = best_neighbors(state)
        if not candidates or candidate_value < value:
            return state, value, iterations,
"local_maximum", total_sideways
        if candidate_value == value:
            if sideways == max_sideways:
                return state, value, iterations,
"max_sideways" if max_sideways else "local_maximum",
total_sideways
            sideways += 1
            total_sideways += 1
        else:
            sideways = 0
        state, value = rng.choice(candidates),
candidate_value
        iterations += 1
        recorder.record(state, value)
    return state, value, iterations, "max_iterations",
total_sideways


def steepest_ascent(initial_state, rng,
max_iterations=1000):
    recorder = SearchRecorder("steepest_ascent",
initial_state, {"max_iterations": max_iterations})
    state, _, iterations, stop_reason, _ =
climb(initial_state, rng, max_iterations, 0, recorder)
    return recorder.finish(state, iterations,
{"stop_reason": stop_reason})


def sideways_move(initial_state, rng, max_sideways=100,
max_iterations=1000):
    recorder = SearchRecorder("sideways_move",
initial_state,
                              {"max_sideways":
max_sideways, "max_iterations": max_iterations})
    state, _, iterations, stop_reason, total_sideways =
```

```
climb(initial_state, rng, max_iterations,

max_sideways, recorder)
    return recorder.finish(state, iterations,
{"stop_reason": stop_reason, "total_sideways":
total_sideways})


def stochastic(initial_state, rng, max_iterations=1000):
    recorder = SearchRecorder("stochastic", initial_state,
{"max_iterations": max_iterations})
    state, value = initial_state,
recorder.record(initial_state)
    iterations, stop_reason = 0, "max_iterations"
    while iterations < max_iterations:
        uphill = [(neighbor, neighbor_value) for neighbor,
neighbor_value in scored_neighbors(state)
                  if neighbor_value > value]
        if not uphill:
            stop_reason = "local_maximum"
            break
        state, value = rng.choice(uphill)
        iterations += 1
        recorder.record(state, value)
    return recorder.finish(state, iterations,
{"stop_reason": stop_reason})


def random_restart(initial_state, rng, max_restart=10,
max_iterations=1000):
    recorder = SearchRecorder("random_restart",
initial_state,
                              {"max_restart": max_restart,
"max_iterations": max_iterations})
    best_state, best_value = None, -1
    iterations_per_restart = []
    for restart in range(max_restart + 1):
        start = initial_state if restart == 0 else
DockState.initial_state(initial_state.vehicles,

initial_state.ships, rng)
```

```
        state, value, iterations, _, _ = climb(start, rng,
max_iterations, 0, recorder)
        iterations_per_restart.append(iterations)
        if value > best_value:
            best_state, best_value = state, value
    return recorder.finish(best_state,
sum(iterations_per_restart),
                           {"restarts": max_restart,
"iterations_per_restart": iterations_per_restart})
```

2.1.2.1. *Steepest Ascent Hill-Climbing*

Pada tiap iterasi, algoritma memeriksa seluruh *neighbor* dan berpindah ke *neighbor* dengan nilai tertinggi. Jika ada beberapa *neighbor* terbaik dengan nilai sama, satu dipilih secara acak.

Pencarian berhenti saat nilai *neighbor* terbaik tidak lebih besar dari nilai *state* sekarang (*local maximum*, termasuk jika sama) atau jika iterasi mencapai `max_iterations`. Parameter: `max_iterations` = 1000. Nilai yang dicatat adalah banyak iterasi sampai berhenti dan alasan berhenti (`stop_reason`).

2.1.2.2. *Stochastic Hill-Climbing*

Pada tiap iterasi, algoritma mengumpulkan semua *neighbor* yang nilainya lebih tinggi dari *state* sekarang (*uphill*), lalu memilih satu secara acak. Berbeda dari *steepest ascent*, *neighbor* yang dipilih belum tentu yang terbaik.

Pencarian berhenti saat tidak ada *neighbor uphill* (*local maximum*) atau saat iterasi mencapai `max_iterations`. Parameter: `max_iterations` = 1000. Nilai yang dicatat adalah banyak iterasi sampai berhenti dan alasan berhenti (`stop_reason`).

2.1.2.3. *Hill-Climbing with Sideways Move*

Cara kerjanya seperti *steepest ascent*, tetapi *neighbor* terbaik yang nilainya sama dengan *state* sekarang (*sideways move*) tetap diterima. Hitungan *sideways* berurutan kembali ke 0 setiap kali nilai naik.

Pencarian berhenti saat *sideways* berurutan mencapai `max_sideways` (`stop_reason = max_sideways`), saat *neighbor* terbaik lebih buruk dari *state* sekarang (*local maximum*), atau saat iterasi mencapai `max_iterations`. Parameter: `max_sideways` = 100 dan `max_iterations` = 1000. Nilai yang dicatat adalah banyak iterasi (termasuk langkah *sideways*), alasan berhenti (`stop_reason`), dan `total_sideways`.

2.1.2.4. *Random Restart Hill-Climbing*

Algoritma menjalankan *steepest ascent* dari *initial state*, lalu mengulanginya dari *state* acak *feasible* yang baru sebanyak `max_restart` kali, sehingga totalnya `max_restart + 1` lintasan. Tiap lintasan berhenti di *local maximum* atau di `max_iterations`.

Hasil akhirnya adalah *state* terbaik dari semua lintasan, Jika ada nilai yang sama, *state* yang ditemukan lebih dulu yang dipakai. Riwayat *objective* semua lintasan disambung dalam satu plot. Parameter: `max_restart` = 10 dan `max_iterations` = 1000 per lintasan. Nilai yang dicatat adalah banyak *restart*, banyak iterasi per *restart* (`iteration_per_restart`), dan total iterasi.

2.1.3. *Simulated Annealing*

Simulated Annealing diimplementasikan pada fungsi simulated_annealing di src/algorithms/simulated_annealing.py. Algoritma menyimpan state yang sedang dijelajahi (current) serta solusi valid terbaik yang pernah ditemukan (best_feasible). State

awal dibuat acak oleh komponen inisialisasi bersama, kemudian disalin agar susunan awal tetap tersedia sebagai pembanding.

Neighbor dibangkitkan melalui random_neighbor menggunakan langkah swap, relocate, atau rotate. Nilai evaluate sama dengan total ShippingFee apabila seluruh constraint terpenuhi, dan 0 apabila terdapat pelanggaran. State yang melanggar tetap dapat dijelajahi selama pencarian, tetapi solusi akhir yang dikembalikan adalah solusi valid terbaik.

Selisih nilai didefinisikan sebagai $\Delta E = f(s_{neighbor}) - f(s_{current})$. Karena objective dimaksimalkan, neighbor dengan $\Delta E \geq 0$ selalu diterima. Neighbor yang lebih buruk diterima dengan peluang $e^{(\Delta E/T)}$, yang dibandingkan dengan bilangan acak rng.random(). Jika ditolak, current dipertahankan dan rejected_count bertambah satu.

Penjadwalan suhu memakai geometric cooling: $T_{(t+1)} = \alpha T_t$. Parameter initial_temp menentukan suhu awal, cooling_rate menentukan α dengan $0 < \alpha < 1$, dan min_temp menentukan batas suhu. Default parameter adalah initial_temp=100, cooling_rate=0,99, min_temp=0,01, serta stagnation_limit=50. Pencarian berhenti ketika $T \leq$ min_temp atau tidak ada neighbor yang dapat dibangkitkan.

Stuck didefinisikan sebagai satu episode ketika nilai best feasible tidak meningkat selama N iterasi berturut-turut, dengan N=stagnation_limit. Saat mencapai N, stuck_count bertambah satu. Jika stagnasi berlanjut, episode yang sama tidak dihitung ulang. Peningkatan nilai terbaik mereset penghitung iterasi dan memungkinkan episode berikutnya. Kembali ke nilai terbaik lama setelah menjelajahi state buruk belum dianggap peningkatan baru.

Jumlah penolakan neighbor dicatat terpisah pada rejected_count. Stuck merupakan indikator stagnasi yang didefinisikan secara operasional, bukan bukti pasti local optimum, dan tidak menghentikan pencarian.

SearchRecorder mencatat state awal, riwayat nilai current, peluang penerimaan, best feasible, dan durasi pencarian. Grafik

berlabel e^(ΔE/T) menggunakan accept_probs: nilai e^(ΔE/T) untuk langkah memburuk dan 1 untuk langkah membaik atau setara. Hasil akhir adalah best_feasible, sedangkan extra.last_value menyimpan nilai current terakhir; karena itu, ujung kurva current dapat lebih rendah daripada objective akhir yang dilaporkan.

```
import math

from .base import SearchRecorder
from ..model.objective import evaluate


def simulated_annealing(initial_state, rng,
initial_temp=100.0,
                        cooling_rate=0.99, min_temp=0.01,
stagnation_limit=50):
    if not (math.isfinite(initial_temp) and
math.isfinite(min_temp)
            and 0 < min_temp < initial_temp):
        raise ValueError(
            "Suhu harus finite dan memenuhi 0 < min_temp <
initial_temp"
        )

    if not 0 < cooling_rate < 1:
        raise ValueError("cooling_rate harus antara 0 dan
1")

    if (isinstance(stagnation_limit, bool)
            or not isinstance(stagnation_limit, int) or
stagnation_limit < 1):
        raise ValueError("stagnation_limit harus bilangan
bulat positif")

    params = {
        "initial_temp": initial_temp,
        "cooling_rate": cooling_rate,
        "min_temp": min_temp,
        "stagnation_limit": stagnation_limit,
```

```
    }

    recorder = SearchRecorder(
        "simulated_annealing", initial_state, params
    )

    current = initial_state.copy()
    current_value = evaluate(current)
    temperature = initial_temp
    iterations = 0
    rejected_count = 0
    stuck_count = 0
    stagnation_iterations = 0

    recorder.record(current, current_value)

    while temperature > min_temp:
        neighbor = current.random_neighbor(rng)

        if neighbor is None:
            break

        neighbor_value = evaluate(neighbor)
        delta = neighbor_value - current_value
        probability = recorder.record_acceptance(delta,
temperature)

        if delta >= 0 or rng.random() < probability:
            current = neighbor
            current_value = neighbor_value
        else:
            rejected_count += 1

        iterations += 1
        previous_best = recorder.best_feasible_value
        recorder.record(current, current_value)
        if recorder.best_feasible_value > previous_best:
            stagnation_iterations = 0
        else:
            stagnation_iterations += 1
            if stagnation_iterations == stagnation_limit:
```

```
                stuck_count += 1
            temperature *= cooling_rate

    final_state = recorder.best_feasible
    if final_state is None:
        final_state = current

    return recorder.finish(
        final_state,
        iterations,
        extra={
            "last_value": current_value,
            "rejected_count": rejected_count,
            "stuck_count": stuck_count,
            "stagnation_iterations": stagnation_iterations,
        },
    )
```

2.1.4. *Genetic Algorithm*

Implementasi ada di `src/algorithms/genetic.py`. Seleksi menggunakan *roulette wheel*, jumlah individu `elite` 1, dan `mutation_rate` 0,1.

- **Kromosom:** Satu kromosom adalah *list* gen `{ship, x, y, orient}`, satu gen per kendaraan, urut sesuai `DockState.order` (id terurut), sehingga gen ke-i selalu milik kendaraan yang sama. Konversi lewat `to_chromosome` dan `from_chromosome`.
- ***Fitness*:** `evaluate(state)`: total *fee* bila *feasible*, 0 bila tidak.
- **Populasi awal:** *State* awal yang diberikan, ditambah `population_size - 1` *state* acak *feasible* dari `DockState.initial_state`.
- **Seleksi:** *Roulette wheel* (`roulette_select`): peluang sebuah individu terpilih sebanding dengan *fitness*-nya. *Wheel* diputar dengan bilangan acak r di [0, total *fitness*], lalu dipilih individu pertama yang jumlah kumulatif *fitness*-nya sudah mencapai r.

Bila total *fitness* 0 (semua individu bernilai 0), individu dipilih acak seragam.

- ***Crossover*:** Satu titik (`one_point_crossover`): gen [0, k) dari orang tua A dan [k, n) dari orang tua B, dengan k acak. Dilakukan dengan peluang `crossover_rate` (*default* 0,9). Bila tidak, anak adalah salinan orang tua A.
- ***Repair*:** Anak hasil *crossover* sering *overlap*, keluar geladak, atau kelebihan berat, padahal *state* seperti itu bernilai 0. Fungsi `repair` membangun ulang *state* dari semua kendaraan OUTSIDE, lalu menempatkan gen satu per satu sesuai urutan dengan `can_place`. Gen yang gagal membuat kendaraannya tetap OUTSIDE. Hasilnya selalu *feasible*.
- **Mutasi:** Dengan peluang `mutation_rate` (*default* 0,1), anak diubah oleh satu langkah `random_neighbor` (*swap*, pindah koordinat, atau ubah orientasi, dengan bobot 0,4/0,4/0,2), lalu di-*repair* lagi.
- ***Elitism*:** `elite` (*default* 1) individu terbaik generasi sekarang dibawa langsung ke generasi berikutnya. Akibatnya *objective* maksimum populasi tidak pernah turun.
- **Kriteria berhenti:** Jumlah generasi tetap (`iterations`). Satu iterasi pada laporan ini berarti satu generasi. Hasil akhir adalah individu terbaik pada generasi terakhir.
- **Pencatatan:** Tiap generasi (termasuk generasi 0) `SearchRecorder.record_population` mencatat nilai maksimum dan rata-rata populasi.

```
from .base import SearchRecorder
from ..model.instance import OUTSIDE
from ..model.objective import evaluate
from ..model.state import DockState
from ..model.constraints import can_place


def one_point_crossover(parent_a, parent_b, rng):
    k = rng.randrange(1, len(parent_a))
     return [dict(g) for g in parent_a[:k]] + [dict(g) for g
in parent_b[k:]]
```

```
def repair(template, chromosome):
    state = template.from_chromosome(chromosome)
    for vid in state.order:
        state.placement[vid] = {
            "ship": OUTSIDE,
            "x": None,
            "y": None,
            "orient": state.placement[vid]["orient"]}
    for vid, gene in zip(state.order, chromosome):
        if gene["ship"] == OUTSIDE:
            continue
          if can_place(state, vid, gene["ship"], gene["x"],
gene["y"], gene["orient"]):
            state.placement[vid] = dict(gene)
    return state


def roulette_select(values, rng):
    total = sum(values)
    if total <= 0:
        return rng.randrange(len(values))
    r = rng.uniform(0, total)
    cumulative = 0
    for i, value in enumerate(values):
        cumulative += value
        if cumulative >= r:
            return i
    return len(values) - 1


def mutate(state, rng):
    neighbor = state.random_neighbor(rng)
    return neighbor if neighbor is not None else state


def          genetic_algorithm(initial_state,          rng,
population_size=50, iterations=100,
                                       crossover_rate=0.9,
mutation_rate=0.1, elite=1):
```

```
    params = {"population_size": population_size,
              "iterations": iterations,
              "crossover_rate": crossover_rate,
              "mutation_rate": mutation_rate,
              "elite": elite}
          recorder   =   SearchRecorder("genetic_algorithm",
initial_state, params)
    elite = min(elite, population_size)

    population = [initial_state.copy()]
    while len(population) < population_size:

population.append(DockState.initial_state(initial_state.veh
icles, initial_state.ships, rng))
    values = [evaluate(s) for s in population]
    recorder.record_population(population, values)

    for _ in range(iterations):
         ranked = sorted(range(len(population)), key=lambda
i: values[i], reverse=True)
          next_population = [population[i].copy() for i in
ranked[:elite]]

        while len(next_population) < population_size:
              parent_a = population[roulette_select(values,
rng)]
              parent_b = population[roulette_select(values,
rng)]
                    if rng.random() < crossover_rate and
len(parent_a.order) > 1:
                                                chromosome   =
one_point_crossover(parent_a.to_chromosome(),
parent_b.to_chromosome(), rng)
            else:
                chromosome = parent_a.to_chromosome()
            child = repair(initial_state, chromosome)
            if rng.random() < mutation_rate:
                 child = repair(initial_state, mutate(child,
rng).to_chromosome())
            next_population.append(child)
```

```
        population = next_population
        values = [evaluate(s) for s in population]
        recorder.record_population(population, values)

        best = max(range(len(population)), key=lambda i:
values[i])
    return recorder.finish(population[best], iterations,
                                    extra={"population_size":
population_size})
```

2.2. Hasil Eksperimen

Eksperimen dijalankan pada enam instance di folder data (Tabel instance di bawah). Semua algoritma dijalankan dengan perintah python -m src.main --instance data/<nama file>.json --all, yang menjalankan keempat varian Hill-Climbing dan SA masing-masing tiga kali serta kedua skenario eksperimen GA. Run ke-1 sampai ke-3 memakai seed 945, 946, dan 947, sehingga state awal tiap run sama untuk semua algoritma. Hasil tiap run (JSON dan gambar state awal, state akhir, serta plot) tersimpan di output/<nama file>/<algoritma>/ pada rilis v1.0, beserta ringkasan semua run di summary.txt. Nilai objective, iterasi, dan state dapat direproduksi persis dengan seed dan kode yang sama. Durasi diukur dengan time.perf_counter pada perangkat masing-masing anggota, sehingga angka durasi dapat berbeda dari file di output, tetapi urutan relatifnya sama.

Untuk Genetic Algorithm, parameter yang dipertahankan adalah crossover_rate 0,9, mutation_rate 0,1, dan elite 1, dengan seleksi roulette wheel. Parameter yang divariasikan adalah ukuran populasi dan jumlah iterasi dalam dua skenario: populasi tetap 50 dengan iterasi 50, 100, dan 200, serta iterasi tetap 100 dengan populasi 20, 50, dan 100. Konfigurasi populasi 50 dan 100 iterasi dipakai bersama oleh kedua skenario sehingga hanya dijalankan sekali, jadi tiap instance menghasilkan 15 run GA. Satu iterasi GA berarti satu generasi.

Nama di Laporan	***File***	***Ship(s)***	***Vehicles***	***Characteristic***

Instance Example	`data/instance_example.json`	1 (10x20 kapasitas 60)	10	dibatasi berat, optimum 495
Multiship+	`data/instance_multiship_hard.json`	3 (10x20 kapasitas 70, 8x12 kapasitas 40, dan 6x10 kapasitas 30)	30	tidak semua kendaraan muat
Multiship	`data/instance_multiship.json`	2 (10x20 kapasitas 60 dan 8x12 kapasitas 35)	10	semua kendaraan muat, optimum 590
Small	`data/instance_small.json`	1 (8x10 kapasitas 30)	8	dibatasi berat, optimum 215
Limited Space	`data/instance_tight_space.json`	1 (8x12 kapasitas 200)	14	total luas 171 untuk 96 sel, dibatasi ruang
Limited Weight	`data/instance_tight_weight.json`	1 (12x16 kapasitas 45)	14	total berat 147 untuk kapasitas 45, optimum 315

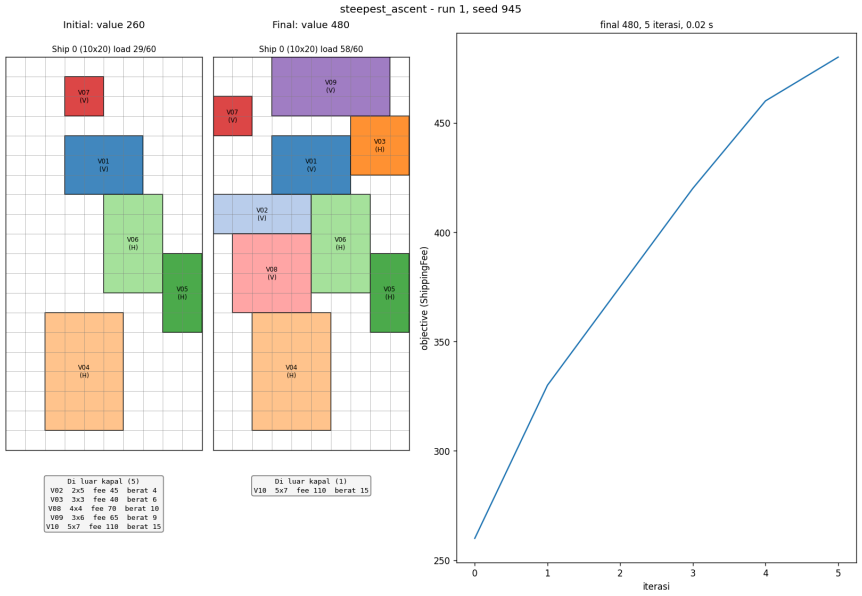
2.2.1. *Steepest Ascent Hill-Climbing*

2.2.1.1. instance_example

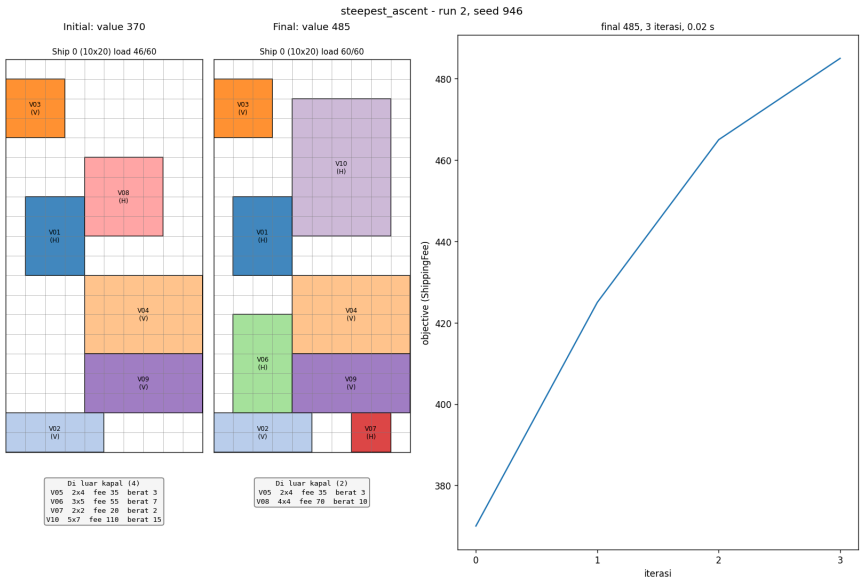
Hasil di bawah berasal dari output/instance_example/steepest_ascent/.

Tabel 2.2.1.1. instance_example *Steepest Ascent Hill-Climbing*

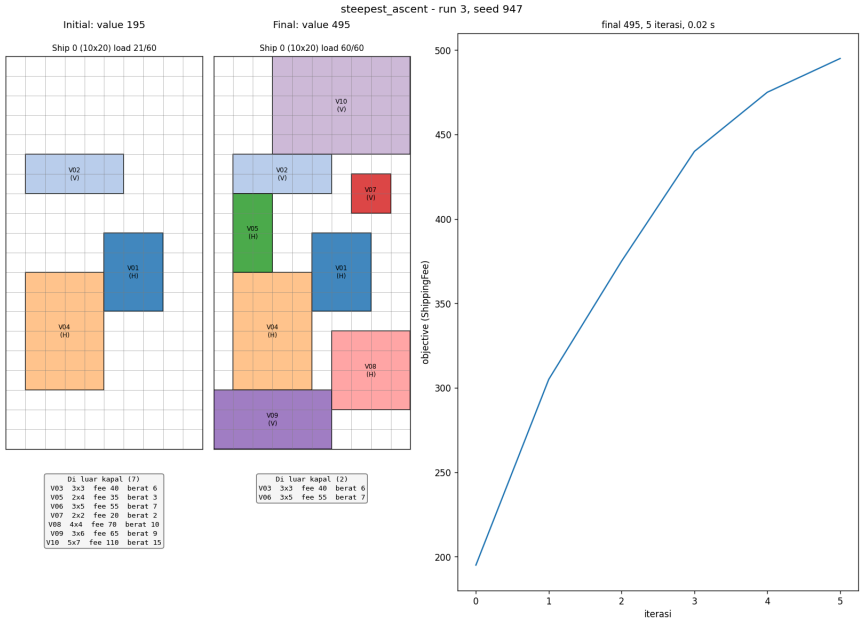
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	260	480	5	0.07	Iterasi: 1000
2	370	485	3	0.05	Iterasi: 1000
3	195	495	5	0.07	Iterasi: 1000



Gambar 2.2.1.1.1. *Steepest ascent*, *seed* 945: *objective* 260 → 480; 5 iterasi.



Gambar 2.2.1.1.2. *Steepest ascent*, *seed* 946: *objective* 370 → 485; 3 iterasi.



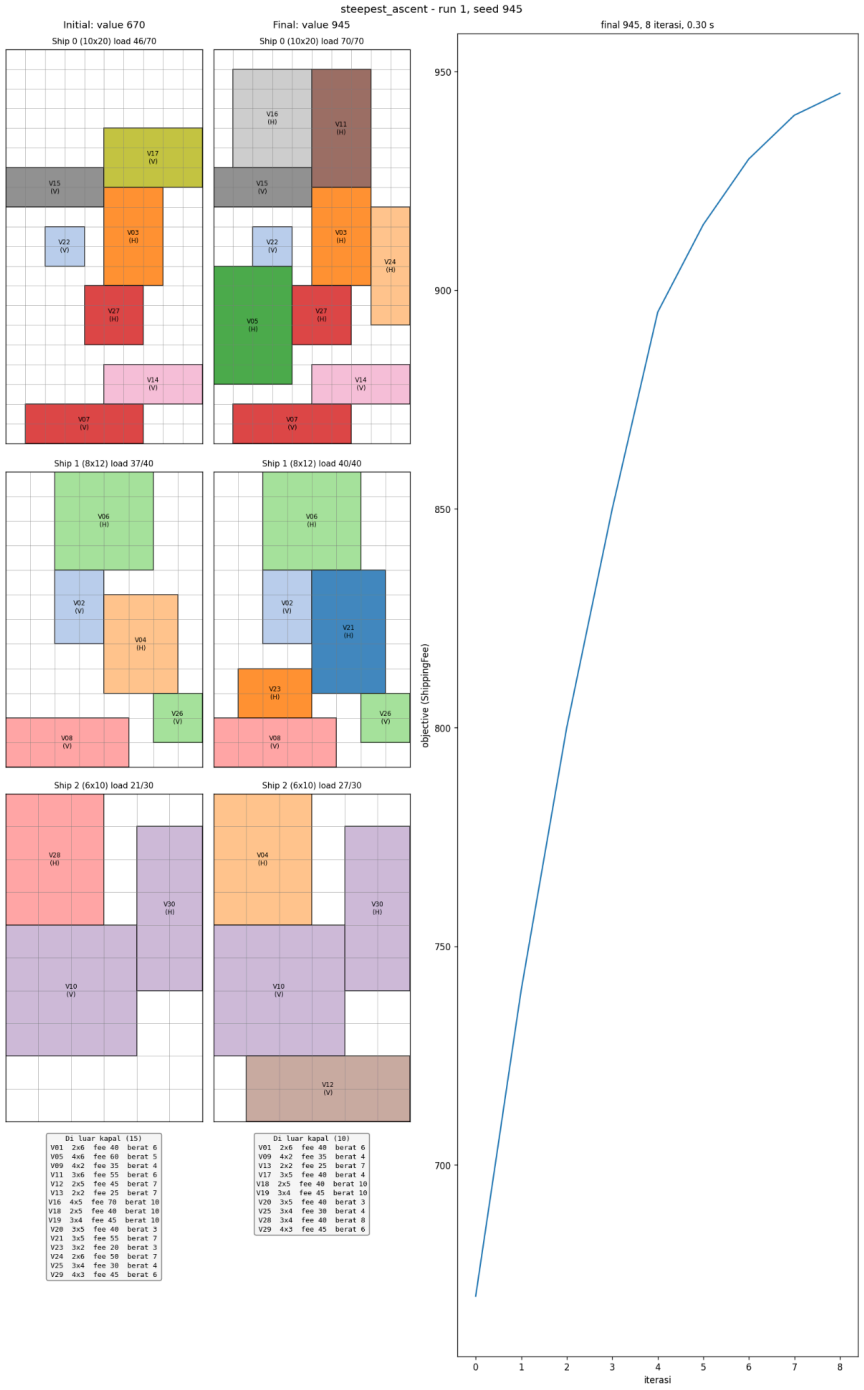
Gambar 2.2.1.1.3. *Steepest ascent*, *seed* 947: *objective* 195 → 495; 5 iterasi.

2.2.1.2. instance_multiship_hard

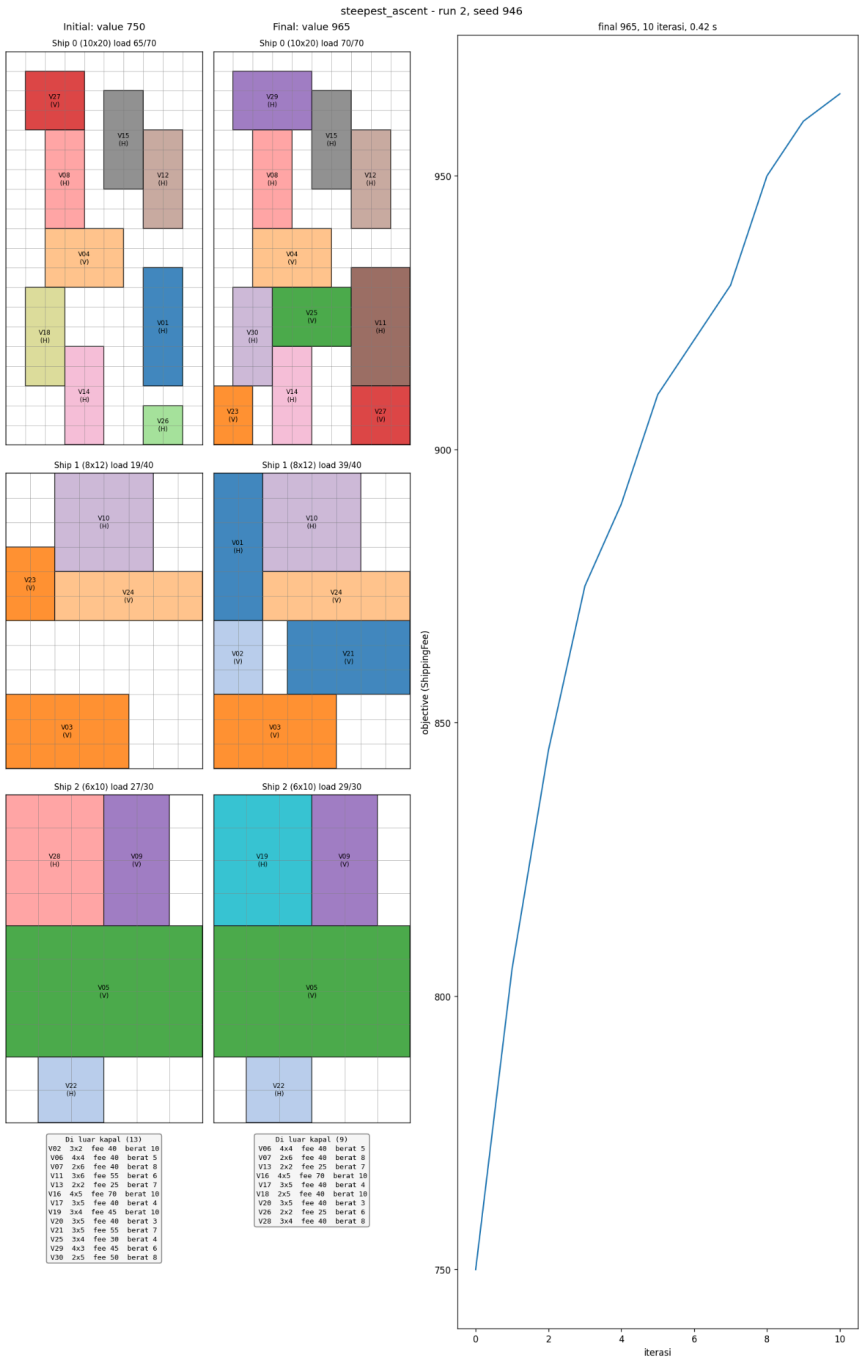
Hasil di bawah berasal dari output/instance_multiship_hard/steepest_ascent/.

Tabel 2.2.1.2. instance_multiship_hard *Steepest Ascent Hill-Climbing*

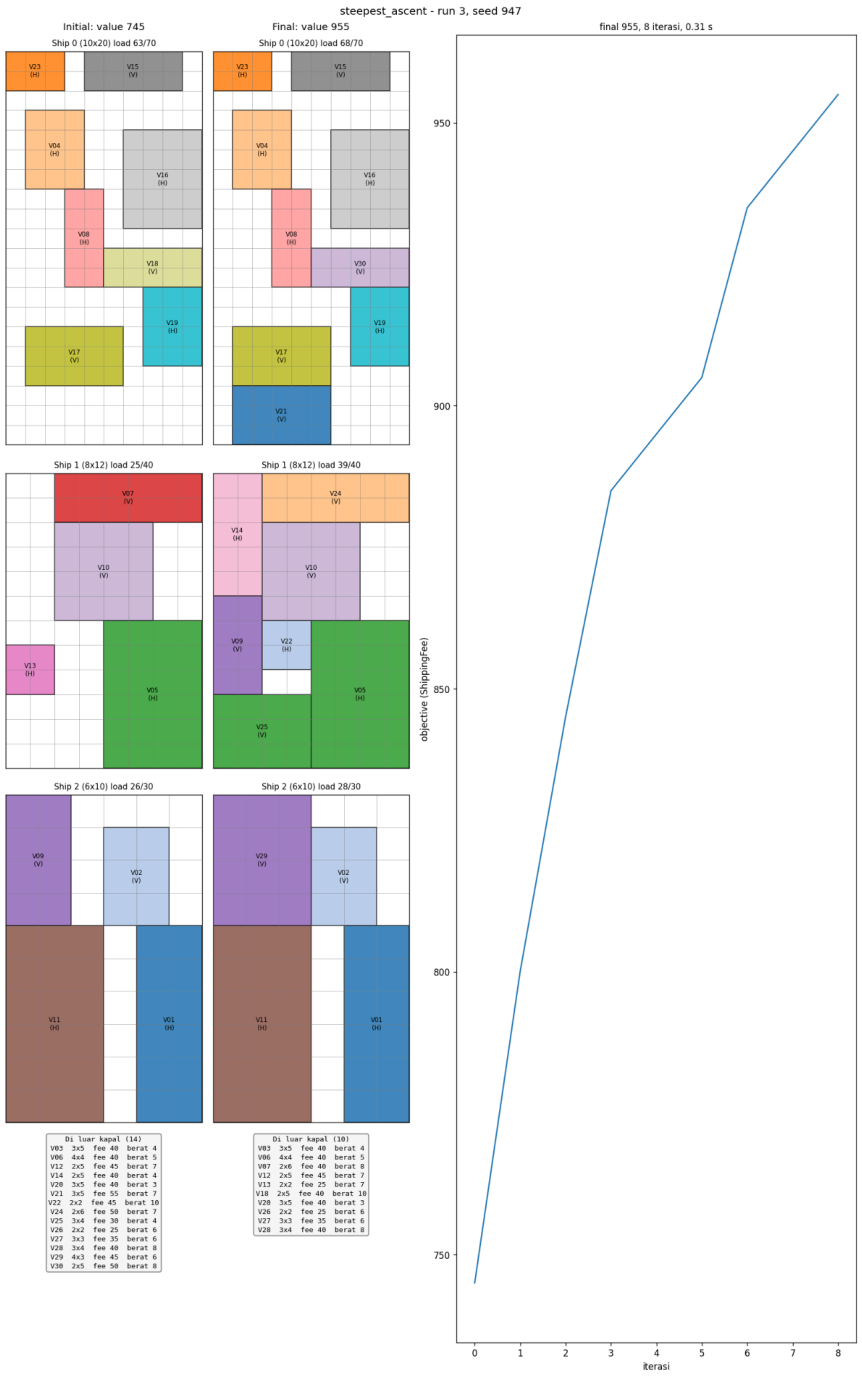
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	670	945	8	0.81	Iterasi: 1000
2	750	965	10	1.17	Iterasi: 1000
3	745	955	8	0.89	Iterasi: 1000



Gambar 2.2.1.2.1. *Steepest ascent*, *seed* 945: *objective* 670 → 945; 8 iterasi.



Gambar 2.2.1.2.2. *Steepest ascent*, *seed* 946: *objective* 750 → 965; 10 iterasi.



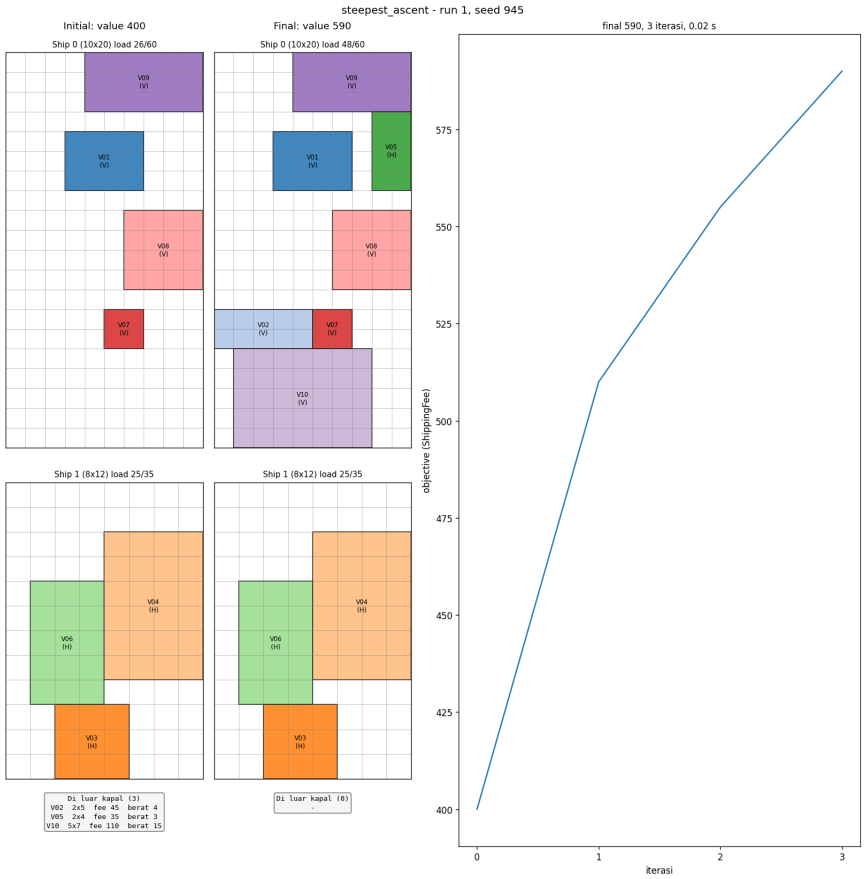
Gambar 2.2.1.2.3. *Steepest ascent*, *seed* 947: *objective* 745 → 955; 8 iterasi.

2.2.1.3. instance_multiship

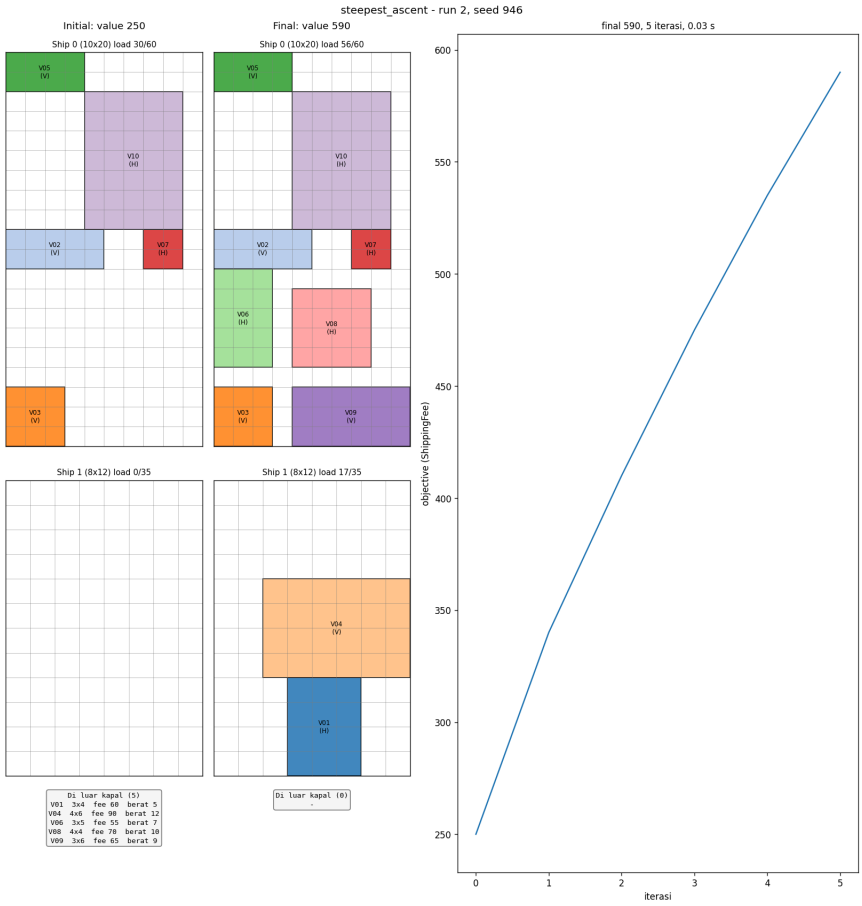
Hasil di bawah berasal dari output/instance_multiship/steepest_ascent/.

Tabel 2.2.1.3. instance_multiship *Steepest Ascent Hill-Climbing*

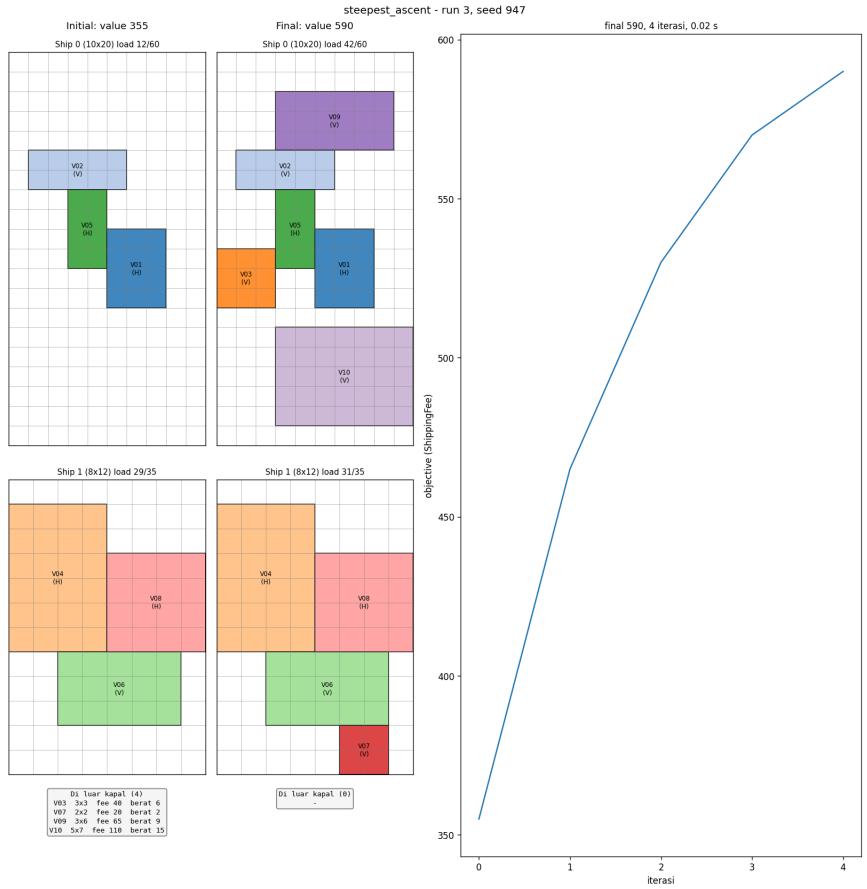
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	400	590	3	0.08	Iterasi: 1000
2	250	590	5	0.11	Iterasi: 1000
3	355	590	4	0.08	Iterasi: 1000



Gambar 2.2.1.3.1. *Steepest ascent*, *seed* 945: *objective* 400 → 590; 3 iterasi.



Gambar 2.2.1.3.2. *Steepest ascent*, *seed* 946: *objective* 250 → 590; 5 iterasi.



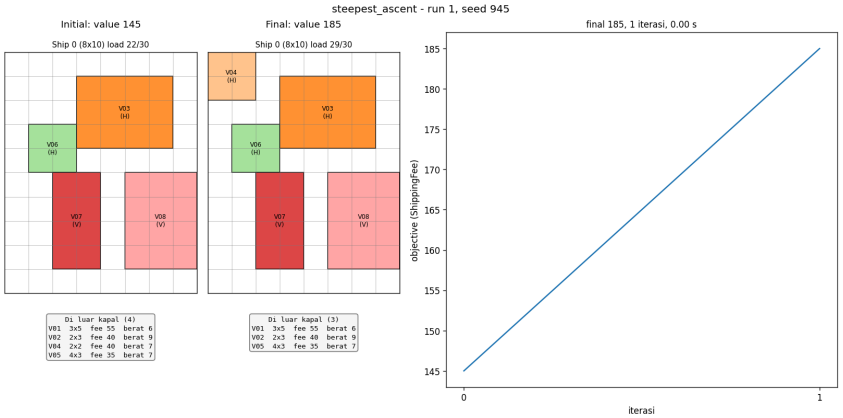
Gambar 2.2.1.3.3. *Steepest ascent*, *seed* 947: *objective* 355 → 590; 4 iterasi.

2.2.1.4. instance_small

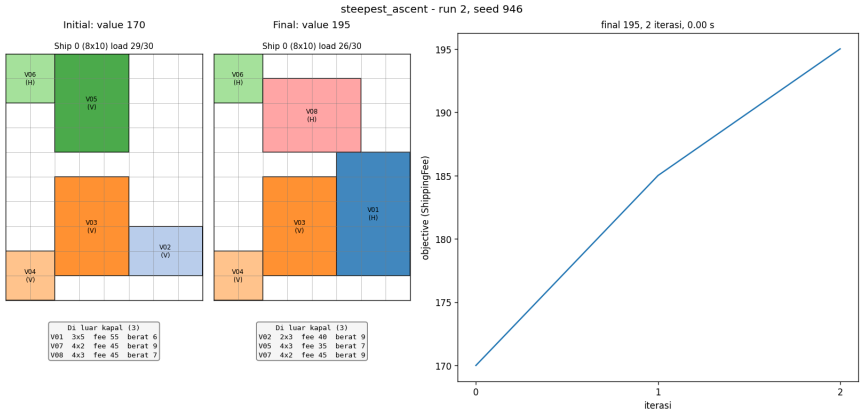
Hasil di bawah berasal dari output/instance_small/steepest_ascent/.

Tabel 2.2.1.4. instance_small *Steepest Ascent Hill-Climbing*

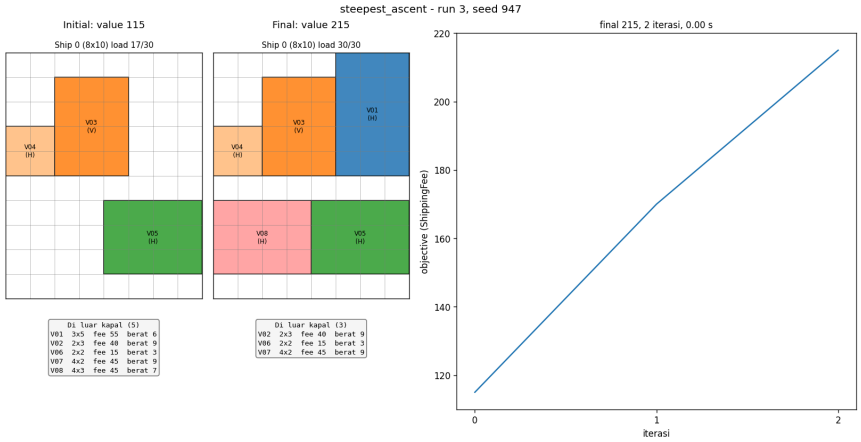
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	145	185	1	0.01	Iterasi: 1000
2	170	195	2	0.01	Iterasi: 1000
3	115	215	2	0.01	Iterasi: 1000



Gambar 2.2.1.4.1. *Steepest ascent*, *seed* 945: *objective* 145 → 185; 1 iterasi.



Gambar 2.2.1.4.2. *Steepest ascent*, *seed* 946: *objective* 170 → 195; 2 iterasi.



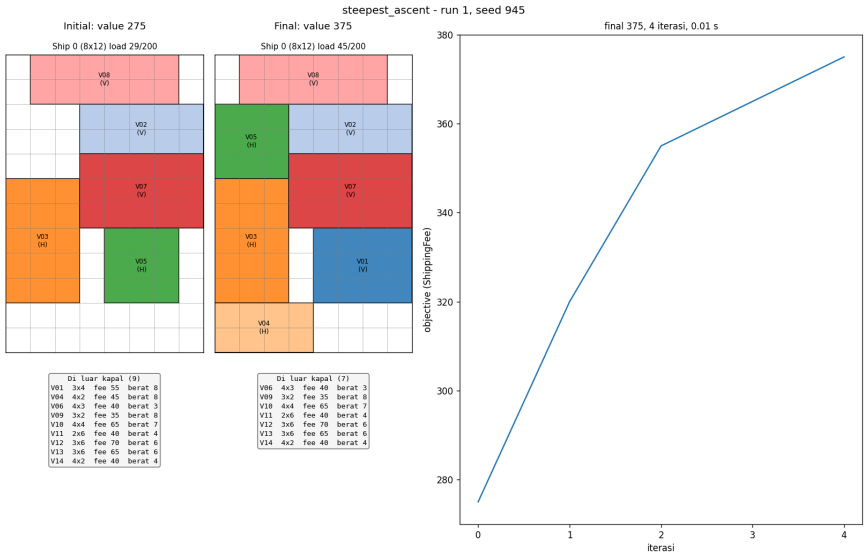
Gambar 2.2.1.4.3. *Steepest ascent*, *seed* 947: *objective* 115 → 215; 2 iterasi.

2.2.1.5. instance_tight_space

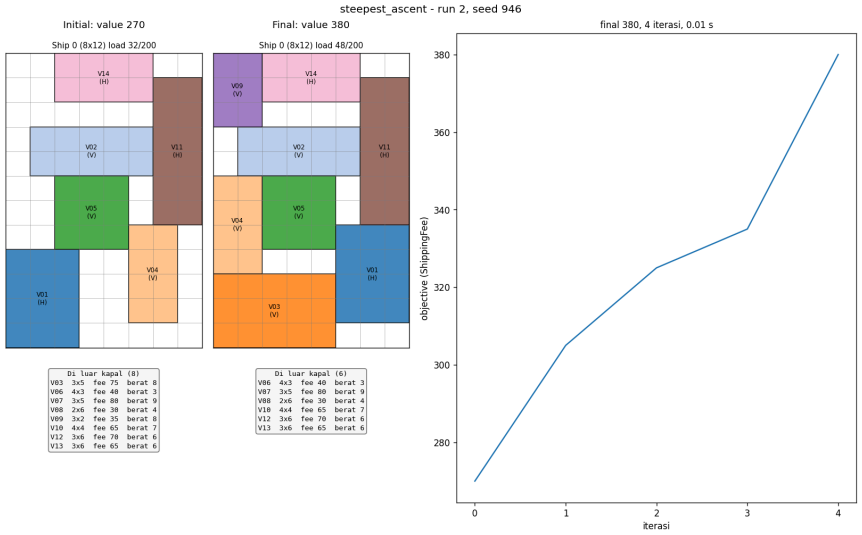
Hasil di bawah berasal dari output/instance_tight_space/steepest_ascent/.

Tabel 2.2.1.5. instance_tight_space *Steepest Ascent Hill-Climbing*

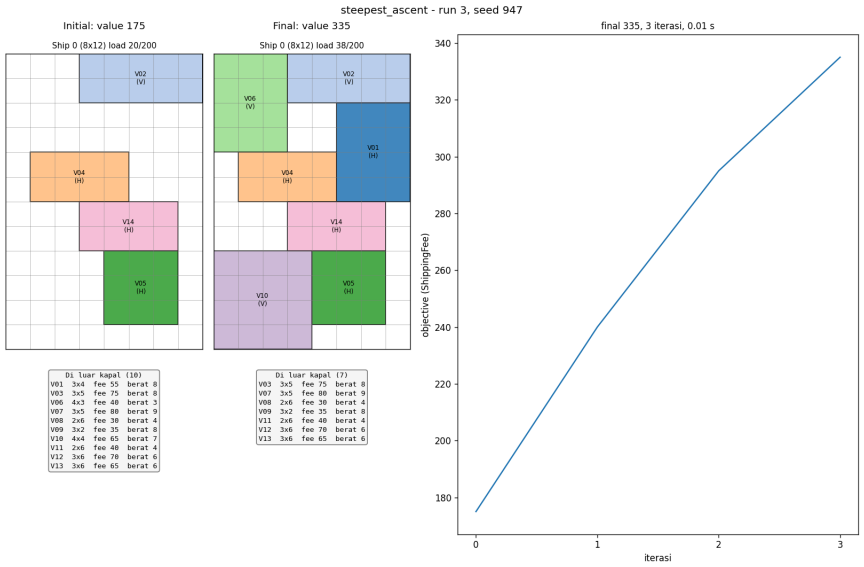
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	275	375	4	0.04	Iterasi: 1000
2	270	380	4	0.05	Iterasi: 1000
3	175	335	3	0.04	Iterasi: 1000



Gambar 2.2.1.5.1. *Steepest ascent*, *seed* 945: *objective* 275 → 375; 4 iterasi.



Gambar 2.2.1.5.2. *Steepest ascent*, *seed* 946: *objective* 270 → 380; 4 iterasi.



Gambar 2.2.1.5.3. *Steepest ascent*, *seed* 947: *objective* 175 → 335; 3 iterasi.

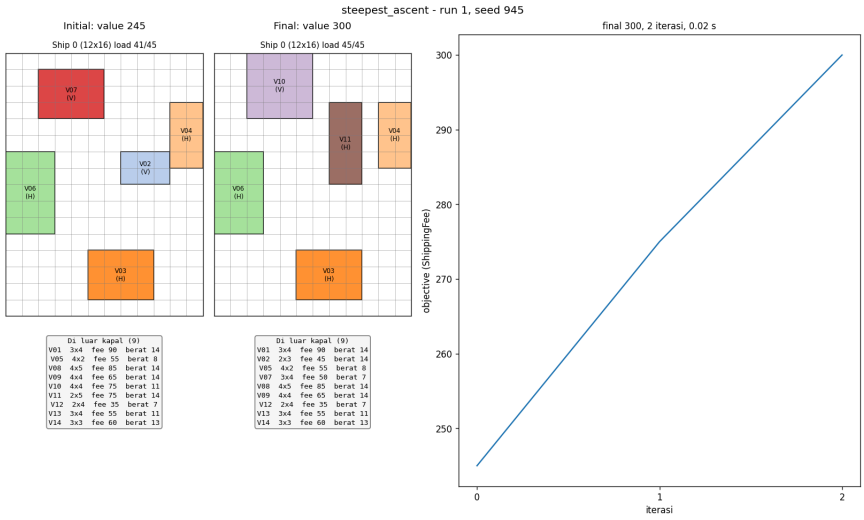
2.2.1.6. instance_tight_weight

Hasil di bawah berasal dari output/instance_tight_weight/steepest_ascent/.

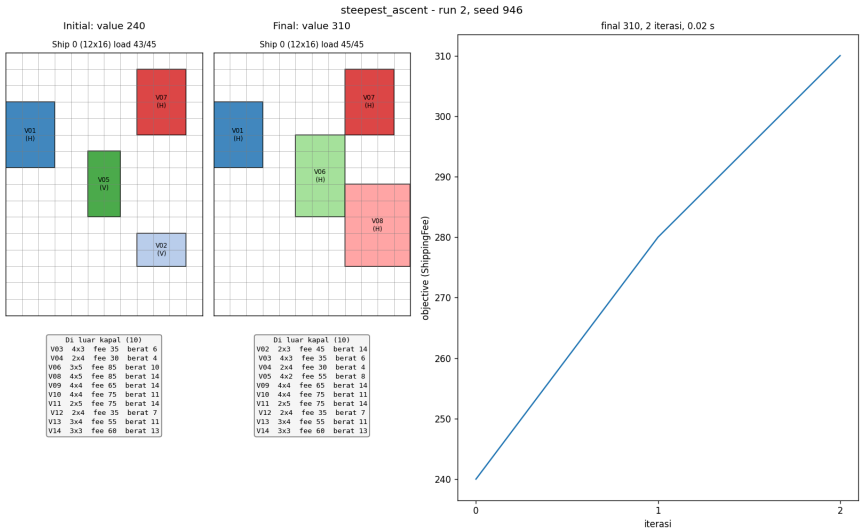
Tabel 2.2.1.6. instance_tight_weight *Steepest Ascent Hill-Climbing*

Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**

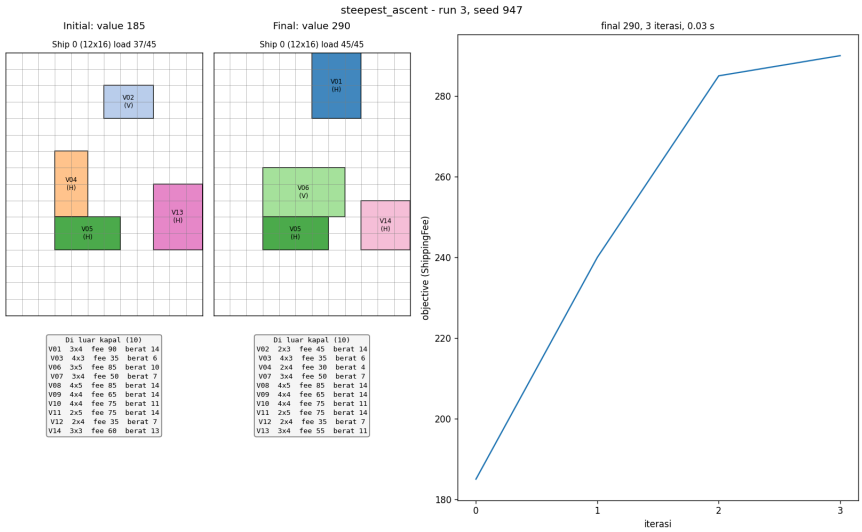
1	245	300	2	0.06	Iterasi: 1000
2	240	310	2	0.06	Iterasi: 1000
3	185	290	3	0.07	Iterasi: 1000



Gambar 2.2.1.6.1. *Steepest ascent*, *seed* 945: *objective* 245 → 300; 2 iterasi.



Gambar 2.2.1.6.2. *Steepest ascent*, *seed* 946: *objective* 240 → 310; 2 iterasi.



Gambar 2.2.1.6.3. *Steepest ascent*, *seed* 947: *objective* 185 → 290; 3 iterasi.

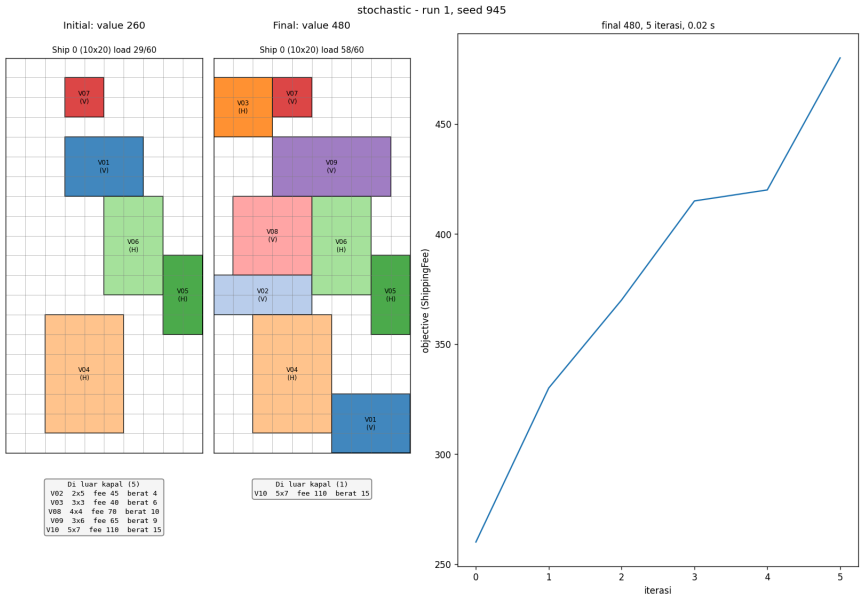
2.2.2. *Stochastic Hill-Climbing*

2.2.2.1. instance_example

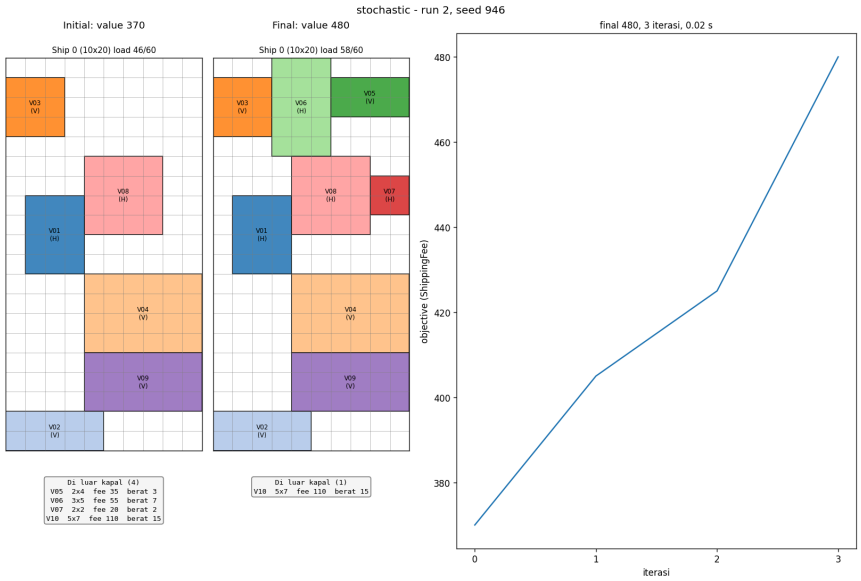
Hasil di bawah berasal dari output/instance_example/stochastic/.

Tabel 2.2.2.1. instance_example *Stochastic Hill-Climbing*

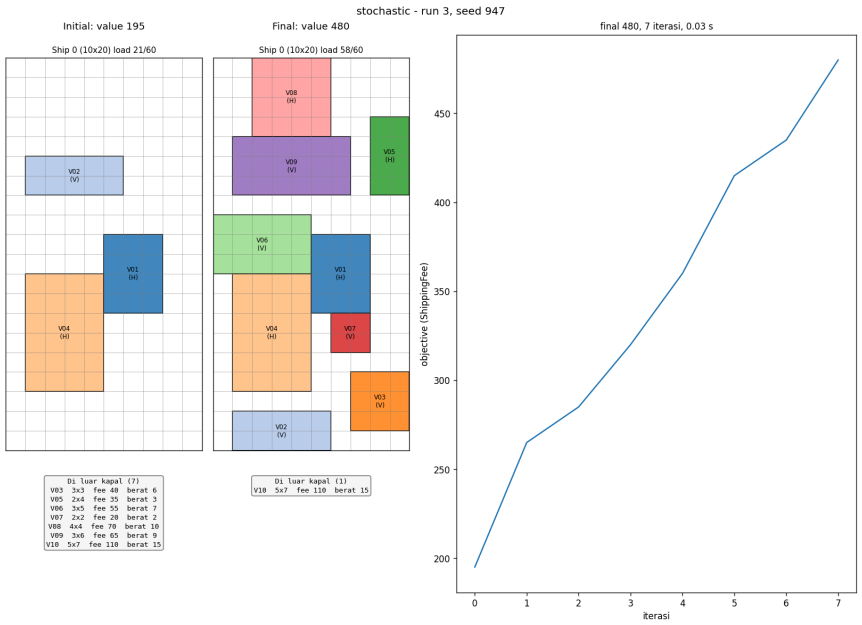
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	260	480	5	0.08	Iterasi: 1000
2	370	480	3	0.05	Iterasi: 1000
3	195	480	7	0.10	Iterasi: 1000



Gambar 2.2.2.1.1. *Stochastic*, *seed* 945: *objective* 260 → 480; 5 iterasi.



Gambar 2.2.2.1.2. *Stochastic*, *seed* 946: *objective* 370 → 480; 3 iterasi.



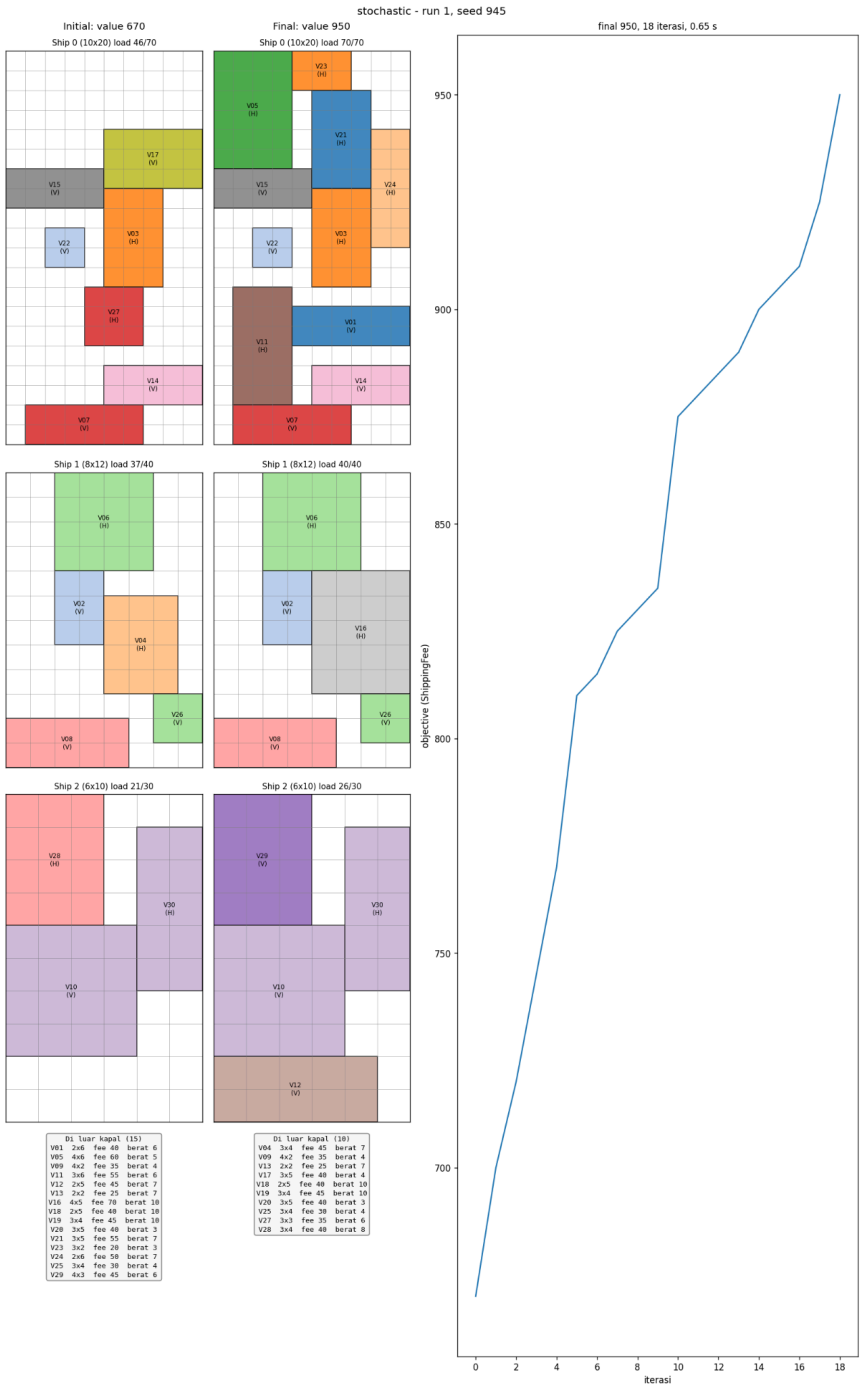
Gambar 2.2.2.1.3. *Stochastic*, *seed* 947: *objective* 195 → 480; 7 iterasi.

2.2.2.2. instance_multiship_hard

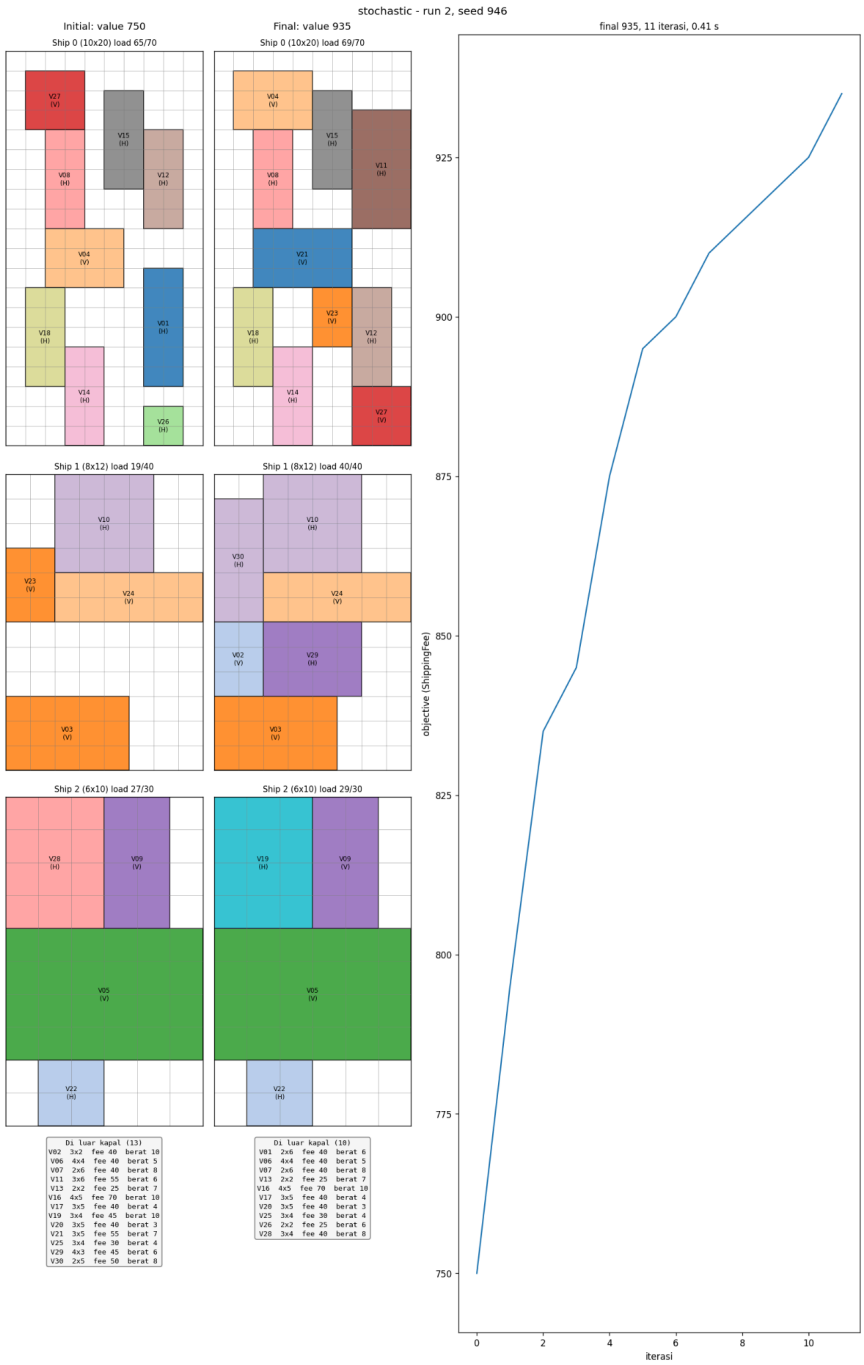
Hasil di bawah berasal dari output/instance_multiship_hard/stochastic/.

Tabel 2.2.2.2. instance_multiship_hard *Stochastic Hill-Climbing*

Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	670	950	18	1.89	Iterasi: 1000
2	750	935	11	1.03	Iterasi: 1000
3	745	955	9	0.61	Iterasi: 1000



Gambar 2.2.2.2.1. *Stochastic*, *seed* 945: *objective* 670 → 950; 18 iterasi.



Gambar 2.2.2.2.2. *Stochastic*, *seed* 946: *objective* 750 → 935; 11 iterasi.



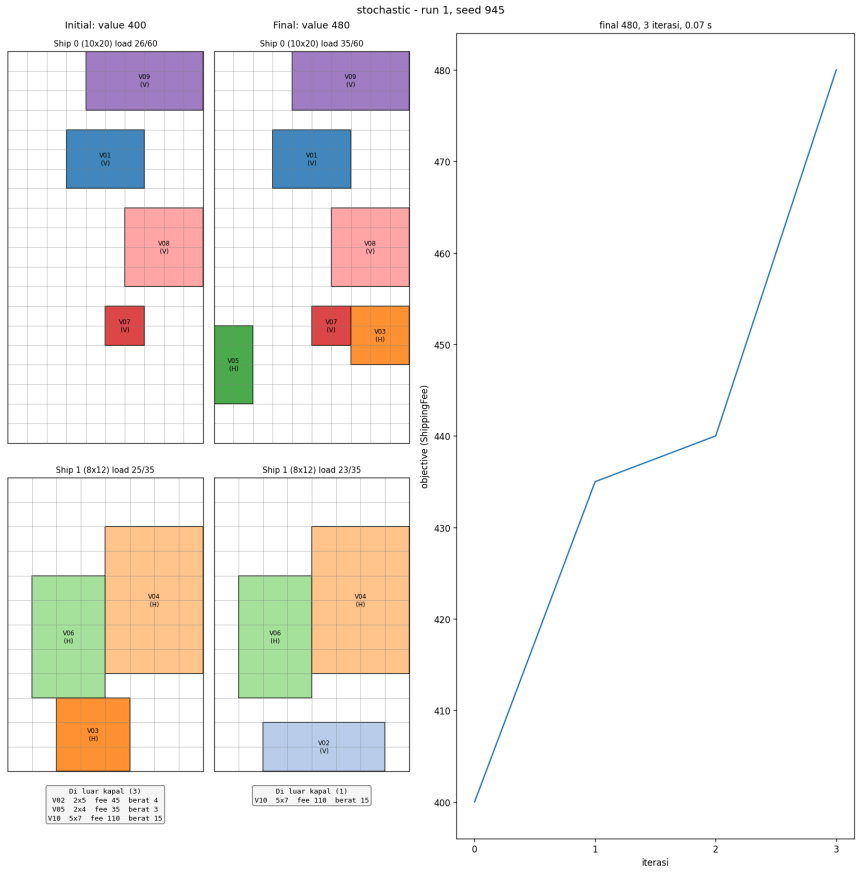
Gambar 2.2.2.2.3. *Stochastic*, *seed* 947: *objective* 745 → 955; 9 iterasi.

2.2.2.3. instance_multiship

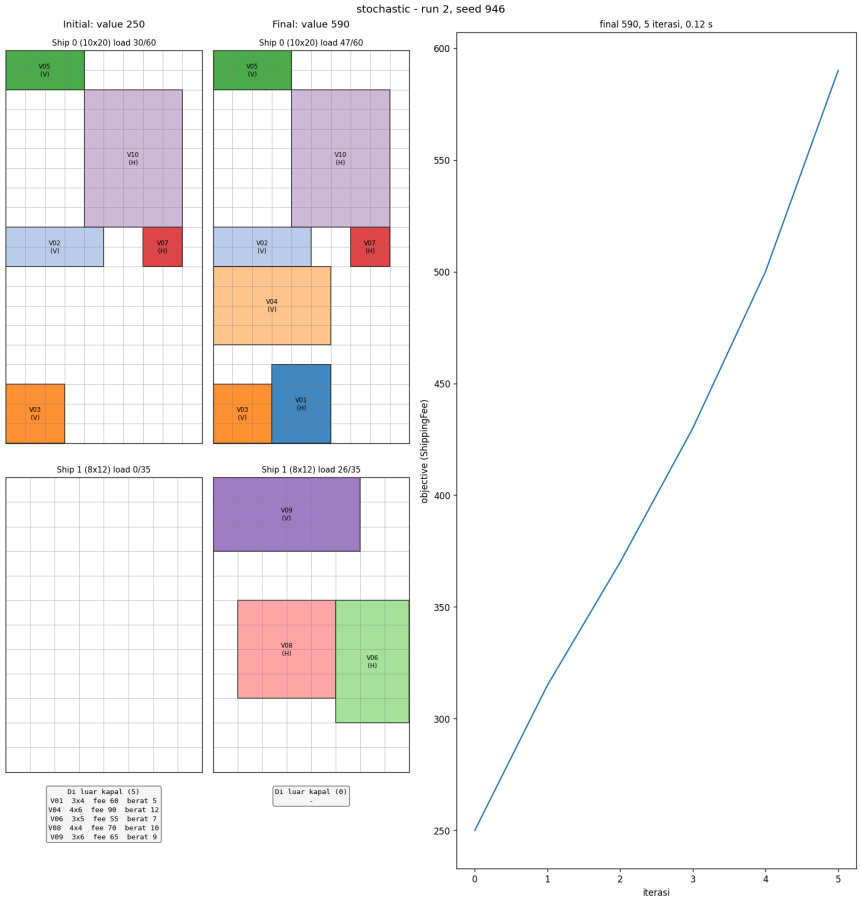
Hasil di bawah berasal dari output/instance_multiship/stochastic.

Tabel 2.2.2.3. instance_multiship *Stochastic Hill-Climbing*

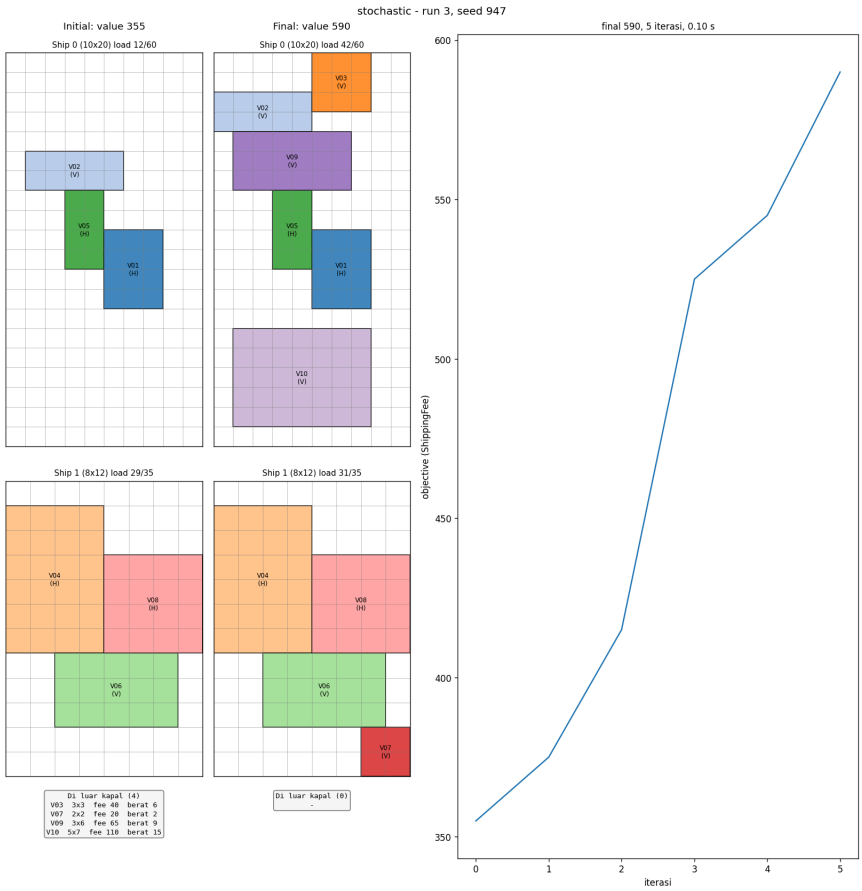
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	400	480	3	0.07	Iterasi: 1000
2	250	590	5	0.12	Iterasi: 1000
3	355	590	5	0.10	Iterasi: 1000



Gambar 2.2.2.3.1. *Stochastic*, *seed* 945: *objective* 400 → 480; 3 iterasi.



Gambar 2.2.2.3.2. *Stochastic*, *seed* 946: *objective* 250 → 590; 5 iterasi.



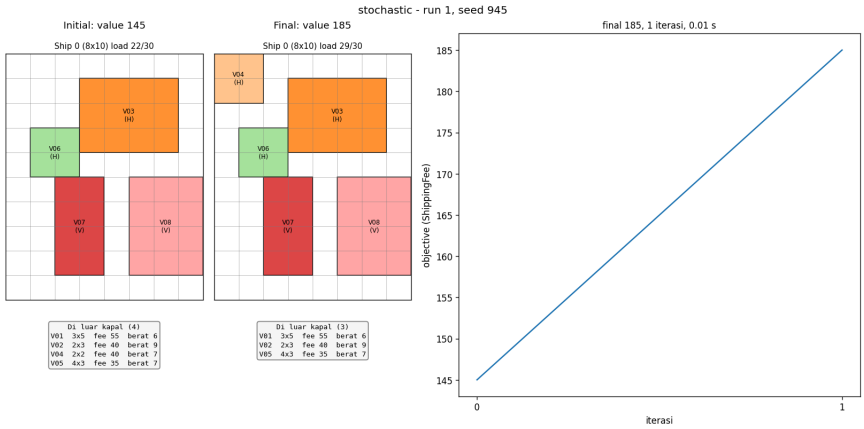
Gambar 2.2.2.3.3. *Stochastic*, *seed* 947: *objective* 355 → 590; 5 iterasi.

2.2.2.4. instance_small

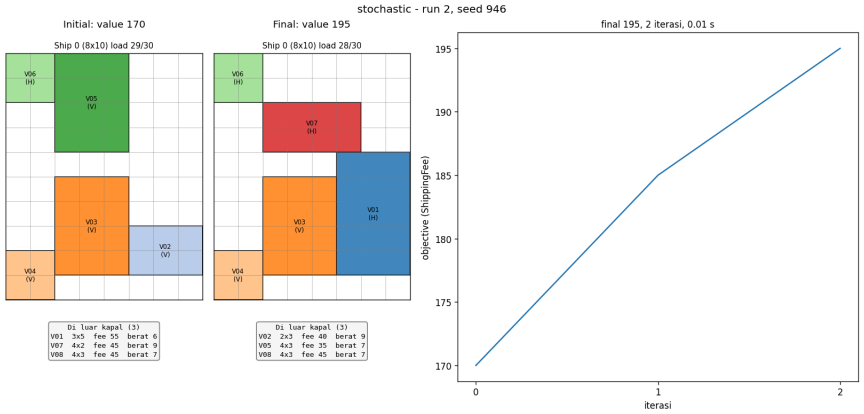
Hasil di bawah berasal dari output/instance_small/stochastic/.

Tabel 2.2.2.4. instance_small *Stochastic Hill-Climbing*

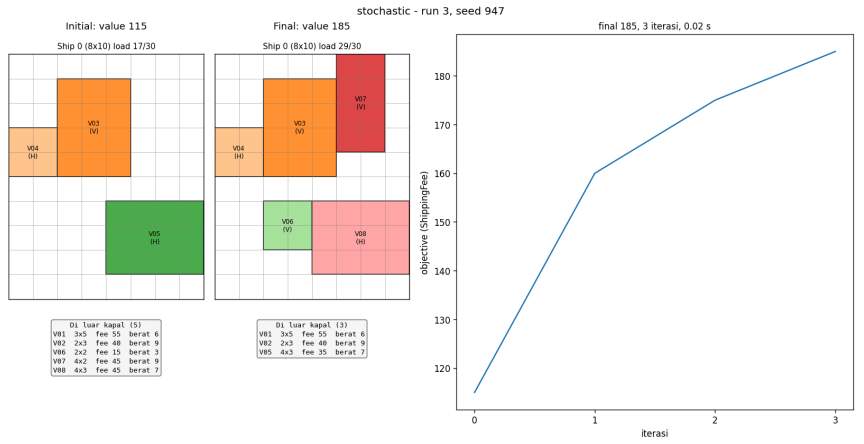
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	145	185	1	0.01	Iterasi: 1000
2	170	195	2	0.01	Iterasi: 1000
3	115	185	3	0.02	Iterasi: 1000



Gambar 2.2.2.4.1. Stochastic, seed 945: objective 145 → 185; 1 iterasi.



Gambar 2.2.2.4.2. Stochastic, seed 946: objective 170 → 195; 2 iterasi.



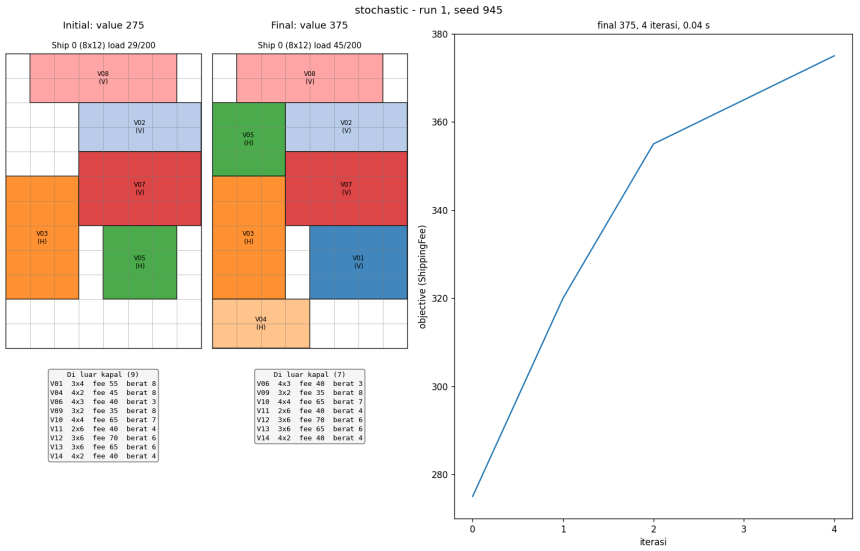
Gambar 2.2.2.4.3. Stochastic, seed 947: objective 115 → 185; 3 iterasi.

2.2.2.5. instance_tight_space

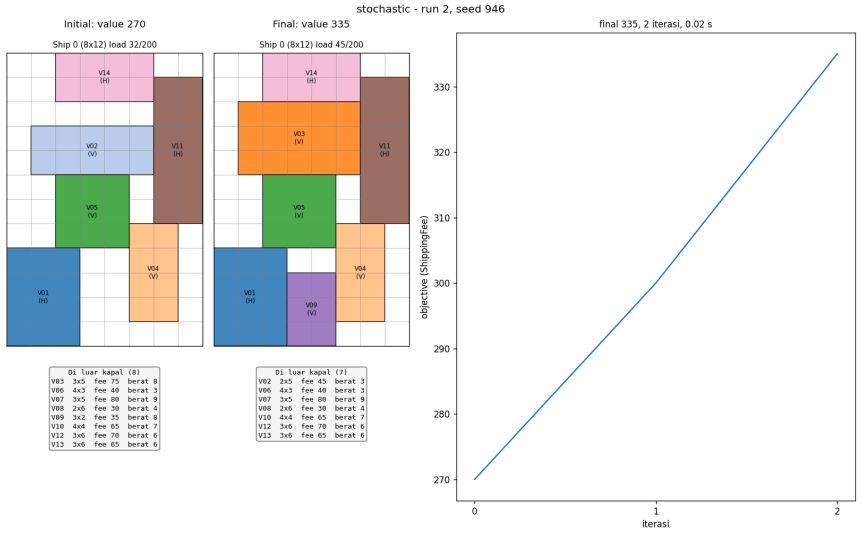
Hasil di bawah berasal dari output/instance_tight_space/stochastic/.

Tabel 2.2.2.5. instance_tight_space *Stochastic Hill-Climbing*

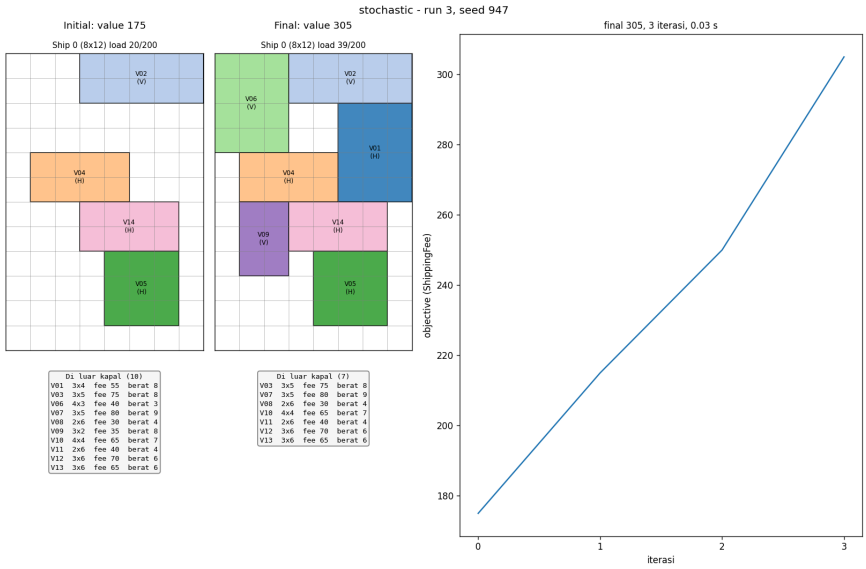
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	275	375	4	0.04	Iterasi: 1000
2	270	335	2	0.02	Iterasi: 1000
3	175	305	3	0.03	Iterasi: 1000



Gambar 2.2.2.5.1. Stochastic, seed 945: objective 275 → 375; 4 iterasi.



Gambar 2.2.2.5.2. Stochastic, seed 946: objective 270 → 335; 2 iterasi.



Gambar 2.2.2.5.3. Stochastic, seed 947: objective 175 → 305; 3 iterasi.

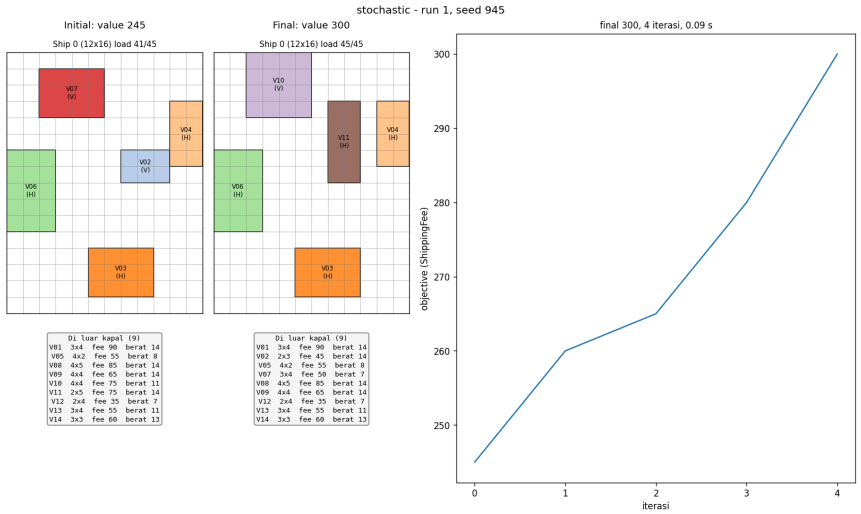
2.2.2.6. instance_tight_weight

Hasil di bawah berasal dari output/instance_tight_weight/stochastic/.

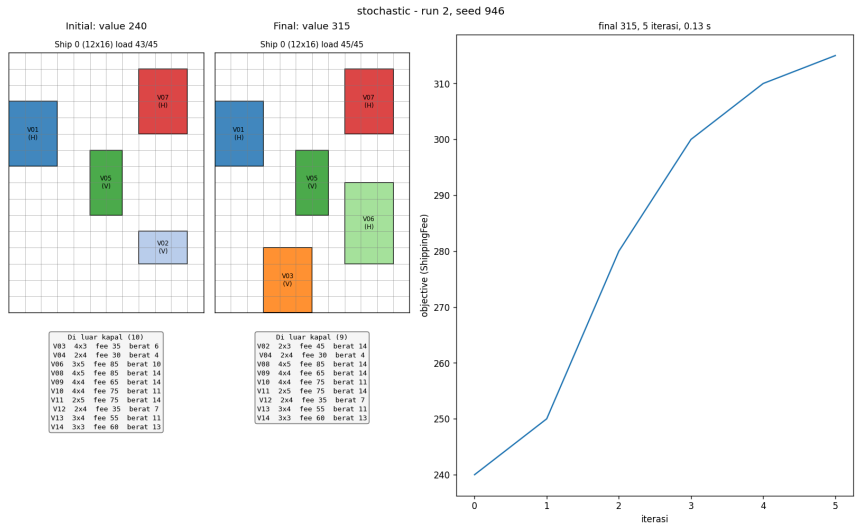
Tabel 2.2.2.6. instance_tight_weight *Stochastic Hill-Climbing*

Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**

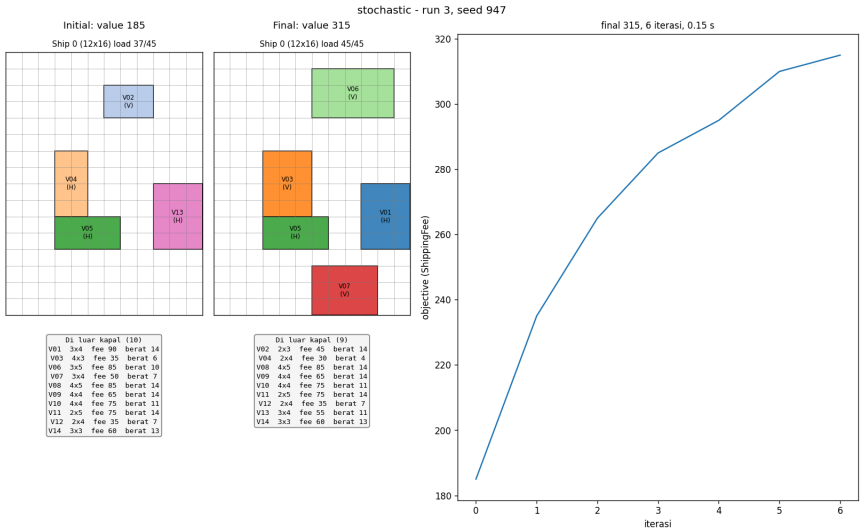
1	245	300	4	0.09	Iterasi: 1000
2	240	315	5	0.13	Iterasi: 1000
3	185	315	6	0.15	Iterasi: 1000



Gambar 2.2.2.6.1. Stochastic, seed 945: objective 245 → 300; 4 iterasi.



Gambar 2.2.2.6.2. Stochastic, seed 946: objective 240 → 315; 5 iterasi.



Gambar 2.2.2.6.3. Stochastic, seed 947: objective 185 → 315; 6 iterasi.

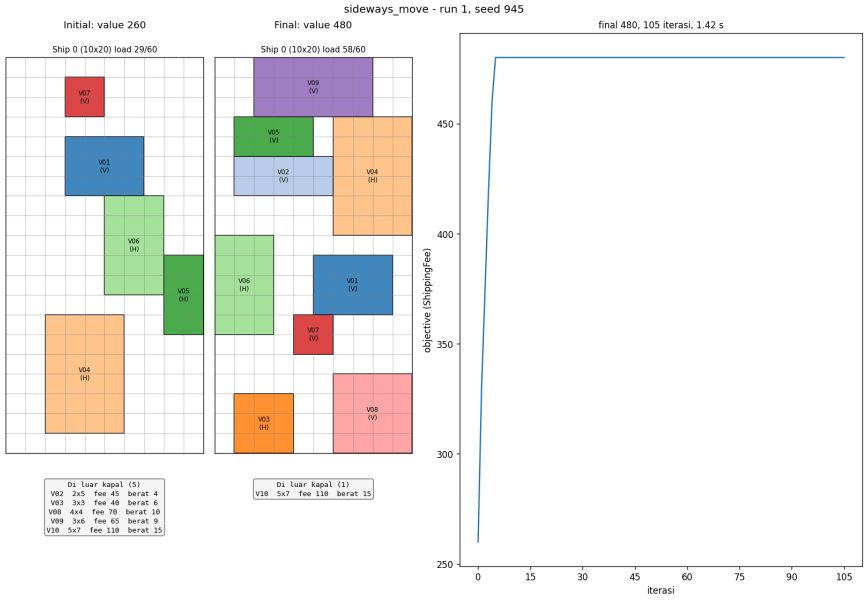
2.2.3. Hill-Climbing with Sideways Move

2.2.3.1. instance_example

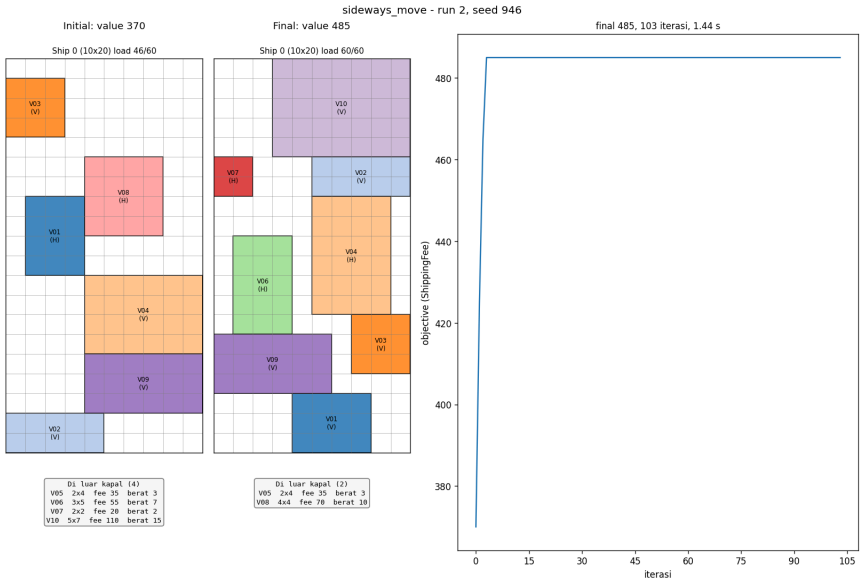
Hasil di bawah berasal dari output/instance_example/sideways_move/.

Tabel 2.2.3.1. instance_example *Hill-Climbing with Sideways Move*

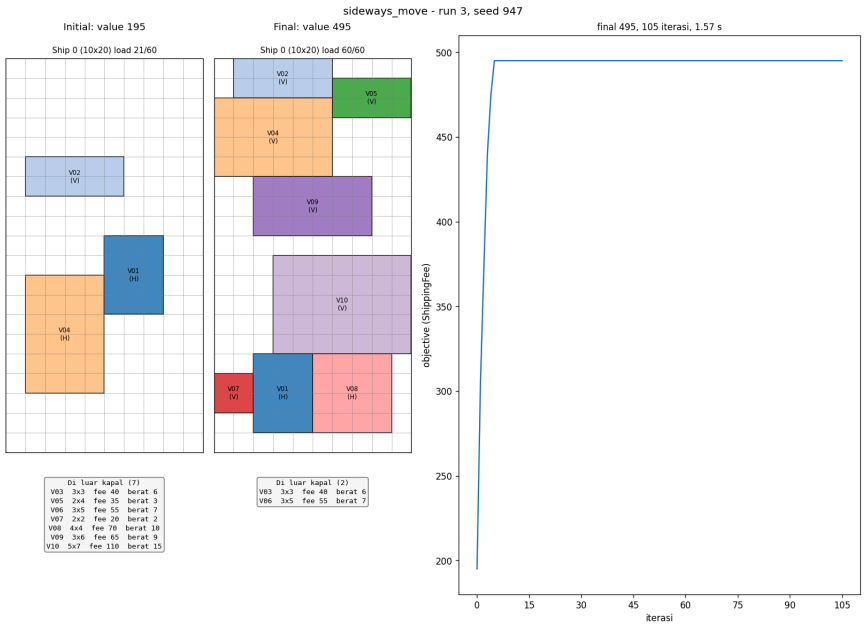
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	260	480	105	1.42	Sideways: 100
2	370	485	103	1.44	Sideways: 100
3	195	495	105	1.57	Sideways: 100



Gambar 2.2.3.1.1. Sideways move, seed 945: objective 260 → 480; 105 iterasi; 100 sideways.



Gambar 2.2.3.1.2. Sideways move, seed 946: objective 370 → 485; 103 iterasi; 100 sideways.



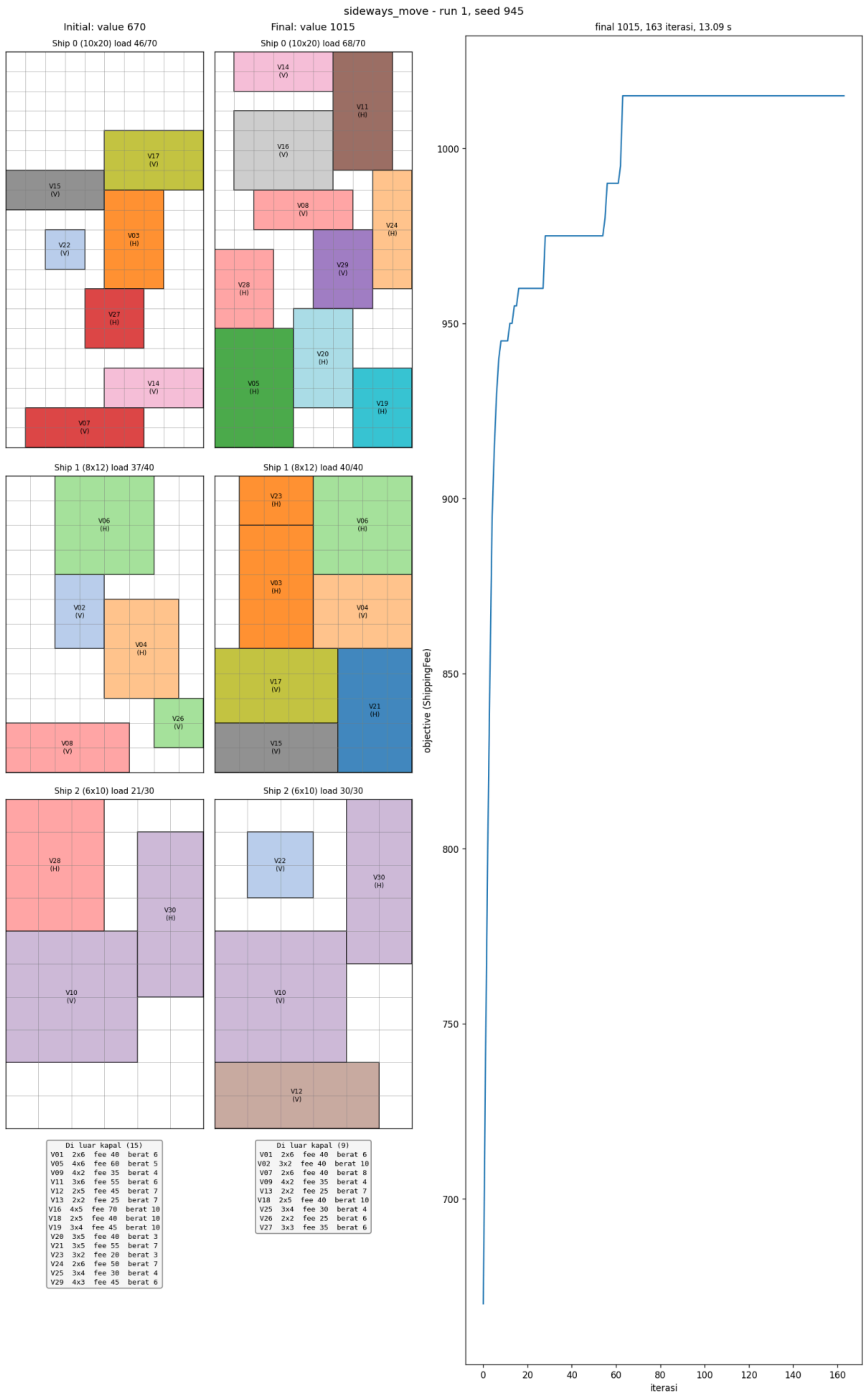
Gambar 2.2.3.1.3. Sideways move, seed 947: objective 195 → 495; 105 iterasi; 100 sideways.

2.2.3.2. instance_multiship_hard

Hasil di bawah berasal dari output/instance_multiship_hard/sideways_move/.

Tabel 2.2.3.2. instance_multiship_hard *Hill-Climbing with Sideways Move*

Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	670	1015	163	13.09	Sideways: 100
2	750	1000	192	20.14	Sideways: 100
3	745	1045	258	25.08	Sideways: 100



Gambar 2.2.3.2.1. Sideways move, seed 945: objective 670 → 1015; 163 iterasi; 147 sideways.



Gambar 2.2.3.2.2. Sideways move, seed 946: objective 750 → 1000; 192 iterasi; 177 sideways.



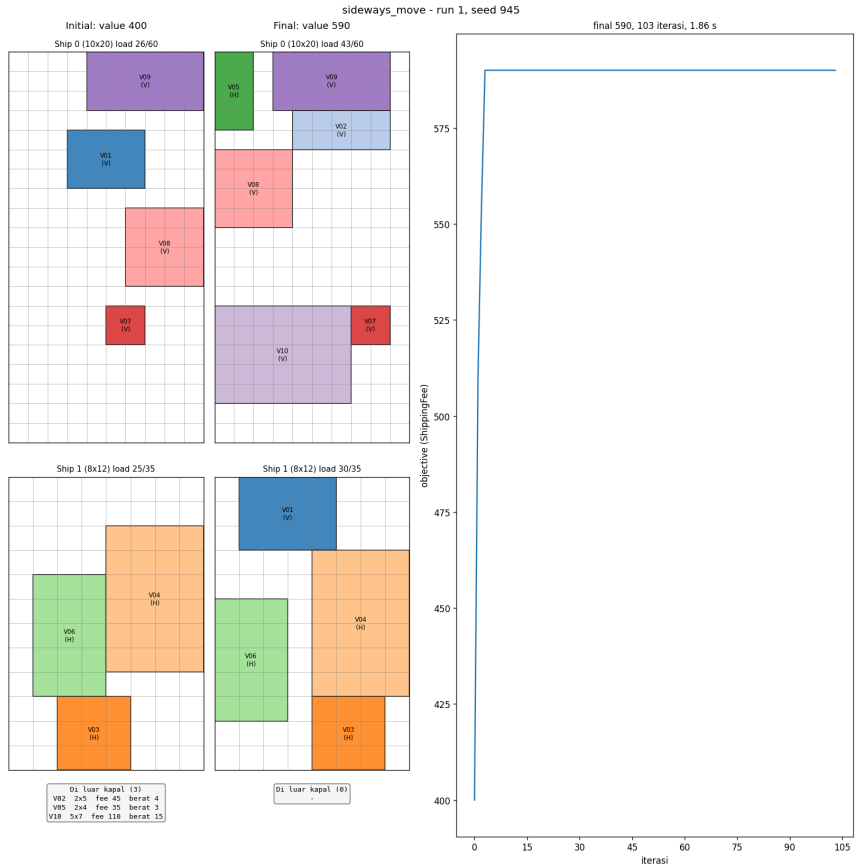
Gambar 2.2.3.2.3. Sideways move, seed 947: objective 745 → 1045; 258 iterasi; 243 sideways.

2.2.3.3. instance_multiship

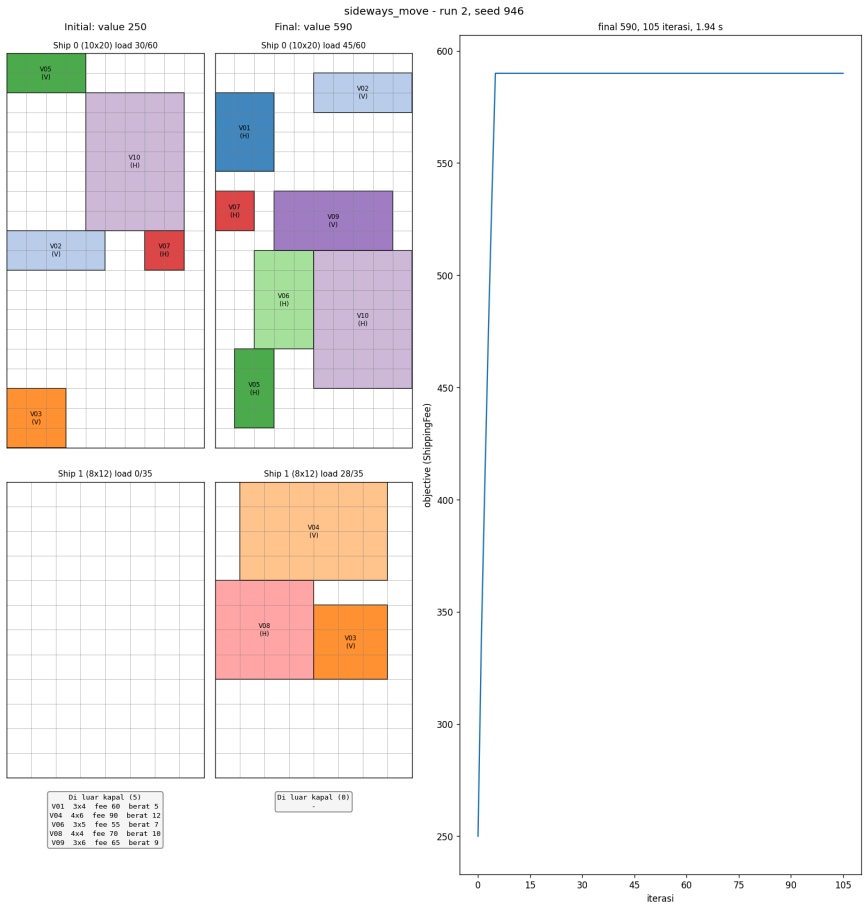
Hasil di bawah berasal dari output/instance_multiship/sideways_move.

Tabel 2.2.3.3. instance_multiship *Hill-Climbing with Sideways Move*

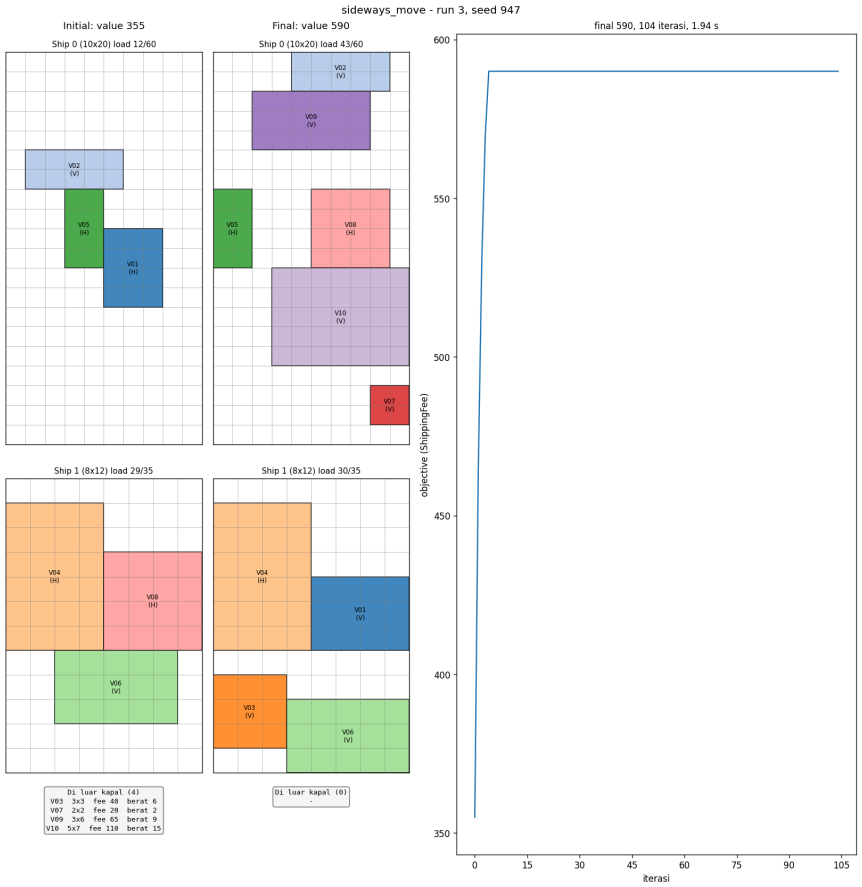
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	400	590	103	1.8583	Sideways: 100
2	250	590	105	1.9380	Sideways: 100
3	355	590	104	1.9394	Sideways: 100



Gambar 2.2.3.3.1. Sideways move, seed 945: objective 400 → 590; 103 iterasi; 100 sideways.



Gambar 2.2.3.3.2. Sideways move, seed 946: objective 250 → 590; 105 iterasi; 100 sideways.



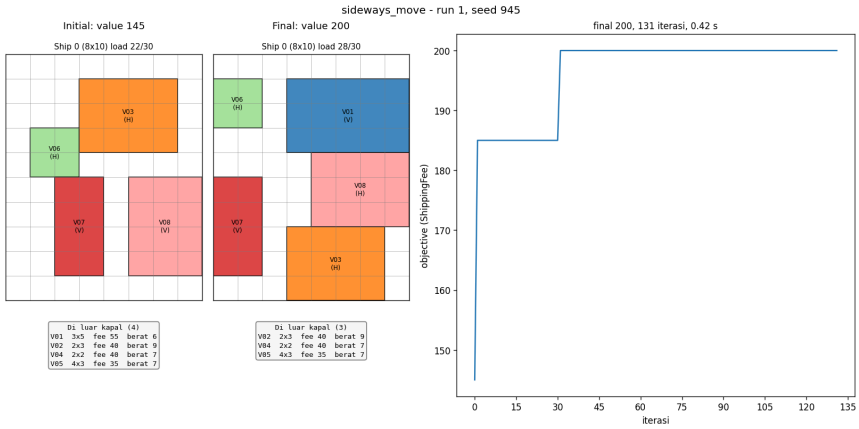
Gambar 2.2.3.3.3. Sideways move, seed 947: objective 355 → 590; 104 iterasi; 100 sideways.

2.2.3.4. instance_small

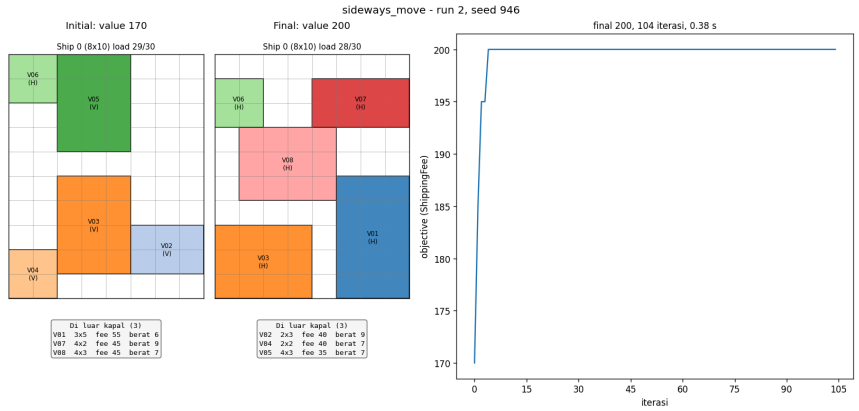
Hasil di bawah berasal dari output/instance_small/sideways_move/.

Tabel 2.2.3.4. instance_small *Hill-Climbing with Sideways Move*

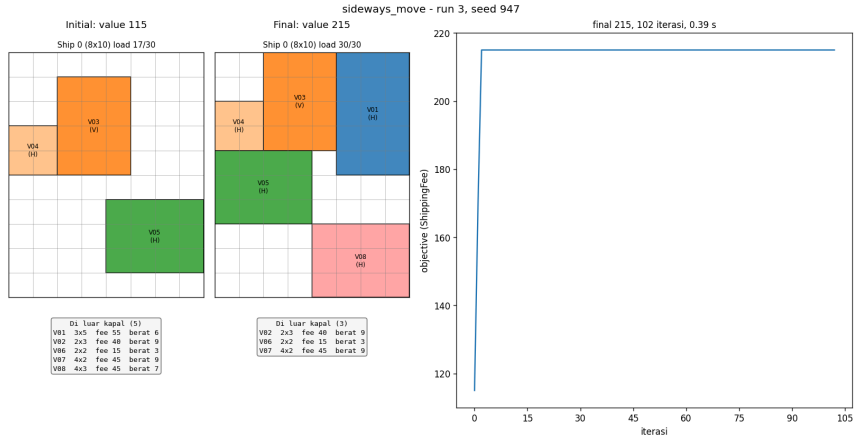
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	145	200	131	0.4198	Sideways: 100
2	170	200	104	0.3772	Sideways: 100
3	115	215	102	0.3940	Sideways: 100



Gambar 2.2.3.4.1. Sideways move, seed 945: objective 145 → 200; 131 iterasi; 129 sideways.



Gambar 2.2.3.4.2. Sideways move, seed 946: objective 170 → 200; 104 iterasi; 101 sideways.



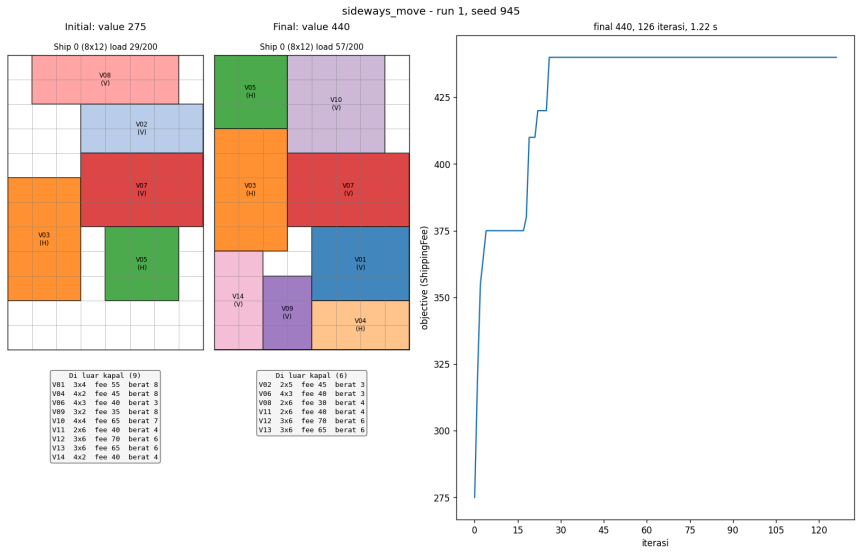
Gambar 2.2.3.4.3. Sideways move, seed 947: objective 115 → 215; 102 iterasi; 100 sideways.

2.2.3.5. instance_tight_space

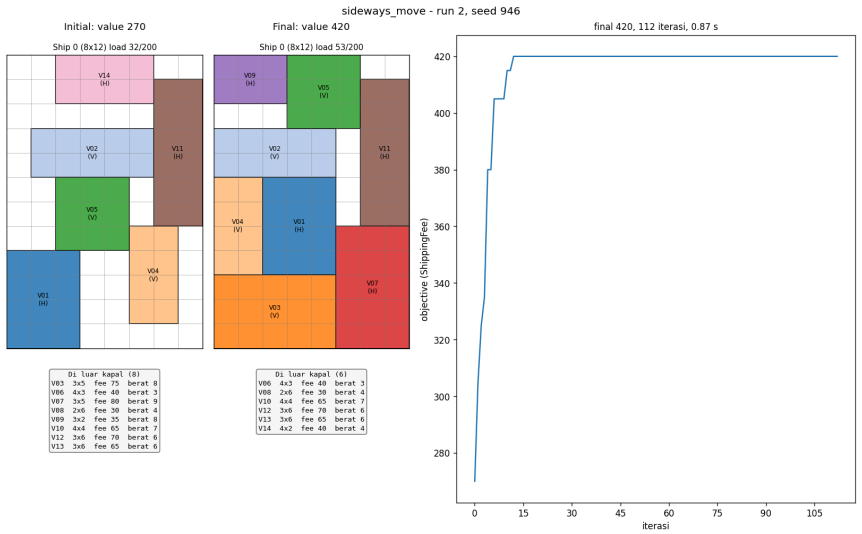
Hasil di bawah berasal dari output/instance_tight_space/sideways_move/.

Tabel 2.2.3.5. instance_tight_space *Hill-Climbing with Sideways Move*

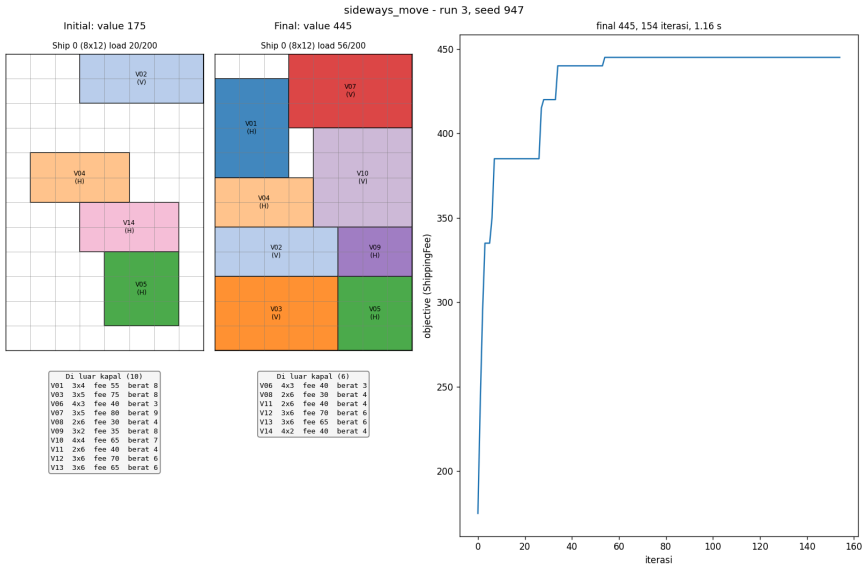
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	275	440	126	1.2175	Sideways: 100
2	270	420	112	0.8743	Sideways: 100
3	175	445	154	1.1617	Sideways: 100



Gambar 2.2.3.5.1. Sideways move, seed 945: objective 275 → 440; 126 iterasi; 118 sideways.



Gambar 2.2.3.5.2. Sideways move, seed 946: objective 270 → 420; 112 iterasi; 105 sideways.



Gambar 2.2.3.5.3. Sideways move, seed 947: objective 175 → 445; 154 iterasi; 145 sideways.

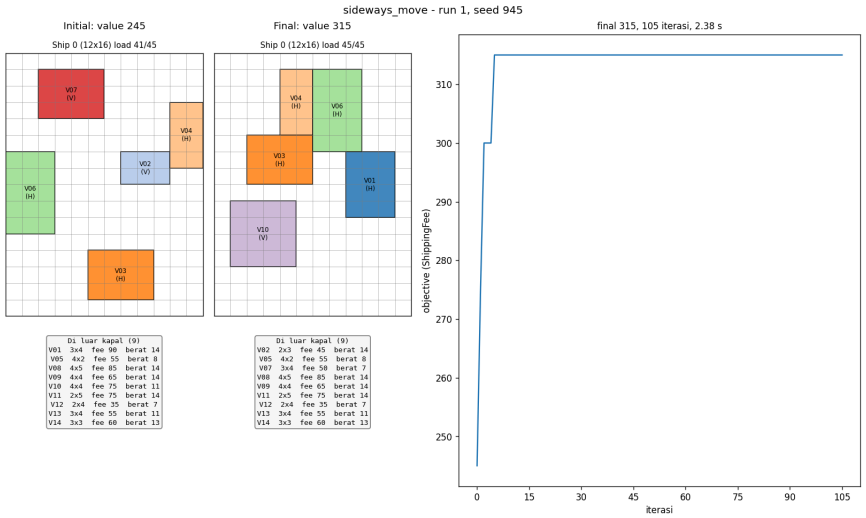
2.2.3.6. instance_tight_weight

Hasil di bawah berasal dari output/instance_tight_weight/sideways_move/.

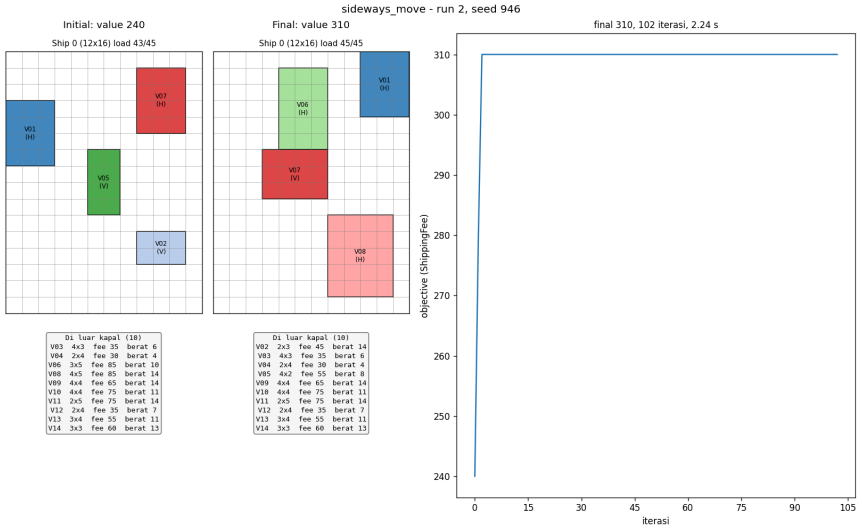
Tabel 2.2.3.6. instance_tight_weight *Hill-Climbing with Sideways Move*

Run	***Objective***	***Objective***	**Banyak**	**Durasi**	**Parameter**

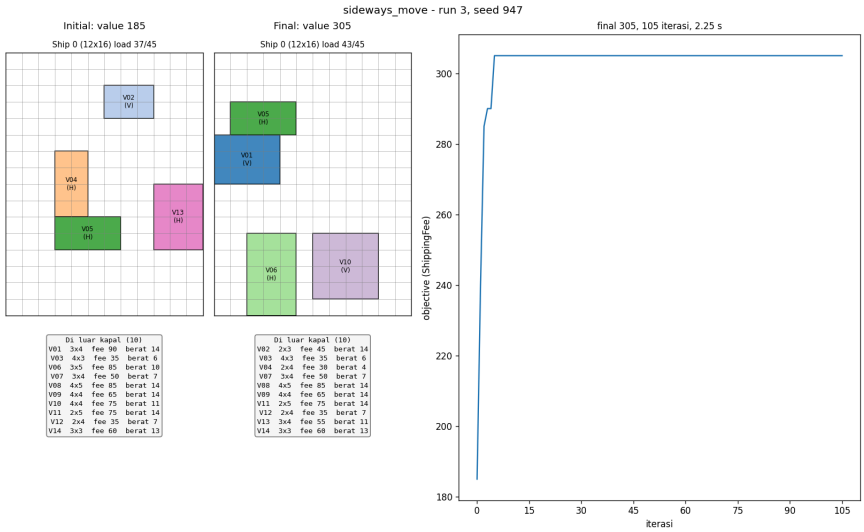
	Awal	Akhir	Iterasi	(s)	Khusus Varian
1	245	315	105	2.3782	Sideways: 100
2	240	310	102	2.2411	Sideways: 100
3	185	305	105	2.2457	Sideways: 100



Gambar 2.2.3.6.1. Sideways move, seed 945: objective 245 → 315; 105 iterasi; 102 sideways.



Gambar 2.2.3.6.2. Sideways move, seed 946: objective 240 → 310; 102 iterasi; 100 sideways.



Gambar 2.2.3.6.3. Sideways move, seed 947: objective 185 → 305; 105 iterasi; 101 sideways.

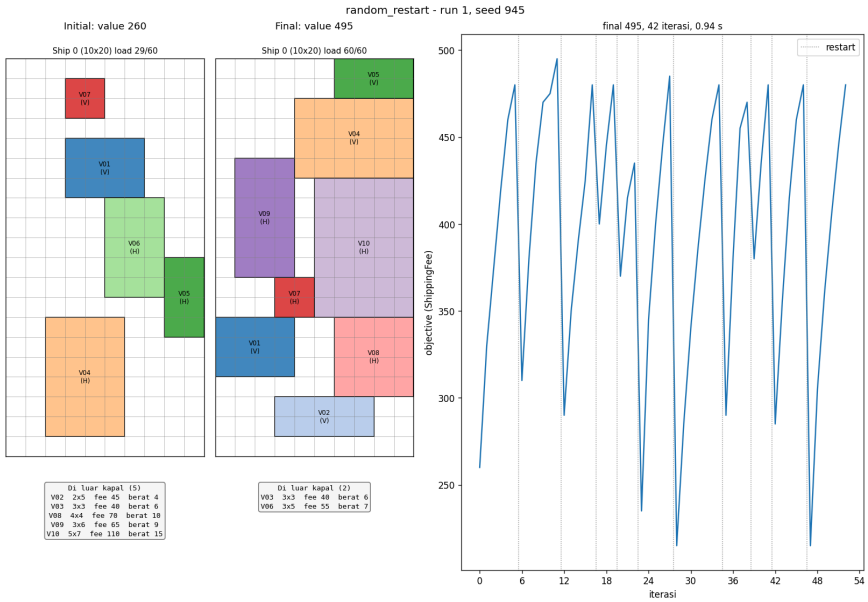
2.2.4. *Random Restart Hill-Climbing*

2.2.4.1. instance_example

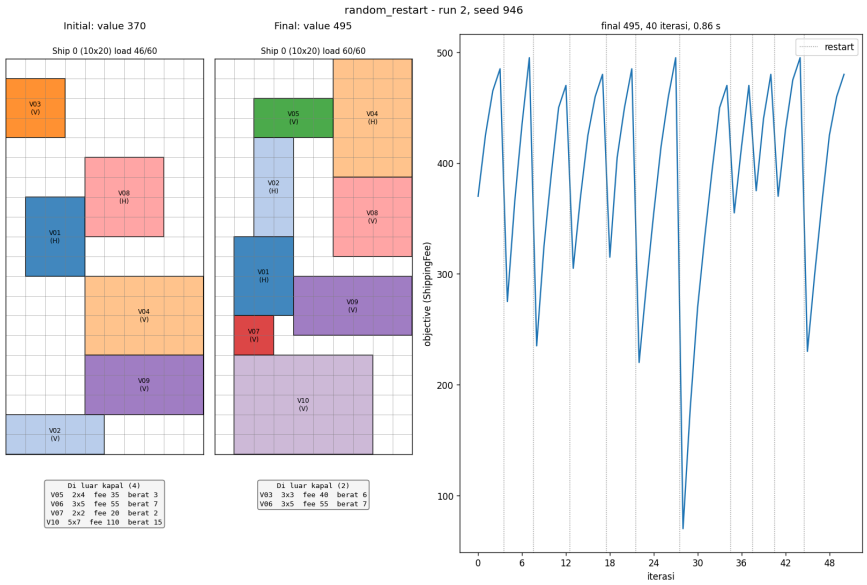
Hasil di bawah berasal dari output/instance_example/random_restart/.

Tabel 2.2.4.1. instance_example *Random Restart Hill-Climbing*

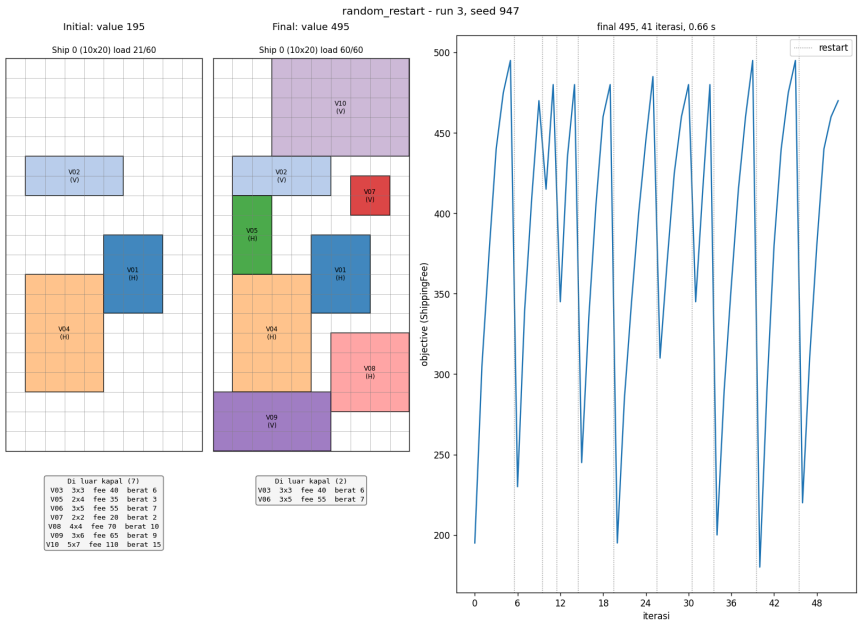
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	260	495	42	0.9383	Restart: 10
2	370	495	40	0.8552	Restart: 10
3	195	495	41	0.6588	Restart: 10



Gambar 2.2.4.1.1. Random restart, seed 945: objective 260 → 495; 42 iterasi; 10 restart.



Gambar 2.2.4.1.2. Random restart, seed 946: objective 370 → 495; 40 iterasi; 10 restart.



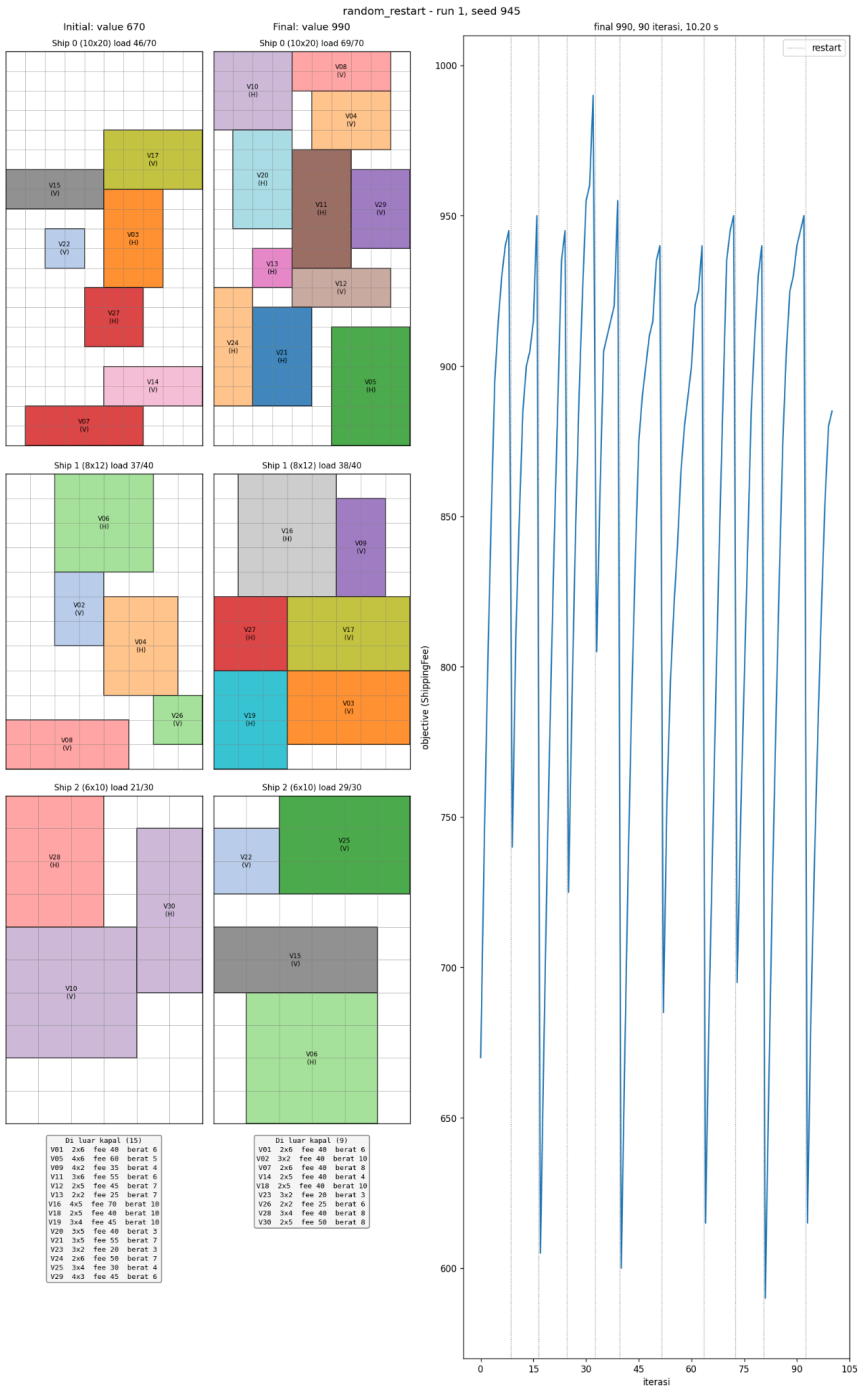
Gambar 2.2.4.1.3. Random restart, seed 947: objective 195 → 495; 41 iterasi; 10 restart.

2.2.4.2. instance_multiship_hard

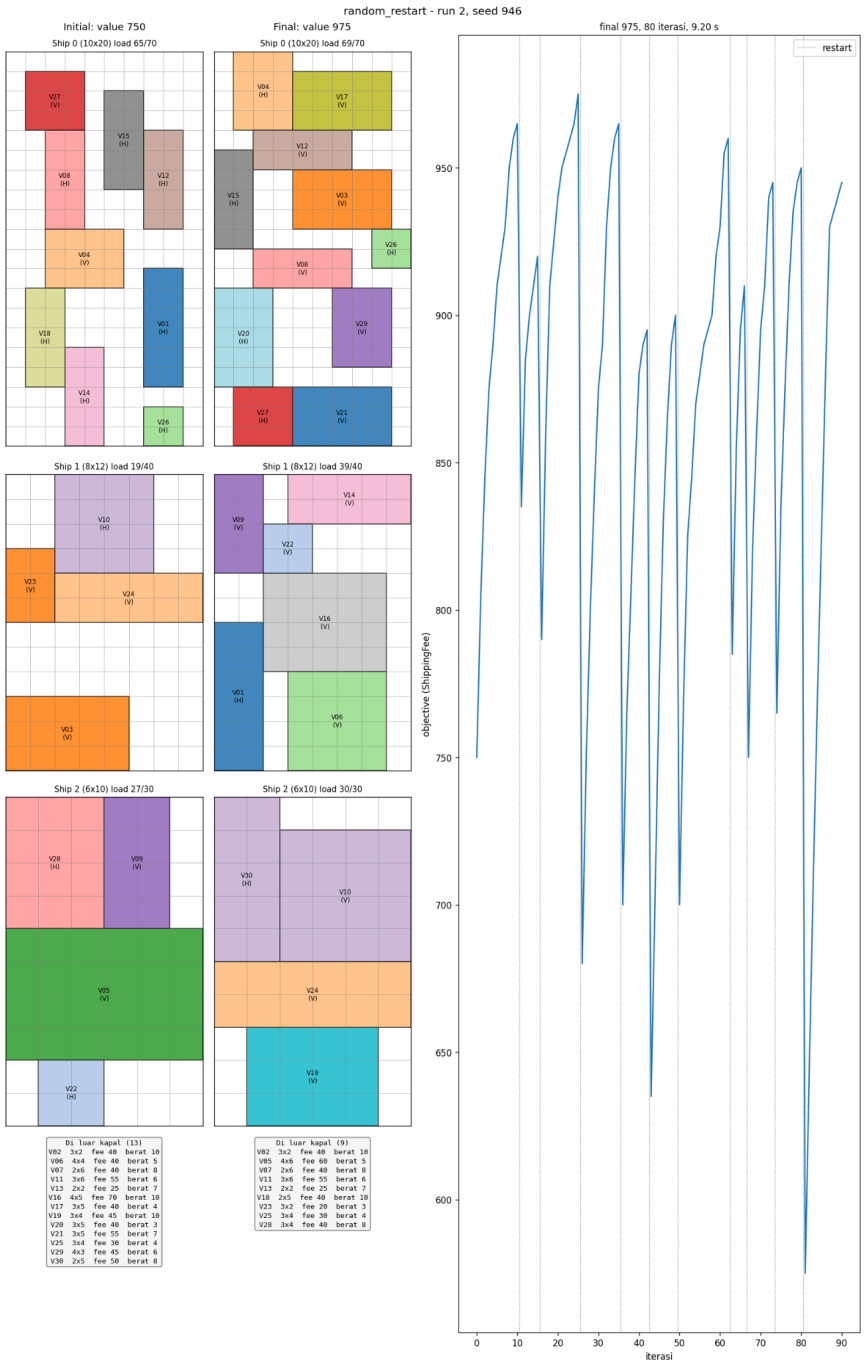
Hasil di bawah berasal dari output/instance_multiship_hard/random_restart/.

Tabel 2.2.4.2. instance_multiship_hard *Random Restart Hill-Climbing*

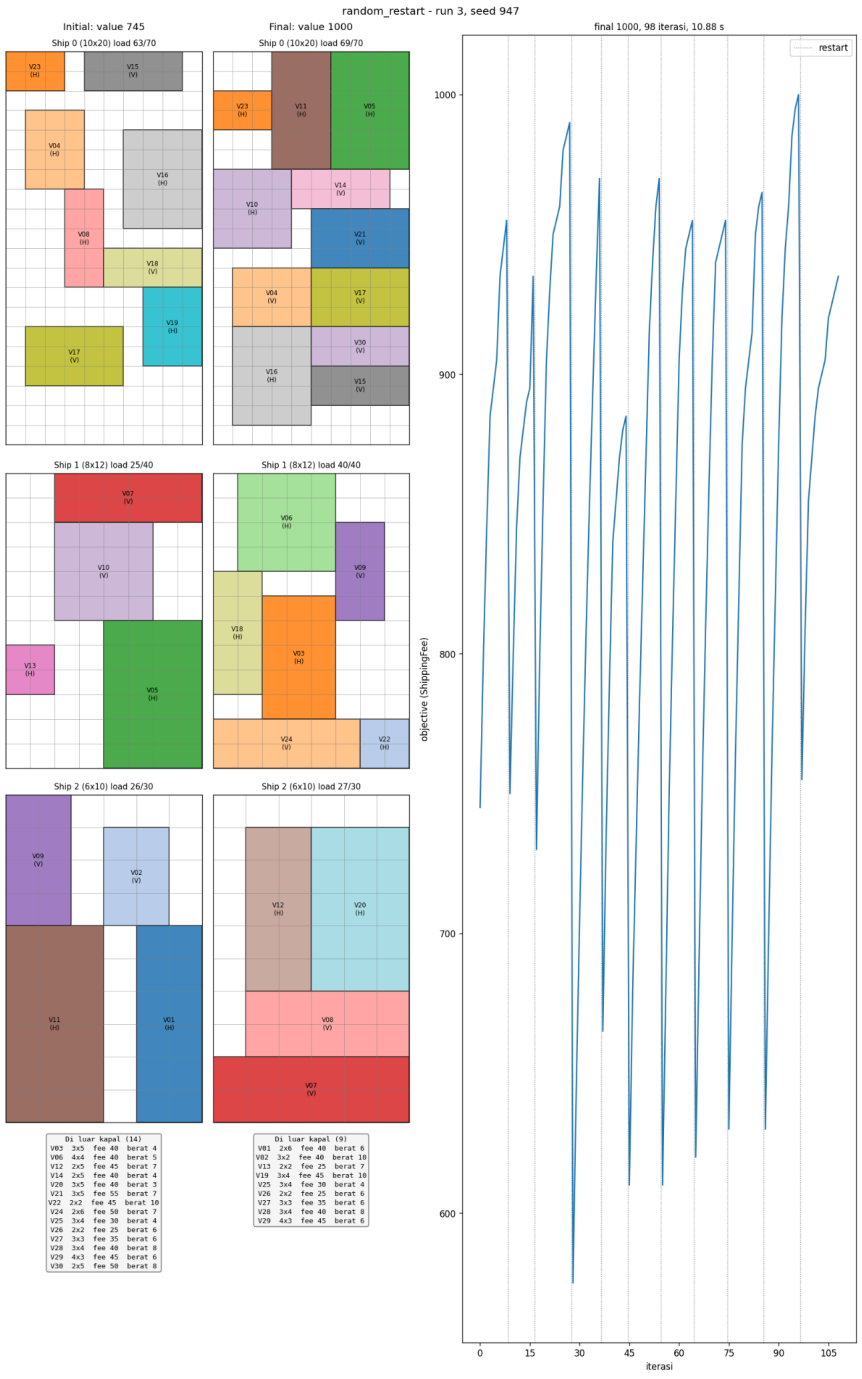
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	670	990	90	10.1957	Restart: 10
2	750	975	80	9.1965	Restart: 10
3	745	1000	98	10.8831	Restart: 10



Gambar 2.2.4.2.1. Random restart, seed 945: objective 670 → 990; 90 iterasi; 10 restart.



Gambar 2.2.4.2.2. Random restart, seed 946: objective 750 → 975; 80 iterasi; 10 restart.



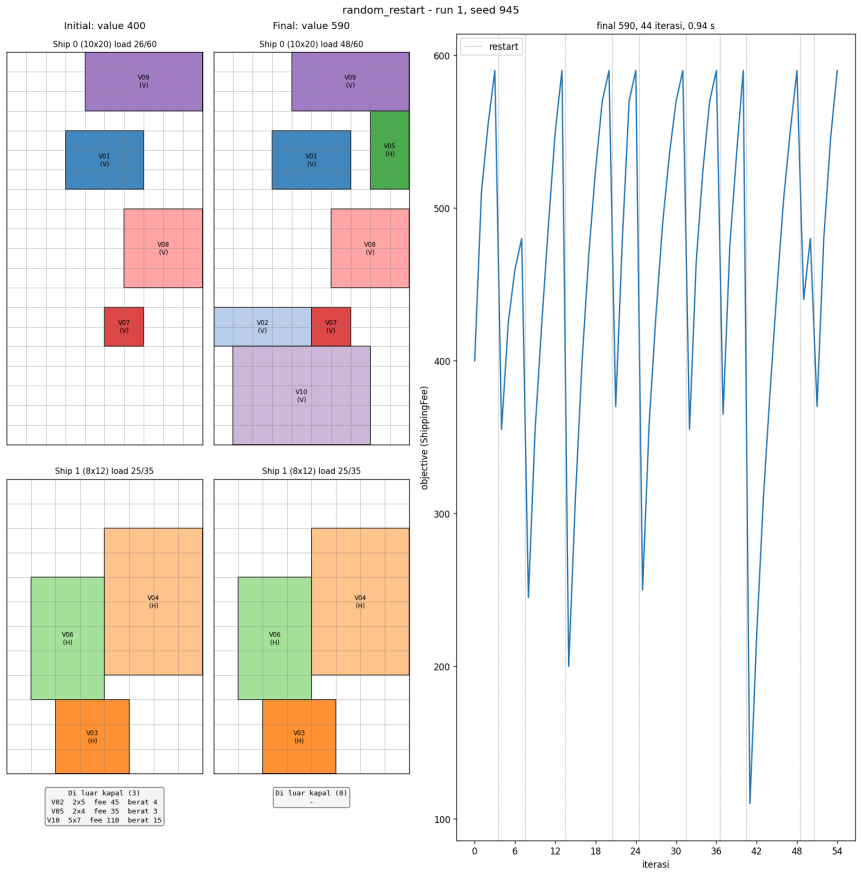
Gambar 2.2.4.2.3. Random restart, seed 947: objective 745 → 1000; 98 iterasi; 10 restart.

2.2.4.3. instance_multiship

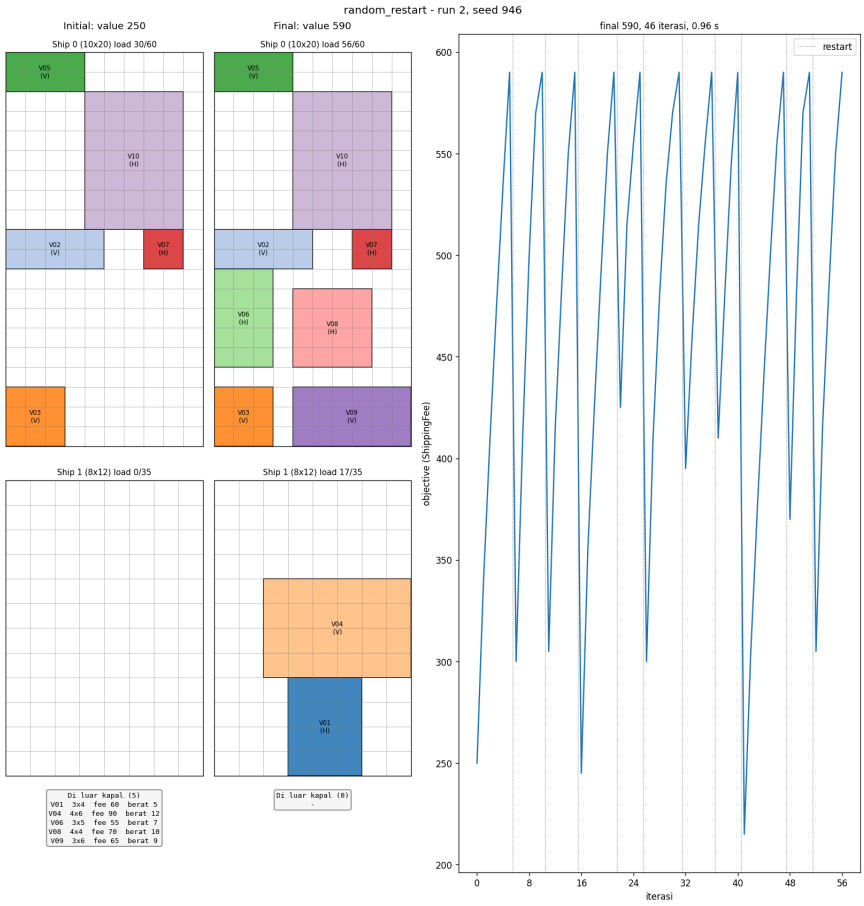
Hasil di bawah berasal dari output/instance_multiship/random_restart/.

Tabel 2.2.4.3. instance_multiship *Random Restart Hill-Climbing*

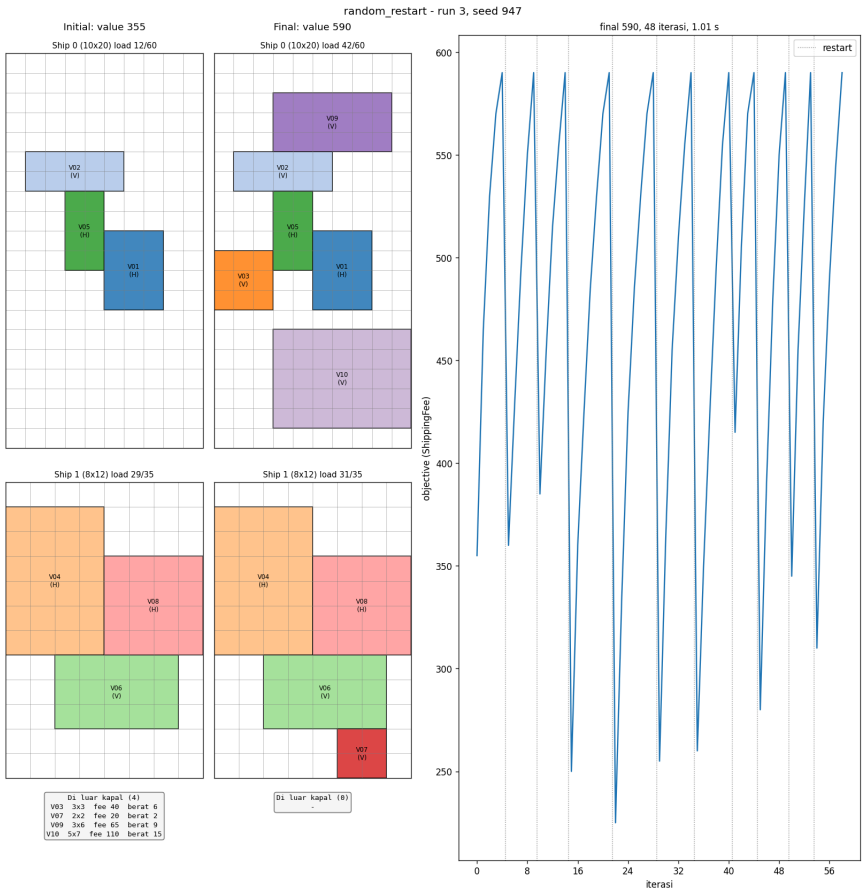
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	400	590	44	0.9439	Restart: 10
2	250	590	46	0.9635	Restart: 10
3	355	590	48	1.0066	Restart: 10



Gambar 2.2.4.3.1. Random restart, seed 945: objective 400 → 590; 44 iterasi; 10 restart.



Gambar 2.2.4.3.2. Random restart, seed 946: objective 250 → 590; 46 iterasi; 10 restart.



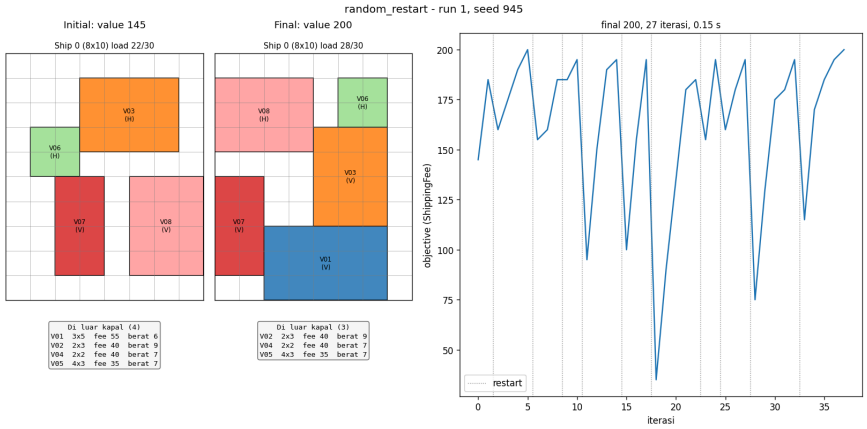
Gambar 2.2.4.3.3. Random restart, seed 947: objective 355 → 590; 48 iterasi; 10 restart.

2.2.4.4. instance_small

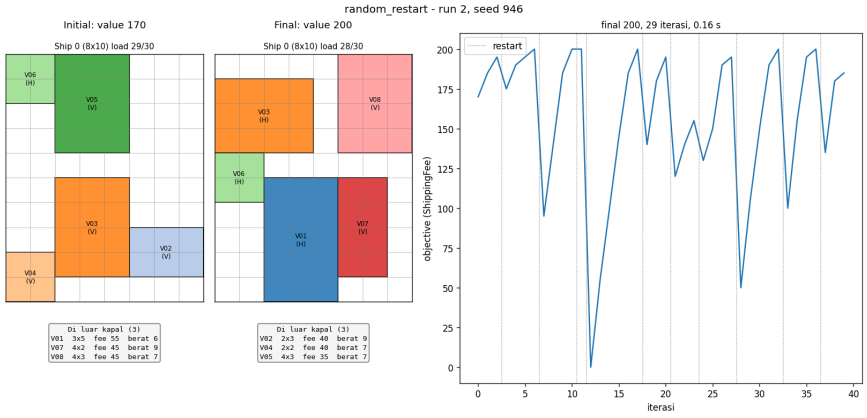
Hasil di bawah berasal dari output/instance_small/random_restart/.

Tabel 2.2.4.4. instance_small *Random Restart Hill-Climbing*

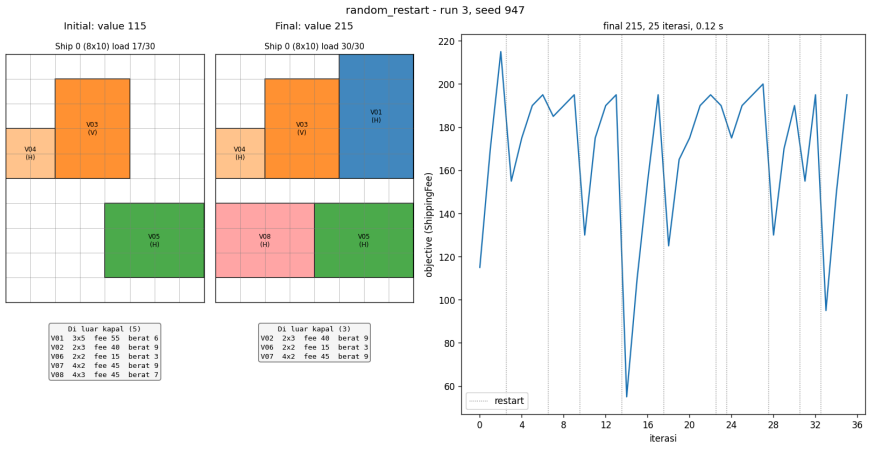
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	145	200	27	0.1502	Restart: 10
2	170	200	29	0.1601	Restart: 10
3	115	215	25	0.1155	Restart: 10



Gambar 2.2.4.4.1. Random restart, seed 945: objective 145 → 200; 27 iterasi; 10 restart.



Gambar 2.2.4.4.2. Random restart, seed 946: objective 170 → 200; 29 iterasi; 10 restart.



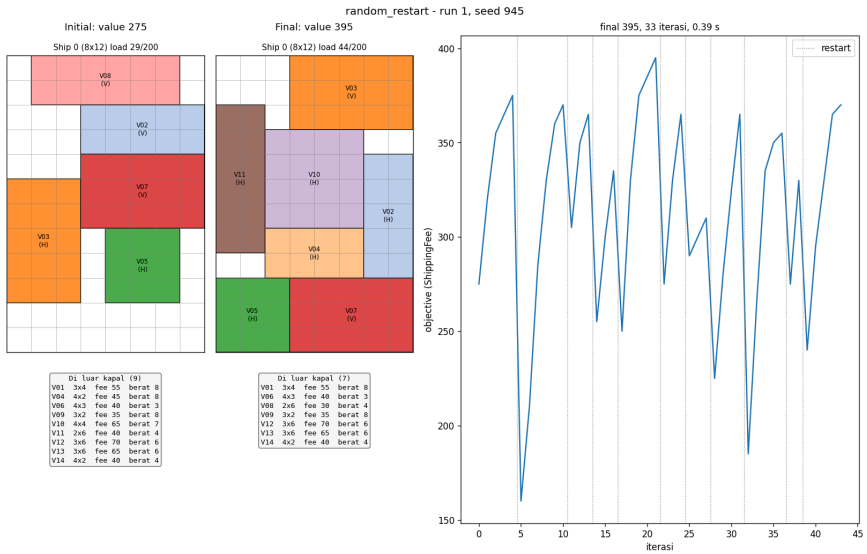
Gambar 2.2.4.4.3. Random restart, seed 947: objective 115 → 215; 25 iterasi; 10 restart.

2.2.4.5. instance_tight_space

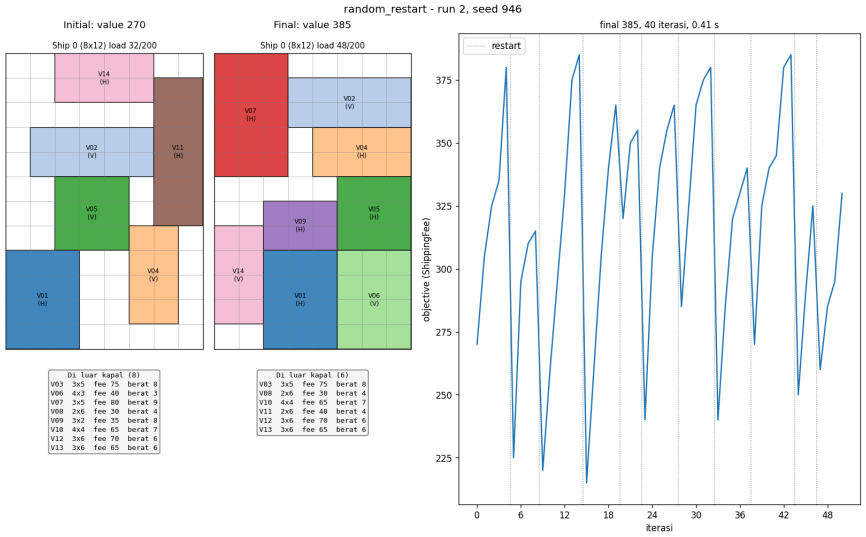
Hasil di bawah berasal dari output/instance_tight_space/random_restart/.

Tabel 2.2.4.5. instance_tight_space *Random Restart Hill-Climbing*

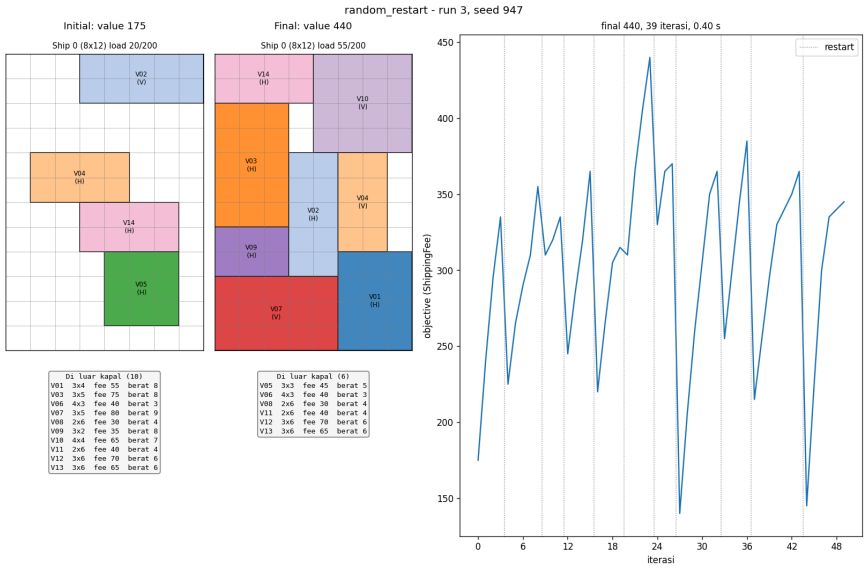
Run	***Objective* Awal**	***Objective* Akhir**	**Banyak Iterasi**	**Durasi (s)**	**Parameter Khusus Varian**
1	275	395	33	0.3903	Restart: 10
2	270	385	40	0.4127	Restart: 10
3	175	440	39	0.3979	Restart: 10



Gambar 2.2.4.5.1. Random restart, seed 945: objective 275 → 395; 33 iterasi; 10 restart.



Gambar 2.2.4.5.2. Random restart, seed 946: objective 270 → 385; 40 iterasi; 10 restart.



Gambar 2.2.4.5.3. Random restart, seed 947: objective 175 → 440; 39 iterasi; 10 restart.

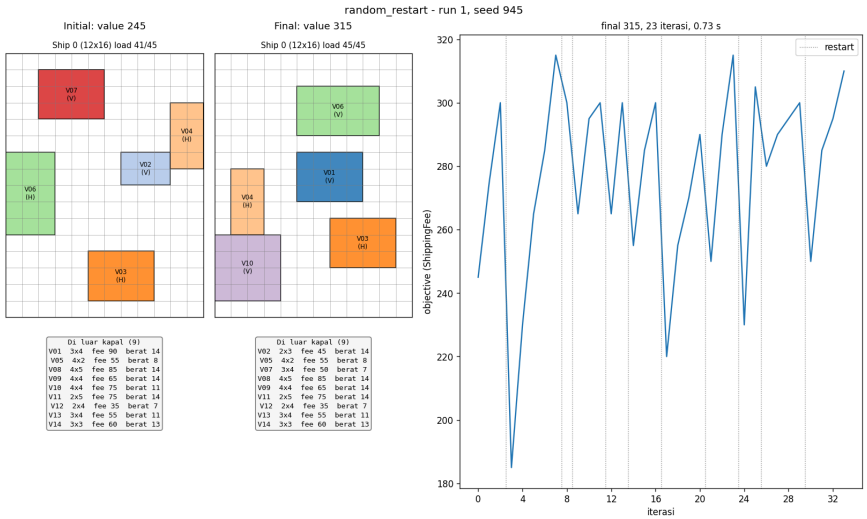
2.2.4.6. instance_tight_weight

Hasil di bawah berasal dari output/instance_tight_weight/random_restart/.

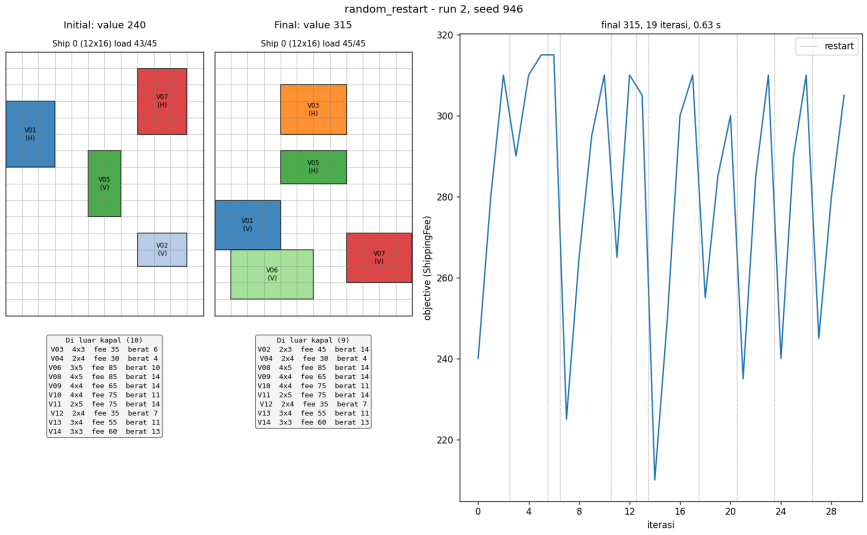
Tabel 2.2.4.6. instance_tight_weight *Random Restart Hill-Climbing*

Run	***Objective***	***Objective***	**Banyak**	**Durasi**	**Parameter**

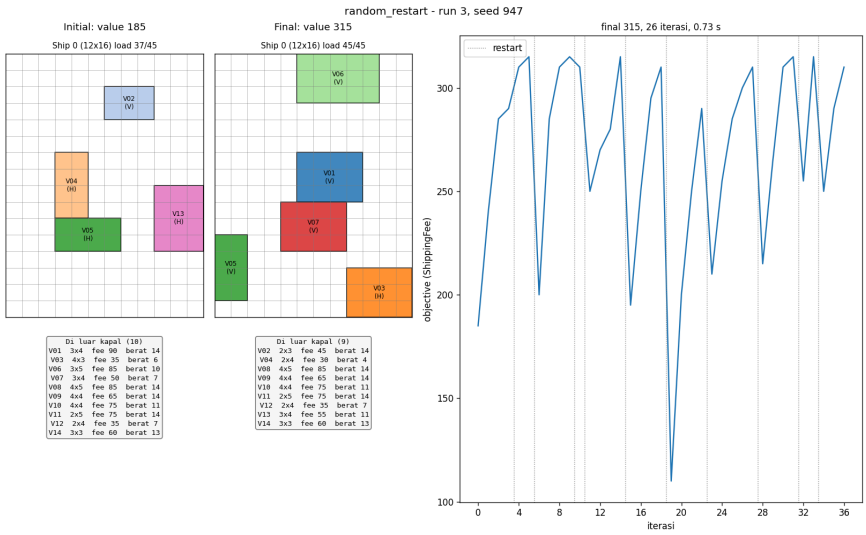
	Awal	Akhir	Iterasi	(s)	Khusus Varian
1	245	315	23	0.7340	Restart: 10
2	240	315	19	0.6253	Restart: 10
3	185	315	26	0.7289	Restart: 10



Gambar 2.2.4.6.1. Random restart, seed 945: objective 245 → 315; 23 iterasi; 10 restart.



Gambar 2.2.4.6.2. Random restart, seed 946: objective 240 → 315; 19 iterasi; 10 restart.



Gambar 2.2.4.6.3. Random restart, seed 947: objective 185 → 315; 26 iterasi; 10 restart.

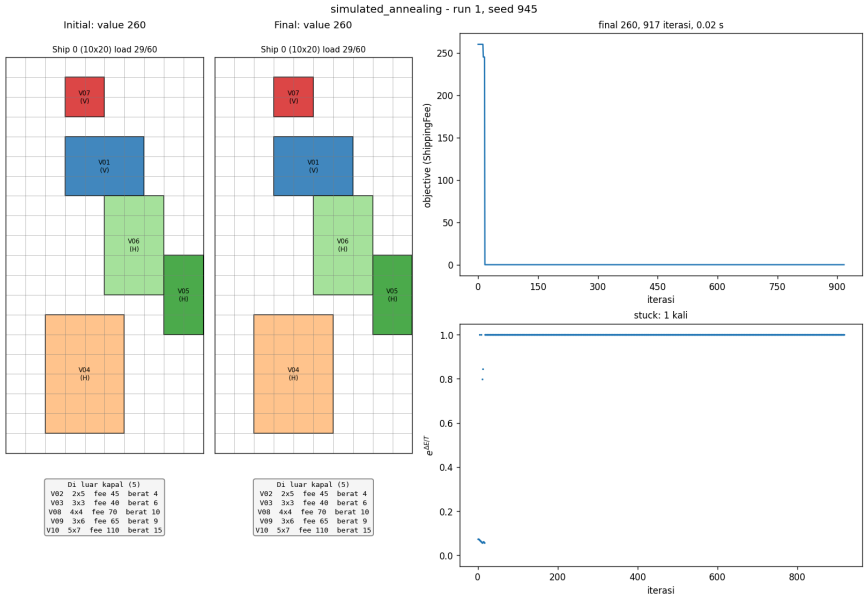
2.2.5. *Simulated Annealing*

2.2.5.1. instance_example

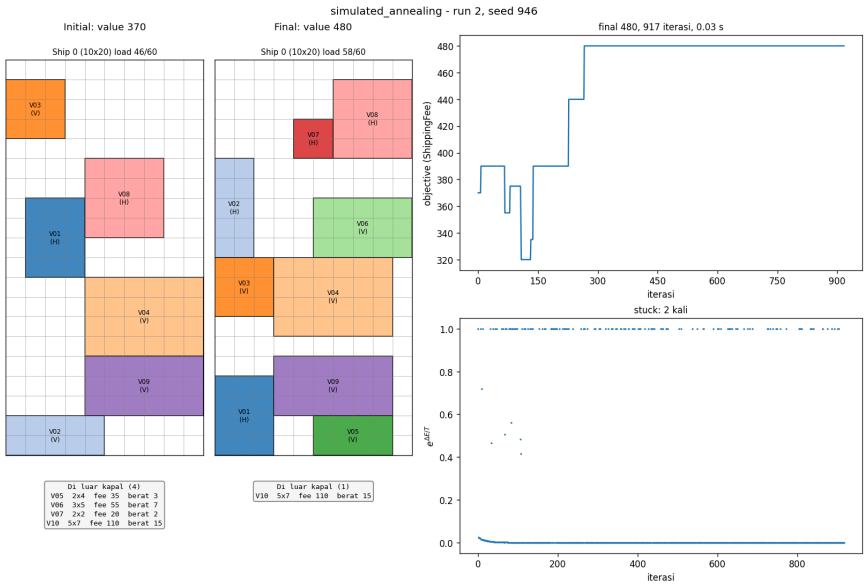
Hasil di bawah berasal dari output/instance_example/simulated_annealing/.

Tabel 2.2.5.1. instance_example *Simulated Annealing*

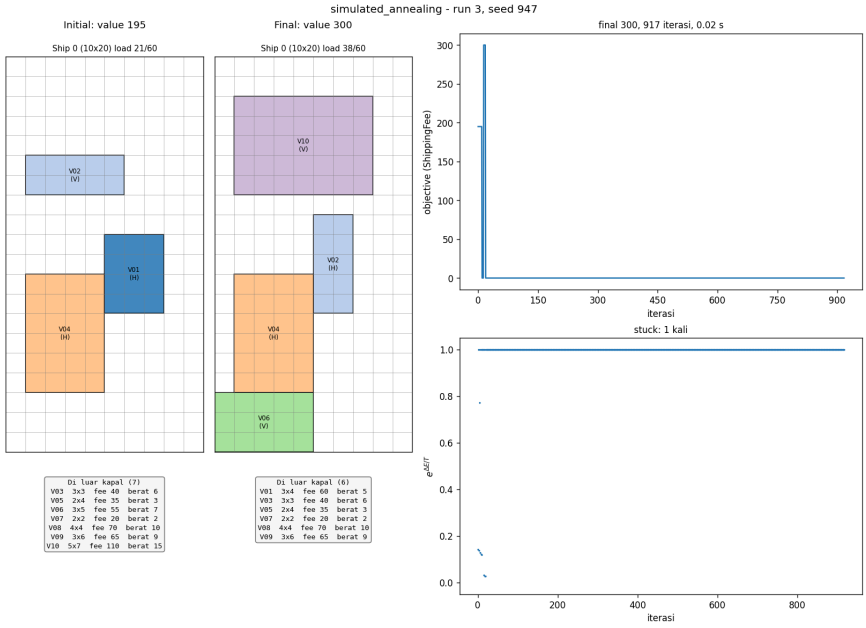
Run	***Objective* Awal**	***Objective* Akhir**	**Durasi (s)**	**Frekuensi *Stuck***
1	260	260	0,0169	1
2	370	480	0,0297	2
3	195	300	0,0160	1



Gambar 2.2.5.1.1. State awal, solusi valid terbaik, dan plot SA pada instance_example, seed 945: objective 260 menjadi 260; stuck 1 episode.



Gambar 2.2.5.1.2. State awal, solusi valid terbaik, dan plot SA pada instance_example, seed 946: objective 370 menjadi 480; stuck 2 episode.



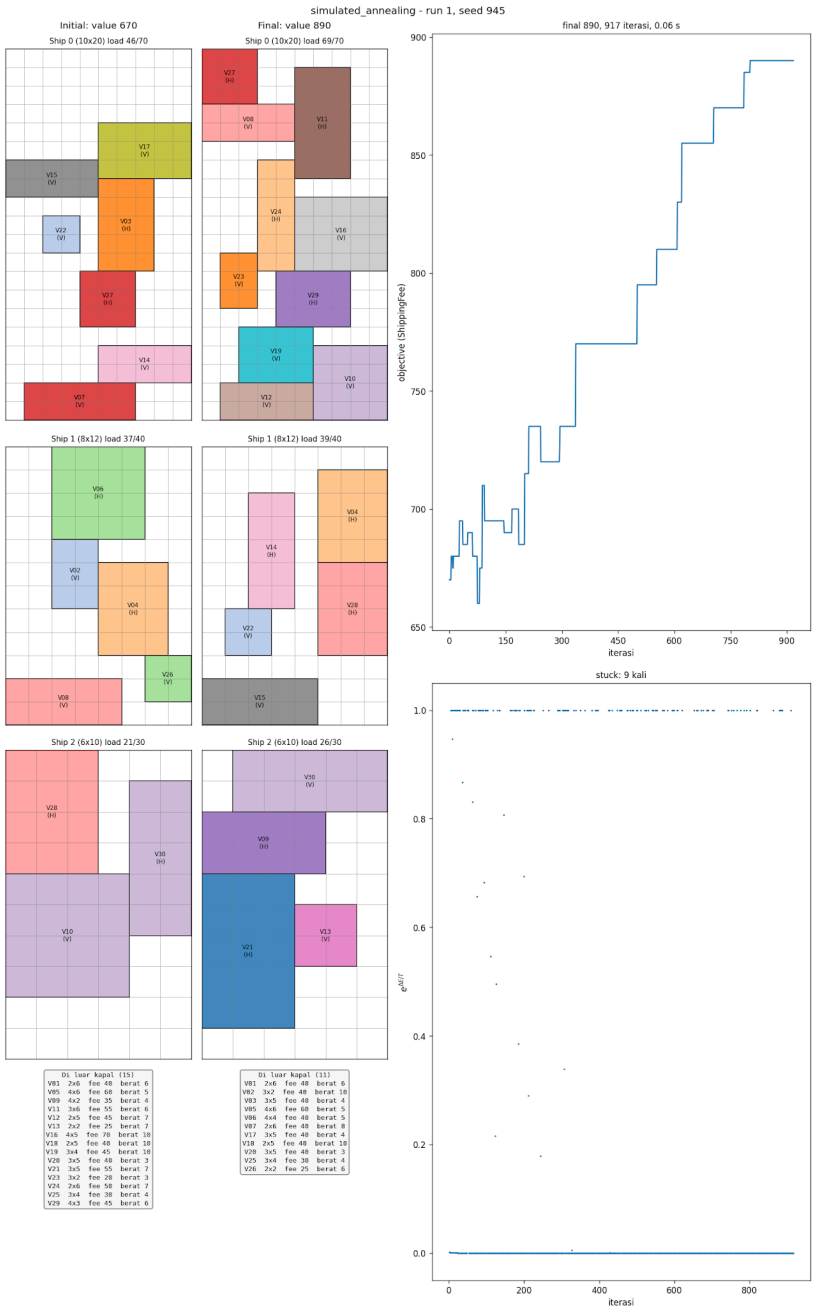
Gambar 2.2.5.1.3. State awal, solusi valid terbaik, dan plot SA pada instance_example, seed 947: objective 195 menjadi 300; stuck 1 episode.

2.2.5.2. instance_multiship_hard

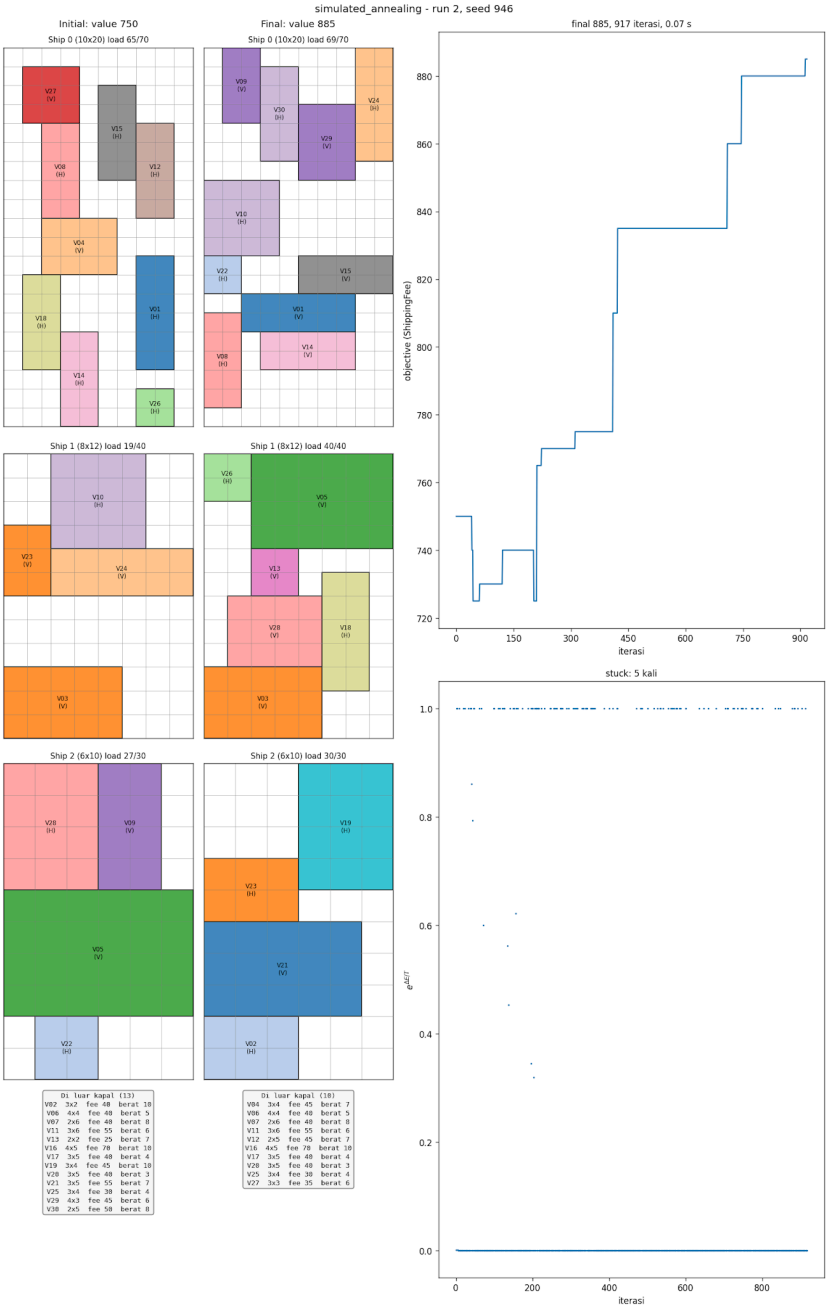
Hasil di bawah berasal dari output/instance_multiship_hard/simulated_annealing/.

Tabel 2.2.5.2. instance_multiship_hard *Simulated Annealing*

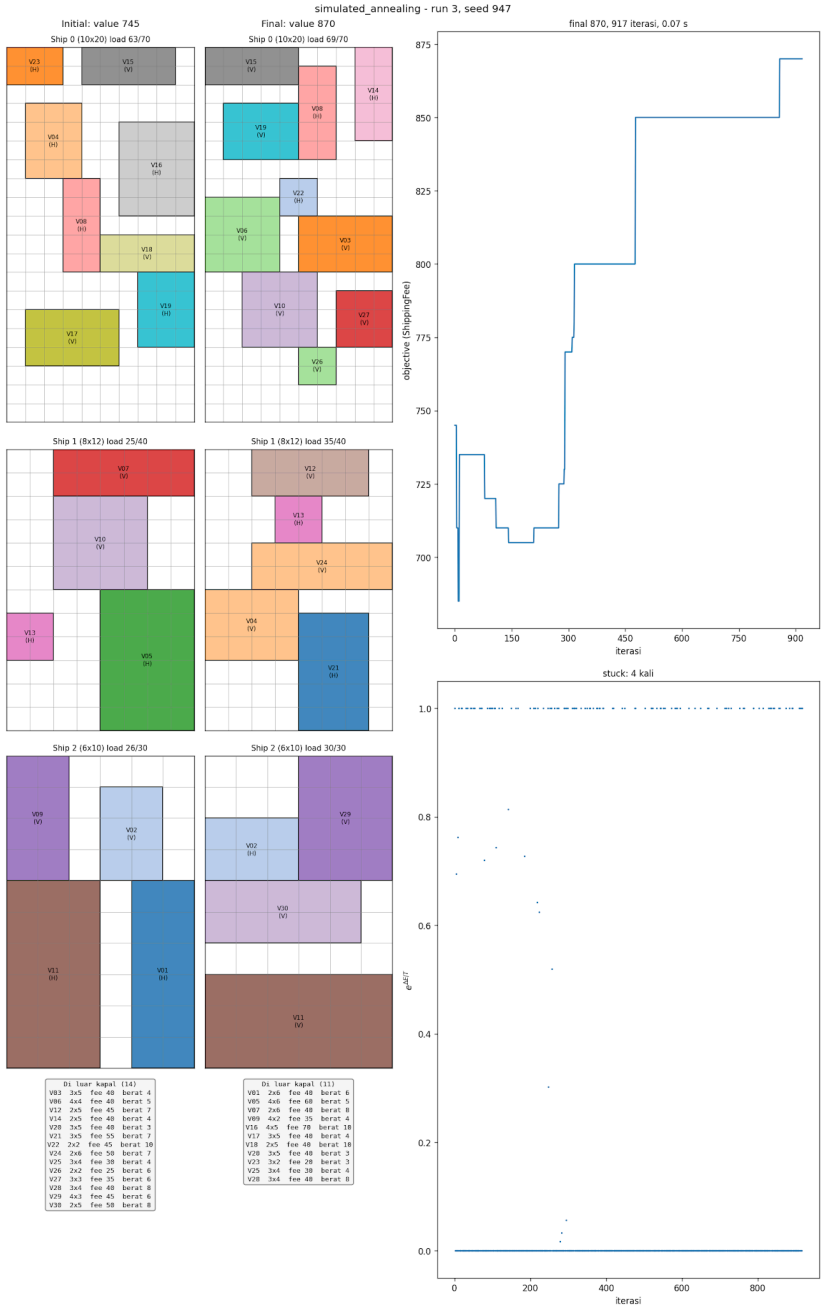
Run	***Objective* Awal**	***Objective* Akhir**	**Durasi (s)**	**Frekuensi *Stuck***
1	670	890	0,0633	9
2	750	885	0,0669	5
3	745	870	0,0661	4



Gambar 2.2.5.2.1. State awal, solusi valid terbaik, dan plot SA pada instance_multiship_hard, seed 945: objective 670 menjadi 890; stuck 9 episode.



Gambar 2.2.5.2.2. State awal, solusi valid terbaik, dan plot SA pada instance_multiship_hard, seed 946: objective 750 menjadi 885; stuck 5 episode.



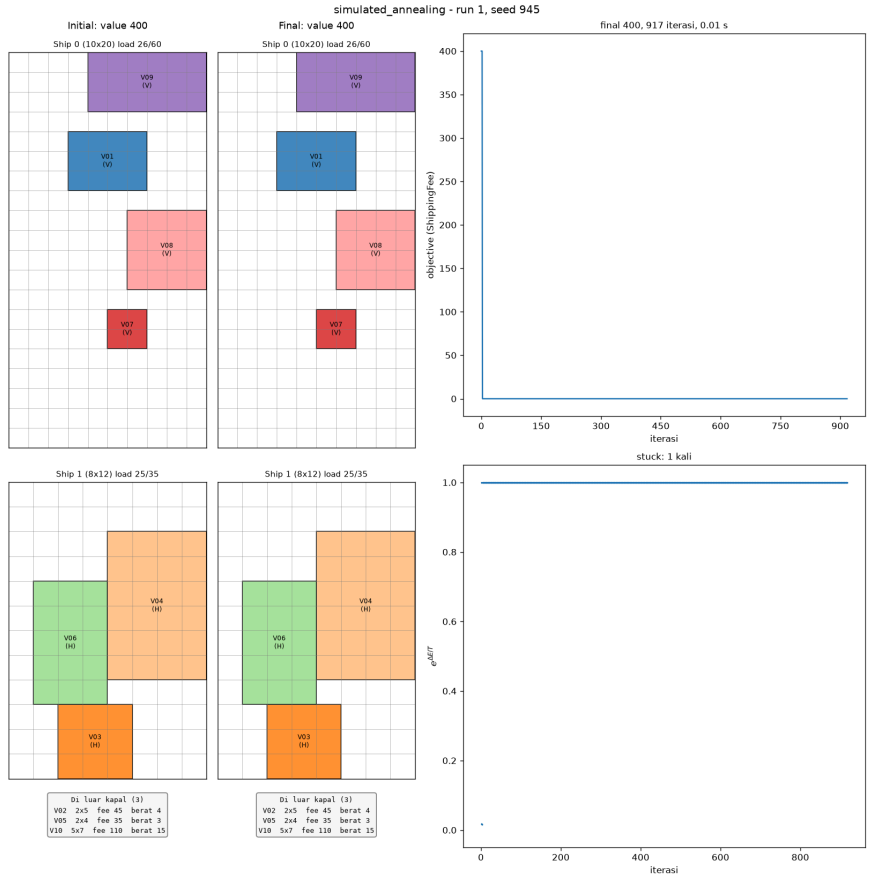
Gambar 2.2.5.2.3. State awal, solusi valid terbaik, dan plot SA pada instance_multiship_hard, seed 947: objective 745 menjadi 870; stuck 4 episode.

2.2.5.3. instance_multiship

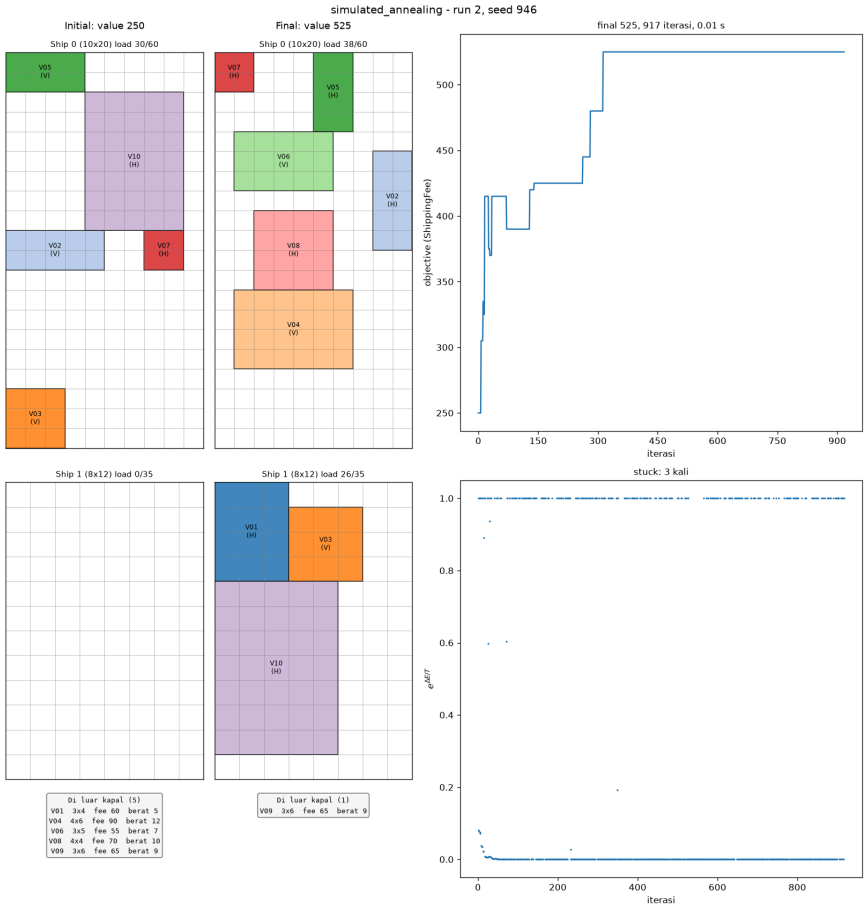
Hasil di bawah berasal dari output/instance_multiship/simulated_annealing/.

Tabel 2.2.5.3. instance_multiship *Simulated Annealing*

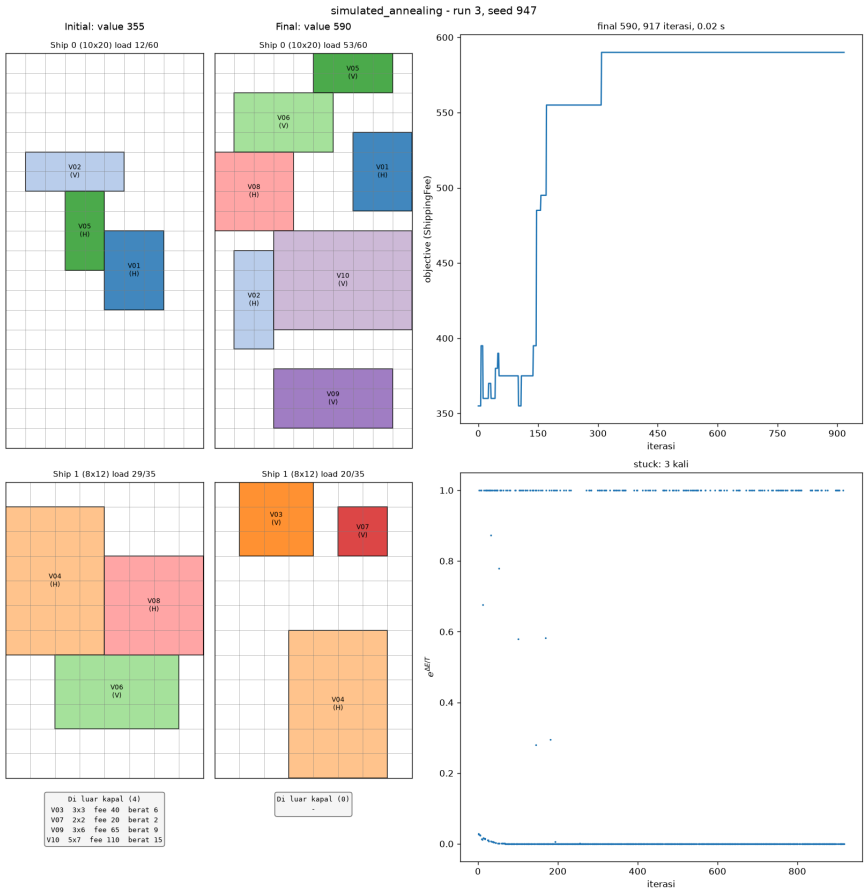
Run	***Objective* Awal**	***Objective* Akhir**	**Durasi (s)**	**Frekuensi *Stuck***
1	400	400	0,0097	1
2	250	525	0,0148	3
3	355	590	0,0176	3



Gambar 2.2.5.3.1. State awal, solusi valid terbaik, dan plot SA pada instance_multiship, seed 945: objective 400 menjadi 400; stuck 1 episode.



Gambar 2.2.5.3.2. State awal, solusi valid terbaik, dan plot SA pada instance_multiship, seed 946: objective 250 menjadi 525; stuck 3 episode.



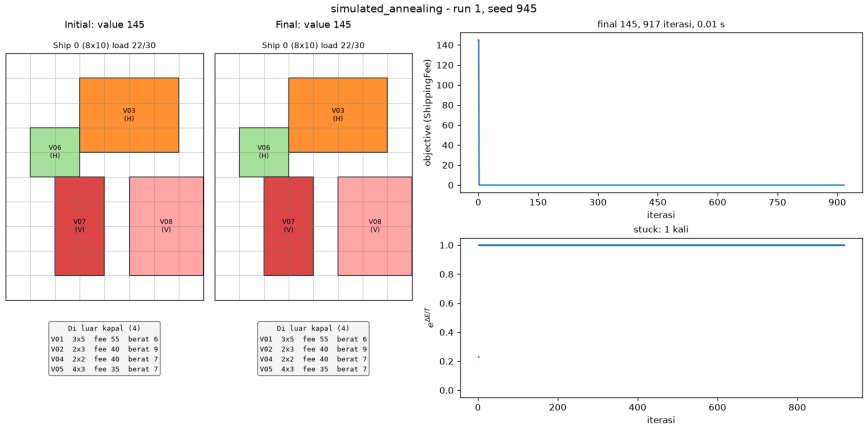
Gambar 2.2.5.3.3. State awal, solusi valid terbaik, dan plot SA pada instance_multiship, seed 947: objective 355 menjadi 590; stuck 3 episode.

2.2.5.4. instance_small

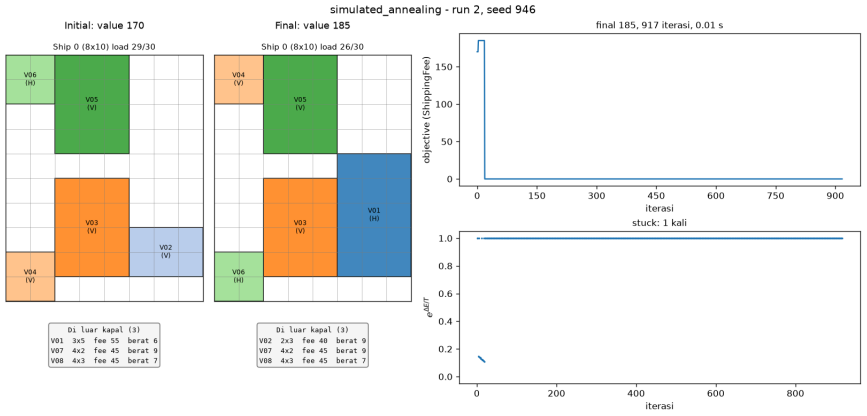
Hasil di bawah berasal dari output/instance_small/simulated_annealing/.

Tabel 2.2.5.4. instance_small *Simulated Annealing*

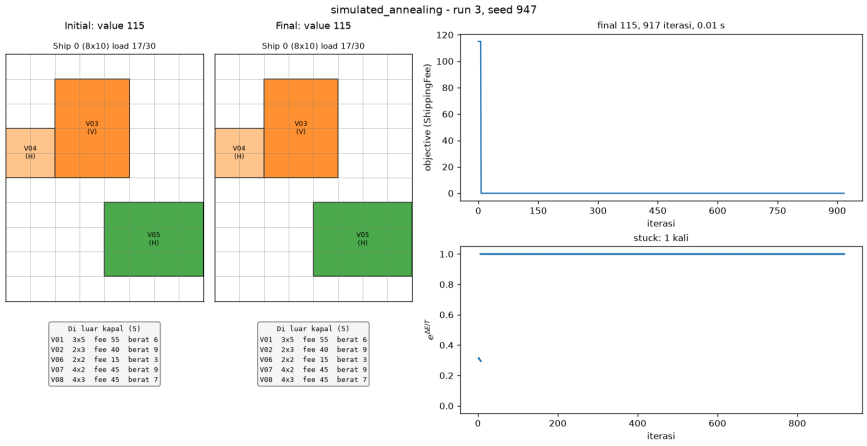
Run	***Objective* Awal**	***Objective* Akhir**	**Durasi (s)**	**Frekuensi *Stuck***
1	145	145	0,0071	1
2	170	185	0,0097	1
3	115	115	0,0076	1



Gambar 2.2.5.4.1. State awal, solusi valid terbaik, dan plot SA pada instance_small, seed 945: objective 145 menjadi 145; stuck 1 episode.



Gambar 2.2.5.4.2. State awal, solusi valid terbaik, dan plot SA pada instance_small, seed 946: objective 170 menjadi 185; stuck 1 episode.



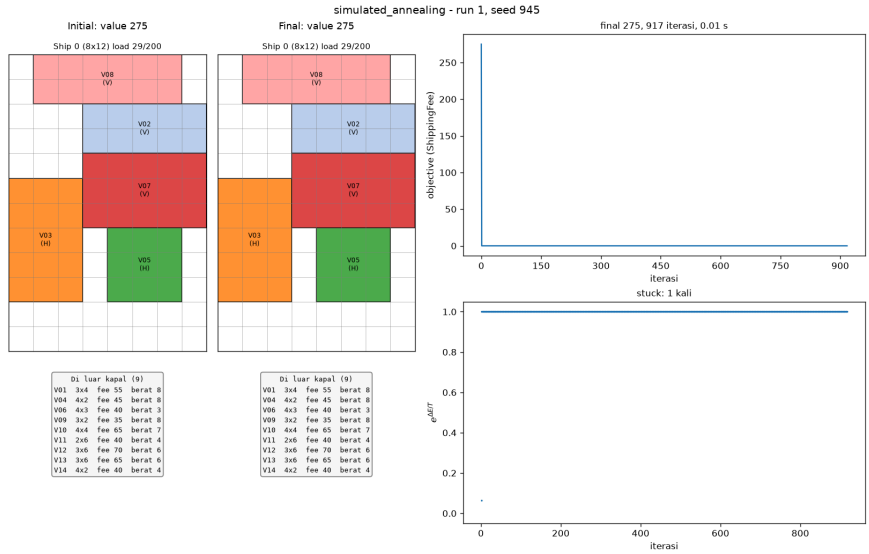
Gambar 2.2.5.4.3. State awal, solusi valid terbaik, dan plot SA pada instance_small, seed 947: objective 115 menjadi 115; stuck 1 episode.

2.2.5.5. instance_tight_space

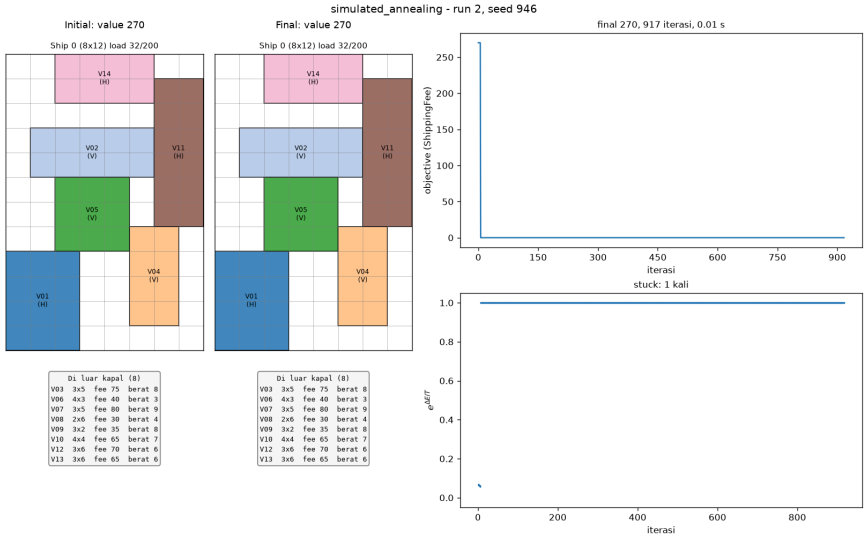
Hasil di bawah berasal dari output/instance_tight_space/simulated_annealing/.

Tabel 2.2.5.5. instance_tight_space *Simulated Annealing*

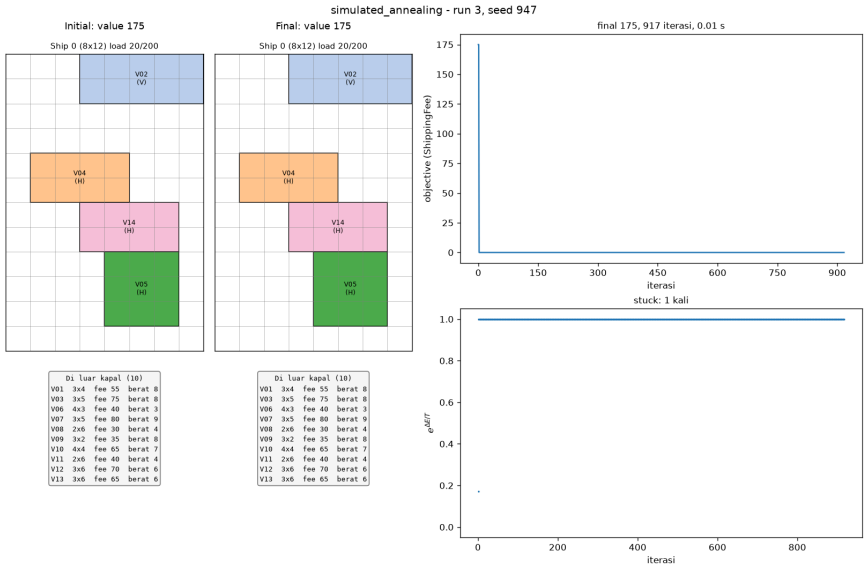
Run	***Objective* Awal**	***Objective* Akhir**	**Durasi (s)**	**Frekuensi *Stuck***
1	275	275	0,0093	1
2	270	270	0,0090	1
3	175	175	0,0086	1



Gambar 2.2.5.5.1. State awal, solusi valid terbaik, dan plot SA pada instance_tight_space, seed 945: objective 275 menjadi 275; stuck 1 episode.



Gambar 2.2.5.5.2. State awal, solusi valid terbaik, dan plot SA pada instance_tight_space, seed 946: objective 270 menjadi 270; stuck 1 episode.



Gambar 2.2.5.5.3. State awal, solusi valid terbaik, dan plot SA pada instance_tight_space, seed 947: objective 175 menjadi 175; stuck 1 episode.

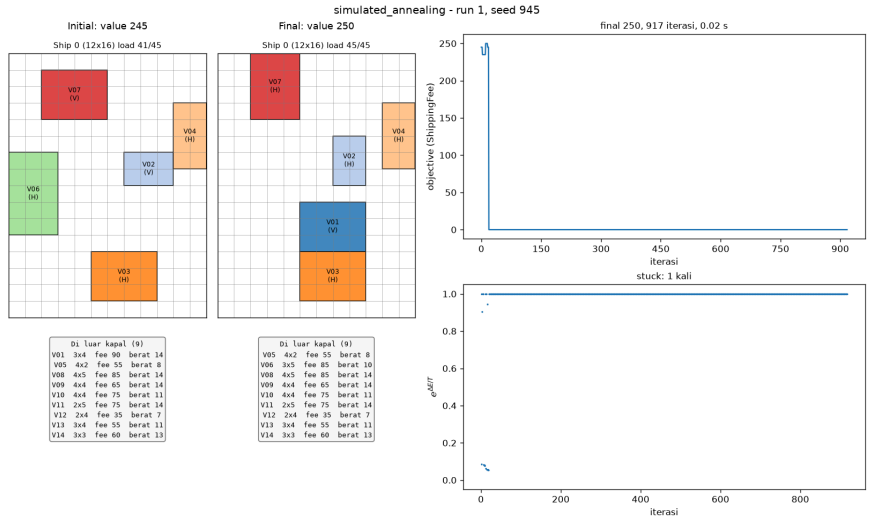
2.2.5.6. instance_tight_weight

Hasil di bawah berasal dari output/instance_tight_weight/simulated_annealing/.

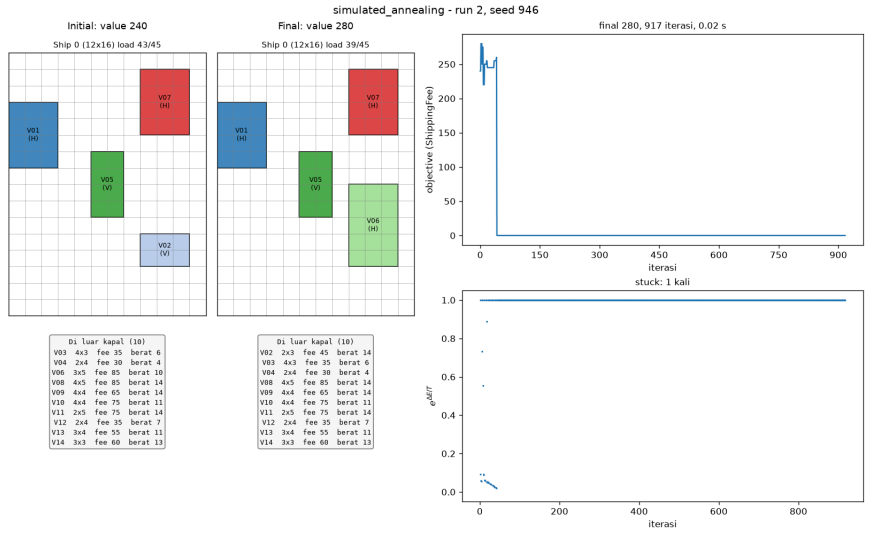
Tabel 2.2.5.6. instance_tight_weight *Simulated Annealing*

Run	***Objective* Awal**	***Objective* Akhir**	**Durasi (s)**	**Frekuensi *Stuck***

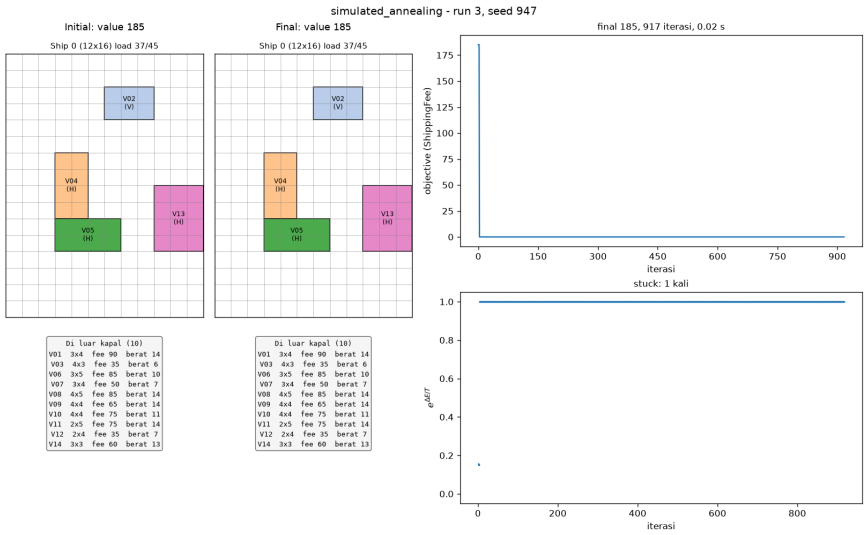
1	245	250	0,0195	1
2	240	280	0,0207	1
3	185	185	0,0198	1



Gambar 2.2.5.6.1. State awal, solusi valid terbaik, dan plot SA pada instance_tight_weight, seed 945: objective 245 menjadi 250; stuck 1 episode.



Gambar 2.2.5.6.2. State awal, solusi valid terbaik, dan plot SA pada instance_tight_weight, seed 946: objective 240 menjadi 280; stuck 1 episode.



Gambar 2.2.5.6.3. State awal, solusi valid terbaik, dan plot SA pada instance_tight_weight, seed 947: objective 185 menjadi 185; stuck 1 episode.

2.2.6. *Genetic Algorithm*

2.2.6.1. instance_example

Hasil di bawah berasal dari 15 file di output/instance_example/genetic_algorithm/. Instance ini memiliki 10 kendaraan dan 1 kapal.

Percobaan 1: Populasi sebagai Kontrol

Jumlah populasi tetap 50, banyak iterasi divariasikan (50, 100, 200).

Tabel 2.2.6.1.1. instance_example *Genetic Algorithm*, Percobaan 1

Populasi	Iterasi	*Run*	*Objective* Awal	*Objective* Akhir	Durasi (s)
50	50	1	260	410	0,09
50	50	2	370	460	0,07
50	50	3	195	425	0,06

50	100	1	260	465	0,13
50	100	2	370	485	0,11
50	100	3	195	485	0,12
50	200	1	260	465	0,24
50	200	2	370	485	0,24
50	200	3	195	485	0,27

Tabel 2.2.6.1.1b. Ringkasan Percobaan 1 instance_example

Populasi	**Iterasi**	***Rata-rata Objective Akhir***	**Min - Maks**	**Rata-rata Durasi (s)**	**Iterasi Pertama Mencapai Nilai Akhir (*run* 1, 2, 3)**
50	50	431,7	410 - 460	0,07	11, 19, 24
50	100	478,3	465 - 485	0,12	92, 93, 57
50	200	478,3	465 - 485	0,25	92, 93, 57

Percobaan 2: Iterasi sebagai Kontrol

Banyak iterasi tetap 100, jumlah populasi divariasikan (20, 50, 100). Konfigurasi populasi 50 dan iterasi 100 adalah *run* yang sama dengan Percobaan 1.

Tabel 2.2.6.1.2. instance_example *Genetic Algorithm*, Percobaan 2

Populasi	**Iterasi**	***Run***	***Objective* Awal**	***Objective* Akhir**	**Durasi (s)**
20	100	1	260	435	0,04

20	100	2	370	485	0,05
20	100	3	195	495	0,05
50	100	1	260	465	0,13
50	100	2	370	485	0,11
50	100	3	195	485	0,12
100	100	1	260	480	0,27
100	100	2	370	485	0,24
100	100	3	195	480	0,24

Tabel 2.2.6.1.2b. Ringkasan Percobaan 2 instance_example

Populasi	**Iterasi**	***Rata-rata Objective Akhir***	**Min - Maks**	**Rata-rata Durasi (s)**	**Iterasi Pertama Mencapai Nilai Akhir (*run* 1, 2, 3)**
20	100	471,7	435 - 495	0,05	81, 49, 86
50	100	478,3	465 - 485	0,12	92, 93, 57
100	100	481,7	480 - 485	0,25	50, 70, 83

2.2.6.2. instance_multiship_hard

Hasil di bawah berasal dari 15 *file* di output/instance_multiship_hard/genetic_algorithm/.
Instance ini memiliki 30 kendaraan dan 3 kapal.

Percobaan 1: Populasi sebagai Kontrol

Jumlah populasi tetap 50, banyak iterasi divariasikan (50, 100, 200).

Tabel 2.2.6.2.1. instance_multiship_hard *Genetic Algorithm*, Percobaan 1

Populasi	Iterasi	*Run*	*Objective* Awal	*Objective* Akhir	Durasi (s)
50	50	1	670	905	0,30
50	50	2	750	825	0,32
50	50	3	745	885	0,21
50	100	1	670	905	0,43
50	100	2	750	830	0,38
50	100	3	745	890	0,39
50	200	1	670	920	0,91
50	200	2	750	835	0,80
50	200	3	745	895	0,78

Tabel 2.2.6.2.1b. Ringkasan Percobaan 1 instance_multiship_hard

Populasi	Iterasi	*Rata-rata Objective Akhir*	Min - Maks	Rata-rata Durasi (s)	Iterasi Pertama Mencapai Nilai Akhir (*run* 1, 2, 3)
50	50	871,7	825 - 905	0,28	36, 0, 45
50	100	875,0	830 - 905	0,40	36, 85, 54
50	200	883,3	835 - 920	0,83	182, 122, 162

Percobaan 2: Iterasi sebagai Kontrol

Banyak iterasi tetap 100, jumlah populasi divariasikan (20, 50, 100). Konfigurasi populasi 50 dan iterasi 100 adalah *run* yang sama dengan Percobaan 1.

Tabel 2.2.6.2.2. instance_multiship_hard *Genetic Algorithm*, Percobaan 2

Populasi	**Iterasi**	***Run***	***Objective* Awal**	***Objective* Akhir**	**Durasi (s)**
20	100	1	670	870	0,17
20	100	2	750	840	0,16
20	100	3	745	815	0,16
50	100	1	670	905	0,43
50	100	2	750	830	0,38
50	100	3	745	890	0,39
100	100	1	670	890	0,82
100	100	2	750	835	0,70
100	100	3	745	865	0,83

Tabel 2.2.6.2.2b. Ringkasan Percobaan 2 instance_multiship_hard

Populasi	**Iterasi**	***Rata-rata Objective Akhir***	**Min - Maks**	**Rata-rata Durasi (s)**	**Iterasi Pertama Mencapai Nilai Akhir (*run* 1, 2, 3)**
20	100	841,7	815 - 870	0,16	80, 45, 37
50	100	875,0	830 - 905	0,40	36, 85, 54
100	100	863,3	835 -	0,78	0, 0, 0

			890		

2.2.6.3. instance_multiship

Hasil di bawah berasal dari 15 *file* di output/instance_multiship/genetic_algorithm/. *Instance* ini memiliki 10 kendaraan dan 2 kapal. Nilai 590 dicapai pada 6 dari 15 *run* dan pada semuanya seluruh kendaraan terangkut, jadi 590 adalah total *fee* semua kendaraan.

Percobaan 1: Populasi sebagai Kontrol

Jumlah populasi tetap 50, banyak iterasi divariasikan (50, 100, 200).

Tabel 2.2.6.3.1. instance_multiship *Genetic Algorithm*, Percobaan 1

Populasi	**Iterasi**	***Run***	***Objective* Awal**	***Objective* Akhir**	**Durasi (s)**
50	50	1	400	480	0,07
50	50	2	250	535	0,11
50	50	3	355	590	0,10
50	100	1	400	480	0,12
50	100	2	250	550	0,11
50	100	3	355	590	0,13
50	200	1	400	525	0,25
50	200	2	250	590	0,24
50	200	3	355	590	0,27

Tabel 2.2.6.3.1b. Ringkasan Percobaan 1 instance_multiship

Populasi	**Iterasi**	***Rata-***	**Min -**	**Rata-**	**Iterasi**

		rata Objective Akhir	Maks	rata Durasi (s)	Pertama Mencapai Nilai Akhir (*run* 1, 2, 3)
50	50	535,0	480 - 590	0,09	0, 18, 46
50	100	540,0	480 - 590	0,12	0, 61, 46
50	200	568,3	525 - 590	0,25	186, 131, 46

Percobaan 2: Iterasi sebagai Kontrol

Banyak iterasi tetap 100, jumlah populasi divariasikan (20, 50, 100). Konfigurasi populasi 50 dan iterasi 100 adalah *run* yang sama dengan Percobaan 1.

Tabel 2.2.6.3.2. instance_multiship *Genetic Algorithm*, Percobaan 2

Populasi	Iterasi	*Run*	*Objective* Awal	*Objective* Akhir	Durasi (s)
20	100	1	400	570	0,05
20	100	2	250	535	0,05
20	100	3	355	475	0,04
50	100	1	400	480	0,12
50	100	2	250	550	0,11
50	100	3	355	590	0,13
100	100	1	400	535	0,26
100	100	2	250	590	0,25
100	100	3	355	590	0,29

Tabel 2.2.6.3.2b. Ringkasan Percobaan 2 instance_multiship

Populasi	Iterasi	*Rata-rata Objective Akhir*	Min - Maks	Rata-rata Durasi (s)	Iterasi Pertama Mencapai Nilai Akhir (*run* 1, 2, 3)
20	100	526,7	475 - 570	0,05	84, 10, 63
50	100	540,0	480 - 590	0,12	0, 61, 46
100	100	571,7	535 - 590	0,27	0, 33, 0

2.2.6.4. instance_small

Hasil di bawah berasal dari 15 *file* di output/instance_small/genetic_algorithm/. *Instance* ini memiliki 8 kendaraan dan 1 kapal. Semua *run* menyisakan 3 dari 8 kendaraan.

Percobaan 1: Populasi sebagai Kontrol

Jumlah populasi tetap 50, banyak iterasi divariasikan (50, 100, 200).

Tabel 2.2.6.4.1. instance_small *Genetic Algorithm*, Percobaan 1

Populasi	Iterasi	*Run*	*Objective* Awal	*Objective* Akhir	Durasi (s)
50	50	1	145	195	0,04
50	50	2	170	190	0,04
50	50	3	115	195	0,04
50	100	1	145	195	0,08
50	100	2	170	195	0,08

50	100	3	115	200	0,08
50	200	1	145	195	0,15
50	200	2	170	195	0,16
50	200	3	115	200	0,16

Tabel 2.2.6.4.1b. Ringkasan Percobaan 1 instance_small

Populasi	**Iterasi**	***Rata-rata Objective Akhir***	**Min - Maks**	**Rata-rata Durasi (s)**	**Iterasi Pertama Mencapai Nilai Akhir (*run* 1, 2, 3)**
50	50	193,3	190 - 195	0,04	0, 0, 0
50	100	196,7	195 - 200	0,08	0, 54, 61
50	200	196,7	195 - 200	0,16	0, 54, 61

Percobaan 2: Iterasi sebagai Kontrol

Banyak iterasi tetap 100, jumlah populasi divariasikan (20, 50, 100). Konfigurasi populasi 50 dan iterasi 100 adalah *run* yang sama dengan Percobaan 1.

Tabel 2.2.6.4.2. instance_small *Genetic Algorithm*, Percobaan 2

Populasi	**Iterasi**	***Run***	***Objective* Awal**	***Objective* Akhir**	**Durasi (s)**
20	100	1	145	195	0,03
20	100	2	170	195	0,04
20	100	3	115	190	0,04
50	100	1	145	195	0,08

50	100	2	170	195	0,08
50	100	3	115	200	0,09
100	100	1	145	195	0,18
100	100	2	170	195	0,17
100	100	3	115	215	0,17

Tabel 2.2.6.4.2b. Ringkasan Percobaan 2 instance_small

Populasi	**Iterasi**	***Rata-rata Objective Akhir***	**Min - Maks**	**Rata-rata Durasi (s)**	**Iterasi Pertama Mencapai Nilai Akhir (*run* 1, 2, 3)**
20	100	193,3	190 - 195	0,04	80, 36, 6
50	100	196,7	195 - 200	0,08	0, 54, 61
100	100	201,7	195 - 215	0,17	0, 14, 0

2.2.6.5. instance_tight_space

Hasil di bawah berasal dari 15 *file* di output/instance_tight_space/genetic_algorithm/. *Instance* ini memiliki 14 kendaraan dan 1 kapal.

Percobaan 1: Populasi sebagai Kontrol

Jumlah populasi tetap 50, banyak iterasi divariasikan (50, 100, 200).

Tabel 2.2.6.5.1. instance_tight_space *Genetic Algorithm*, Percobaan 1

Populasi	**Iterasi**	***Run***	***Objective***	***Objective***	**Durasi**

			Awal	Akhir	(s)
50	50	1	275	385	0,08
50	50	2	270	340	0,07
50	50	3	175	345	0,07
50	100	1	275	385	0,15
50	100	2	270	345	0,13
50	100	3	175	345	0,13
50	200	1	275	385	0,28
50	200	2	270	345	0,27
50	200	3	175	365	0,27

Tabel 2.2.6.5.1b. Ringkasan Percobaan 1 instance_tight_space

Populasi	Iterasi	*Rata-rata Objective Akhir*	Min - Maks	Rata-rata Durasi (s)	Iterasi Pertama Mencapai Nilai Akhir (*run* 1, 2, 3)
50	50	356,7	340 - 385	0,07	37, 0, 24
50	100	358,3	345 - 385	0,14	37, 89, 24
50	200	365,0	345 - 385	0,27	37, 89, 191

Percobaan 2: Iterasi sebagai Kontrol

Banyak iterasi tetap 100, jumlah populasi divariasikan (20, 50, 100). Konfigurasi populasi 50 dan iterasi 100 adalah *run* yang sama dengan Percobaan 1.

Tabel 2.2.6.5.2. instance_tight_space *Genetic Algorithm*, Percobaan 2

Populasi	Iterasi	*Run*	*Objective* Awal	*Objective* Akhir	Durasi (s)
20	100	1	275	370	0,10
20	100	2	270	340	0,10
20	100	3	175	345	0,07
50	100	1	275	385	0,15
50	100	2	270	345	0,13
50	100	3	175	345	0,13
100	100	1	275	375	0,27
100	100	2	270	360	0,24
100	100	3	175	360	0,29

Tabel 2.2.6.5.2b. Ringkasan Percobaan 2 instance_tight_space

Populasi	Iterasi	*Rata-rata Objective Akhir*	Min - Maks	Rata-rata Durasi (s)	Iterasi Pertama Mencapai Nilai Akhir (*run* 1, 2, 3)
20	100	351,7	340 - 370	0,09	84, 0, 39
50	100	358,3	345 - 385	0,14	37, 89, 24
100	100	365,0	360 - 375	0,27	77, 9, 100

2.2.6.6. instance_tight_weight

Hasil di bawah berasal dari 15 *file* di output/instance_tight_weight/genetic_algorithm/. *Instance* ini memiliki 14 kendaraan dan 1 kapal. Pada semua *run*, 9 dari 14 kendaraan tidak terangkut pada *state* akhir. Nilai 315 dicapai pada 13 dari 15 *run* dan tidak pernah terlampaui.

Percobaan 1: Populasi sebagai Kontrol

Jumlah populasi tetap 50, banyak iterasi divariasikan (50, 100, 200).

Tabel 2.2.6.6.1. instance_tight_weight *Genetic Algorithm*, Percobaan 1

Populasi	Iterasi	*Run*	*Objective* Awal	*Objective* Akhir	Durasi (s)
50	50	1	245	315	0,07
50	50	2	240	315	0,07
50	50	3	185	315	0,06
50	100	1	245	315	0,21
50	100	2	240	315	0,16
50	100	3	185	315	0,11
50	200	1	245	315	0,23
50	200	2	240	315	0,22
50	200	3	185	315	0,22

Tabel 2.2.6.6.1b. Ringkasan Percobaan 1 instance_tight_weight

Populasi	Iterasi	*Rata-rata Objective*	Min - Maks	Rata-rata Durasi	Iterasi Pertama Mencapai

		Akhir		(s)	Nilai Akhir (*run* 1, 2, 3)
50	50	315,0	315 - 315	0,07	4, 0, 7
50	100	315,0	315 - 315	0,16	4, 0, 7
50	200	315,0	315 - 315	0,23	4, 0, 7

Percobaan 2: Iterasi sebagai Kontrol

Banyak iterasi tetap 100, jumlah populasi divariasikan (20, 50, 100). Konfigurasi populasi 50 dan iterasi 100 adalah *run* yang sama dengan Percobaan 1.

Tabel 2.2.6.6.2. instance_tight_weight *Genetic Algorithm*, Percobaan 2

Populasi	Iterasi	*Run*	*Objective* Awal	*Objective* Akhir	Durasi (s)
20	100	1	245	310	0,05
20	100	2	240	295	0,05
20	100	3	185	315	0,05
50	100	1	245	315	0,21
50	100	2	240	315	0,16
50	100	3	185	315	0,11
100	100	1	245	315	0,25
100	100	2	240	315	0,25
100	100	3	185	315	0,25

Tabel 2.2.6.6.2b. Ringkasan Percobaan 2 instance_tight_weight

Populasi	Iterasi	*Rata-rata Objective Akhir*	Min - Maks	Rata-rata Durasi (s)	Iterasi Pertama Mencapai Nilai Akhir (*run* 1, 2, 3)
20	100	306,7	295 - 315	0,05	36, 0, 39
50	100	315,0	315 - 315	0,16	4, 0, 7
100	100	315,0	315 - 315	0,25	0, 0, 3

2.3. Analisis

1. **Seberapa dekat tiap algoritma dengan *global optima*, dan mengapa?**

   Optimum global diketahui pada empat instance, yaitu Instance Example (495), Small (215), Limited Weight (315), dan Multiship (590). Tiga nilai pertama kami hitung dengan brute force atas semua subset kendaraan yang memenuhi kapasitas berat dan luas geladak. Susunannya pasti ada karena algoritma berhasil mencapainya. Pada Multiship, semua kendaraan muat sehingga optimumnya sama dengan total fee.

   Random Restart mencapai optimum di semua run pada Instance Example, Multiship, dan Limited Weight, tetapi hanya sekali pada Small. Steepest Ascent dan Sideways Move kadang mencapainya. GA mencapai 315 pada 13 dari 15 run Limited Weight, sedangkan 495 dan 215 masing-masing hanya tercapai sekali. SA dengan suhu awal 100 tidak pernah mencapai optimum pada instance satu kapal karena sering terjebak di plateau nilai 0.

Optimum Limited Space dan Multiship+ tidak diketahui. Batas atas Limited Space dari brute force adalah 460, sedangkan nilai terbaik yang kami dapat 445. Nilai terbaik Multiship+ adalah 1045. Keduanya didapat oleh Sideways Move.

2. **Bagaimana perbandingan hasil antar algoritma?**
   Random Restart dan Sideways Move paling sering menghasilkan nilai terbaik. Random Restart unggul di instance kecil karena memulai pencarian 11 kali dari titik yang berbeda sudah cukup untuk menemukan optimum. Sideways Move unggul di instance yang geladaknya sempit, dengan hasil 420 sampai 445 pada Limited Space dan 1000 sampai 1045 pada Multiship+. Langkah sideways menggeser kendaraan tanpa menurunkan nilai sehingga ruang untuk kendaraan lain terbuka.

   Steepest Ascent dan Stochastic berhenti di local maximum dalam 1 sampai 18 iterasi. GA dengan populasi 50 dan 100 iterasi setara dengan Steepest Ascent pada instance kecil, tetapi pada Multiship+ hanya mencapai 830 sampai 905. SA dengan suhu awal 100 memberi hasil terendah.

3. **Bagaimana perbandingan durasi pencarian antar algoritma?**
   Durasi di tabel diukur di perangkat masing-masing anggota, jadi angkanya bisa berbeda dengan file di folder output, tetapi urutannya tetap sama. Pada Multiship+ yang dijalankan di satu perangkat, rata-rata durasi per run adalah 0,10 s untuk SA, 0,96 s untuk Steepest Ascent, 1,18 s untuk Stochastic, 1,33 s untuk GA (populasi 50, 100 iterasi), 10,1 s untuk Random Restart, dan 19,4 s untuk Sideways Move.

   SA paling cepat karena tiap iterasinya hanya menilai satu neighbor. Steepest Ascent, Sideways Move, dan Random Restart menilai semua neighbor di tiap iterasi sehingga makin lambat jika kendaraan makin banyak atau geladak makin besar. Sideways

Move paling lama karena baru berhenti setelah 100 langkah sideways berturut-turut. Durasi GA naik seiring hasil kali populasi dan iterasi.

Jumlah iterasi juga tidak bisa dibandingkan langsung antar algoritma. Satu iterasi Hill-Climbing menilai ratusan sampai ribuan neighbor, sedangkan satu iterasi SA hanya menilai satu. Pada Multiship+, Sideways Move membutuhkan 163, 192, dan 258 iterasi dengan total sideways 147, 177, dan 243. Total itu bisa lebih dari 100 karena hitungan sideways berturut-turut kembali ke 0 setiap kali nilai naik. Random Restart menjalankan 11 pencarian (max_restart = 10), dan iterasi yang dicatat adalah jumlah dari semuanya. Batas 1000 iterasi berlaku untuk tiap pencarian. Pada plotnya, penurunan nilai di setiap batas restart menandai state awal yang baru.

4. **Seberapa konsisten hasil akhir antareksperimen?**
   Random Restart paling konsisten. Ketiga run-nya memberi nilai yang sama pada Instance Example, Multiship, dan Limited Weight. SA dengan suhu awal 100 paling tidak konsisten. Nilai akhirnya berkisar 260 sampai 480 pada Instance Example dan 175 sampai 275 pada Limited Space, bergantung pada apakah pencarian jatuh ke plateau nilai 0.

5. **Bagaimana pengaruh banyak iterasi dan jumlah populasi pada GA?**

   Dengan populasi tetap 50, menaikkan iterasi dari 50 ke 100 menaikkan rata-rata Instance Example dari 431,7 menjadi 478,3. Menaikkan lagi ke 200 iterasi tidak mengubah hasil pada Instance Example, Small, dan Limited Weight karena nilai terbaiknya sudah tercapai sebelum generasi ke-100. Pada instance yang lebih sulit,

200 iterasi masih menambah sedikit, misalnya Multiship+ dari 875 menjadi 883,3.

Dengan iterasi tetap 100, populasi yang lebih besar umumnya menaikkan rata-rata (lihat tabel ringkasan Percobaan 2 tiap instance). Pengecualiannya adalah Multiship+. Pada instance ini, populasi 100 justru menghasilkan rata-rata lebih rendah (863,3) daripada populasi 50 (875). Durasi naik hampir sebanding dengan hasil kali populasi dan iterasi. Karena ada elitism, nilai maksimum populasi tidak pernah turun.

6. **Bagaimana pengaruh suhu, plateau, dan definisi stuck terhadap hasil SA?**
   Semua state yang melanggar constraint bernilai 0. Perpindahan dari state seperti itu ke state lain yang juga melanggar memiliki $\Delta E = 0$ sehingga selalu diterima. Akibatnya, SA bisa terus berjalan di daerah bernilai 0 tanpa pernah memperbaiki best feasible. Pada Limited Space run 1 dan 3, rejected_count bernilai 0, tetapi nilai terbaiknya tidak pernah naik. Jadi, rejected_count dan stuck_count mengukur dua hal yang berbeda. Stagnasi yang panjang tetap dihitung sebagai satu episode sehingga stuck_count yang kecil belum tentu berarti hasilnya baik.

   Kami juga mencoba SA pada Instance Example dengan $T_0 = 10$ dan Tmin = 0,001 agar jumlah iterasinya tetap 917. Hasilnya 475, 480, dan 480, lebih tinggi dan lebih stabil daripada $T_0 = 100$. Angka ini tidak dimasukkan ke tabel utama.

   Hasil akhir SA adalah best feasible, bukan state terakhir. Pada Instance Example run 1 dan 3, state terakhirnya bernilai 0 (last_value = 0), tetapi solusi valid terbaiknya tetap 260 dan 300.

Hal yang sama terjadi pada semua run Small, Limited Space, dan Limited Weight.

3. Kesimpulan dan Saran

3.1. Kesimpulan

Semua algoritma menghasilkan solusi valid pada keenam instance, termasuk instance dengan dua dan tiga kapal. Random Restart paling sering mencapai optimum pada instance yang optimumnya diketahui. Sideways Move memberi hasil terbaik pada instance dengan geladak sempit, tetapi waktunya paling lama. GA stabil dan mendekati optimum pada instance kecil, tetapi tertinggal dari varian Hill-Climbing pada Multiship+. SA dengan suhu awal 100 paling lemah karena aturan nilai 0 membentuk plateau, sedangkan suhu awal 10 memperbaiki hasilnya. Karena SA menyimpan best feasible, solusi akhirnya selalu valid.

3.2. Saran

Suhu awal dan cooling rate SA perlu diuji pada semua instance dengan jumlah iterasi yang sama. Fungsi evaluasi yang memberi nilai berbeda sesuai besar pelanggaran bisa dicoba untuk mengurangi plateau nilai 0, selama solusi akhirnya tetap valid. Untuk Hill-Climbing, max_sideways dan max_restart bisa divariasikan dengan batas waktu atau jumlah evaluasi neighbor yang sama agar perbandingannya adil. Pengujian dengan lebih banyak seed dan pencatatan jumlah evaluasi objective juga akan membantu membedakan pengaruh strategi pencarian dari biaya tiap iterasi. Semua algoritma sebaiknya dijalankan di satu perangkat agar durasinya bisa dibandingkan langsung.

4. Pembagian Tugas

Nama	NIM	Kontribusi
M Rafi' Abdurrahman	18224004	• Pembuatan model • Pembuatan *constraint* • Implementasi *hill climbing* • Eksperimen *hill climbing* • *Proofread* dokumen • Integrasi implementasi
Muhammad Arkan Dhaifullah	18224116	• Implementasi dan revisi SA • Eksperimen dan analisis SA • penyusunan laporan SA
Rachel Elnora Mannuelle Sidanta	18224125	• Implementasi *genetic algorithm* • Eksperimen dan analisis *genetic algorithm* • *Formatting* dokumen laporan • *Proofread* dokumen • Penyesuaian tabel

5. Referensi

- S. Russell dan P. Norvig, *Artificial Intelligence: A Modern Approach*, ed. ke-4, Bab 4.
- M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. https://www.youtube.com/watch?v=oGTa9kGE3P8
- M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. https://www.youtube.com/watch?v=_6rla37QorM
- M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. https://www.youtube.com/watch?v=L0b0CN3Fds8
- M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. https://www.youtube.com/watch?v=AdhG7HnhYrk
- M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. https://www.youtube.com/watch?v=aI41TGo9J9c

- A. Lodi, S. Martello, dan M. Monaci, "Two-dimensional packing problems: A survey," EJOR, vol. 141, no. 2, 2002.
- OptWhiz, "How to Optimally Pack a Container | 2D Knapsack," YouTube. https://www.youtube.com/watch?v=4CfV7A_RWrw
- Computerphile, "The Knapsack Problem & Genetic Algorithms," YouTube. https://www.youtube.com/watch?v=MacVqujSXWE
- GeeksforGeeks, "Introduction to Hill Climbing in Artificial Intelligence." https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/
- Shippeo, "ETA meaning in business: a critical metric for supply chain success." https://www.shippeo.com/resources/explore/blog-newsletter/eta-meaning-in-business-a-critical-metric-for-supply-chain-success

6. Lampiran

6.1. Form Penggunaan *Generative AI*

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM	: 1. M Rafi' Abdurrahman / 18224004 2. Muhammad Arkan Dhaifullah / 18224116 3. Rachel Elnora Mannuelle Sidanta / 18224125
Fakultas/Sekolah	: Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah	: IF3070
Nama Mata Kuliah	: Dasar Inteligensi Artifisial
Judul Tugas/Proposal	: Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial

Menyatakan bahwa ~~saya~~/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya/Tidak*	4

Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/Tidak*	6
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/Tidak*	7
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya/Tidak*	2
Lainnya, Jelaskan: Penggunaan AI dalam kode, terutama dalam debugging dan cek redundansi pada kode	Ya/Tidak*	7

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 9 Oktober 2026

M Rafi' Abdurrahman
18224004

Muhammad Arkan Dhaifullah
18224116

Rachel Elnora M. S.
18224125